Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Kevin Kudakwashe Murera

Tuesday 25th August, 2026

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Kevin Kudakwashe Murera

Day of issue	:	TODO: date of issue
Day of release	:	TODO: date of submission

Reviewer	:	Prof. Dr. Karsten Berns
Supervisor	:	TODO: supervisor

Hereby I declare that I have self-dependently composed the Master Thesis at hand. The sources and additives used have been marked in the text and are exhaustively given in the bibliography.

Tuesday 25th August, 2026 – Kaiserslautern

(Kevin Kudakwashe Murera)

Acknowledgements

TODO: acknowledgements.

Abstract

Hydraulic manipulators are difficult to control precisely, because the actuator response is nonlinear, depends on load and supply pressure, and varies between individual machines. Simulating that response faithfully is possible, but the models that do so are far too slow to be evaluated inside reinforcement learning. Ignoring the hydraulics and commanding joint positions directly avoids that cost, but discards the behaviour that makes the machine difficult to control in the first place.

This thesis investigates how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation, and used for reinforcement-learning control of a verge mowing arm. Two actuator representations are identified from randomised simulation campaigns. The first is a modular chain of an inverse model, which maps a desired cylinder motion to a valve command, and a forward model, which predicts the resulting response. The second is a physics-structured reduced-order model, in which pressure, force and extension propagation remain explicit while learning covers valve realisation and unmodelled loading. Both representations are realised in the deployment simulator and used as the actuation interface for policy learning. A damped least-squares controller provides privileged demonstrations during training and is removed afterwards, so that the distilled policy becomes the sole deployed controller.

The inverse and forward representation is shown to transfer to the deployment simulator without deployment-specific retraining, while the reduced-order model is transferred with a small bounded calibration against the deployment plant. Supervised accuracy is shown to be an unreliable guide to control, because a forward model achieved a high predictive score while learning to ignore the valve command. Excitation design and input ablation are established as the tests that expose this failure. Both representations then support closed-loop terrain following, and the choice of representation is shown to change the control strategy that the policy learns. Online adaptation is examined last, against the robustness already obtained. Plant identity proves recoverable only from deliberate excitation and not from the data the task controller produces, and the trained stacks absorb the plant variation studied here without assistance, which bounds the benefit that conditioning on plant identity can provide. The two measurements that establish this bound, an identifiability envelope and an oracle ceiling read from the true plant factors, are cheap and both precede estimator training. Together they form a decision procedure for whether online adaptation is worth building on a given machine, which this thesis takes as its principal contribution.

Contents

1 Introduction **1**

1.1 The Role of Mechanised Verge Maintenance 1

1.2 Advances in Learning-Based Control for Hydraulic Manipulators 1

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms 2

1.4 Contribution of This Thesis 3

1.5 Thesis Structure 5

2 Fundamentals and Related Work **9**

2.1 Manipulator Actuation and Control Fundamentals 9

2.1.1 Application Context and Implement Carriers 9

2.1.2 The Electro-Hydraulic Actuation Chain 10

2.1.3 High-Fidelity Simulation as a Data Source 10

2.2 Kinematics and Task-Space Tracking 11

2.2.1 Differential Kinematics 11

2.2.2 From Joint Space to Cylinder Space 11

2.3 Model-Based and Conventional Control Approaches 11

2.4 Data-Driven System Identification and Learned Dynamics 12

2.4.1 Learned Forward and Inverse Dynamics 12

2.4.2 Closed-Loop Identifiability 12

2.5 Reduced-Order Modeling 12

2.6 Domain Randomization and Latent Adaptation 13

2.6.1 Robustness Through Randomized Training 13

2.6.2 From Robustness to Adaptation 14

2.7 Reinforcement Learning and Structured Priors 14

2.7.1 Policy Optimization 14

2.7.2 Structured Priors for Policy Learning 15

2.7.3 Reinforcement Learning on Hydraulic Machines 15

2.8 Research Gap and Thesis Positioning 15

3 System Fundamentals and Problem Formulation 18
3.1 Manipulator Platform . . . 18
3.2 Kinematic Structure and Coordinate Mappings . . . 20
3.2.1 Joint Space and Cylinder Space . . . 20
3.2.2 Task Space and the Calibrated Cutter Jacobian . . . 21
3.3 Actuation and Measured State . . . 21
3.3.1 Sensing and Observability . . . 21
3.4 High-Fidelity Simscape Model . . . 23
3.4.1 The Command ROM as Deployment Actuator . . . 23
3.4.2 Mechanical Subsystem . . . 24
3.4.3 Hydraulic Subsystem . . . 24
3.4.4 Validated Scope of the Model . . . 25
3.5 IsaacLab Environment . . . 26
3.5.1 Simulation and Control Rates . . . 26
3.5.2 Environment Configuration Details . . . 26
3.6 The Verge-Following Task . . . 26
3.6.1 Task Definition . . . 26
3.6.2 Task Feasibility and Plant Capability . . . 27
3.7 Control Architecture . . . 27
3.8 Problem Formulation . . . 28
3.9 System-Level Evaluation Criteria . . . 28
3.9.1 The Eight-Gate Promotion Criterion . . . 29
3.9.2 Cylinder and Model Checks . . . 29

4 Data Generation and Preparation 32
4.1 Dataset Objectives and Design Requirements . . . 32
4.2 Randomized Parameter Space of the Simscape Campaign . . . 34
4.2.1 Sampling Design . . . 34
4.2.2 Hydraulic Parameters . . . 34
4.2.3 The Compressibility Axis . . . 35
4.2.4 Mechanical Parameters . . . 35
4.3 Closed-Loop Data Collection . . . 36
4.3.1 The Controller as a Data-Collection Instrument . . . 36
4.4 Open-Loop Excitation Experiments . . . 36
4.4.1 Limitations of the Closed-Loop Batch . . . 36
4.4.2 Design of the Excitation Signal . . . 37
4.4.3 Plant Draws and Acceptance . . . 38
4.5 Combining Closed-Loop and Open-Loop Datasets . . . 38
4.6 Closed-Loop Calibration Data for the Command ROM . . . 39
4.7 The IsaacLab Adaptation Collection . . . 40
4.7.1 Provenance of the Adaptation Data . . . 40
4.7.2 The Physical Factor Vector . . . 40
4.7.3 Domains and Factor Design . . . 40
4.7.4 Collection Procedure . . . 41
4.7.5 The Frozen Dataset . . . 43
4.7.6 Learning Timebase and Hydraulic State . . . 43
4.8 Feature and Target Construction . . . 43
4.8.1 The Actuator Models . . . 43
4.8.2 Observable History of the Adaptation Models . . . 44
4.9 Preprocessing and Data Splits . . . 45
4.10 Dataset Diagnostics . . . 45

5 Learned Models and Online Adaptation 48
5.1 Modelling Overview and Common Notation . . . 48
5.2 Inverse Dynamics Model . . . 50
5.2.1 Paired Control Fine-Tuning . . . 51
5.2.2 Transfer of the Frozen Inverse Model . . . 51
5.3 Forward Dynamics Model . . . 52
5.3.1 Assessment of the Supervised Score . . . 53
5.4 Reduced-Order Model . . . 53
5.4.1 Closed-Loop Calibration Against the Deployment Plant . . . 56
5.5 The Privileged Latent and Its Limits . . . 56
5.6 Arm Adaptation Module . . . 58
5.6.1 Information Content of the Estimated Latent . . . 58
5.7 Latent-Conditioned Inverse-Model Adapter . . . 58
5.7.1 Open-Loop Evidence . . . 59
5.7.2 Closed-Loop Evidence and Its Bound . . . 59
5.7.3 Propagation of the Latent Advantage . . . 60
5.8 Training and Model-Selection Protocol . . . 60

6 Control Framework and Simulation Integration 62
6.1 The DLS Teacher and Its Two Roles . . . 62
6.1.1 Scope and Role of the Controller . . . 62
6.1.2 Privileged Information . . . 62
6.1.3 From Task Error to Cylinder Command . . . 63
6.1.4 Execution Through the Student Interface . . . 64
6.1.5 Teacher Performance . . . 65
6.2 Policy Formulation . . . 65
6.2.1 Policy Definition . . . 65
6.2.2 The Observation Vector . . . 66
6.2.3 The Action and Its Bounds . . . 67
6.2.4 The Reward Function . . . 68
6.2.5 Optimization . . . 70
6.3 Training Protocol and the Distillation Finding . . . 71
6.3.1 Distillation by DAgger . . . 71
6.3.2 Objective Mismatch in Unconstrained Fine-Tuning . . . 73
6.4 Reference-Anchored PPO . . . 74
6.4.1 Critic Warm-Up . . . 74
6.4.2 The Reference Anchor . . . 74
6.4.3 Trust Regions and Rollback . . . 75
6.4.4 The Block Ratchet . . . 75
6.4.5 Theoretical Status . . . 75
6.5 R: Reference Actuator Baseline . . . 77
6.6 IF: Non-Adaptive Learned Baseline . . . 78
6.7 AIF: The Adaptive Stack . . . 79
6.8 IsaacLab Integration and Timing . . . 81
6.9 Constraints and Fair-Comparison Rules . . . 82
6.9.1 Limits and Safeguards . . . 82
6.9.2 Matching Rules for the Three-Stack Comparison . . . 82

7 Experimental Evaluation and Discussion 85
7.1 Experimental Protocol . . . 85
7.1.1 Hypotheses . . . 85
7.1.2 Test Conditions and Experimental Matrix . . . 86
7.1.3 Metrics, Statistics and Fairness . . . 86
7.1.4 What the Three Stacks Can and Cannot Be Compared For . . . 89
7.1.5 Checkpoint Selection and the Scoring Horizon . . . 89
7.2 Closed-Loop Task Performance . . . 90
7.3 Actuator Representation and Dynamic Response . . . 95
7.3.1 Dead Time in the Plant . . . 96
7.3.2 The Chain Compared Across the Three Stacks . . . 97
7.3.3 Step Response to a Commanded Change . . . 99
7.3.4 The Latency Budget of the Closed Loop . . . 100
7.3.5 Control Effort and Command Chatter . . . 100
7.4 Generalisation and Robustness . . . 102
7.4.1 How Much of the Requested Stress Reached the Plant . . . 103
7.4.2 What a Robustness Boundary Would Require . . . 105
7.5 Identifiability of Plant Variation . . . 106
7.5.1 Excitation Design and the Identifiability Envelope . . . 106
7.5.2 Closed-Loop Data Against Excitation Data . . . 108
7.5.3 Information Content of the Adaptive Latent . . . 108
7.6 Online Adaptation . . . 110
7.6.1 The Model-Level Gain and the Frozen Inverse Model . . . 110
7.6.2 Estimated Against Privileged Latents . . . 112
7.6.3 Response to a Change of Plant . . . 112
7.6.4 The Benefit Does Not Scale With Plant Deviation . . . 113
7.6.5 Response to a Mid-Episode Plant Change . . . 115
7.6.6 An Oracle Reference for Plant-Identity Conditioning . . . 118
7.7 Why the Model-Level Gain Does Not Reach the Controller . . . 120
7.7.1 The Ceiling Bounds the Estimator Out of the Question . . . 120
7.7.2 The Baseline Is Not Plant-Blind . . . 121
7.7.3 The Two Consumers of the Latent . . . 121
7.7.4 Latent Bandwidth on the Policy Path . . . 121
7.7.5 The Loop Is Slower Than Anything the Latent Changes . . . 123
7.8 Integrated Discussion . . . 123
7.9 Threats to Validity . . . 126

8 Conclusion and Future Work 130
8.1 Summary of Findings . . . 130
8.2 Design Implications . . . 133
8.3 Contributions . . . 135
8.4 Limitations . . . 136
8.5 Future Work . . . 137

A System Parameters and Coordinate Mappings **139**
A.1 Rates . . . 139
A.2 Command-Path Limits . . . 140
A.3 Course and Episode Geometry . . . 140

B Dataset Manifests and Factor Ranges **141**
B.1 Trajectories . . . 141
B.2 Execution and Logging . . . 141
B.3 Parameter Ranges and Dataset Audit . . . 142
B.4 The Frozen Adaptation Collection . . . 142

C Model Architectures, Normalizers and Checkpoint Hashes **144**
C.1 The Inverse and Forward Models . . . 144
C.2 The Adaptation Module . . . 145

D PPO and IsaacLab Configuration **146**
D.1 The Observation Vector . . . 146
D.2 The Action . . . 146
D.3 The Reward . . . 146
D.4 PPO Hyperparameters . . . 148

E Supplementary Results **149**
E.1 Per-Family Results on the Reference Mesh . . . 149
E.2 The Latent Substitution Arms . . . 150
E.3 Reproduction . . . 150

Bibliography **151**

1. Introduction

1.1 The Role of Mechanised Verge Maintenance

Road verges are cut because they are infrastructure. Vegetation left to grow closes sight lines at junctions and bends, obscures signs and barriers, blocks the drainage channels that keep water off the carriageway, and encroaches on the running surface itself. A national road network carries tens of thousands of kilometres of verge, and each kilometre is cut several times in a growing season, so the work is continuous, repetitive and performed almost entirely by machine.

It is also performed from a moving vehicle. Stopping to cut each section would require closing the lane, so a carrier drives the road at a few kilometres per hour while a boom-mounted mowing head works the verge alongside it. The head must hold a cut height of a few centimetres over ground that rises and falls continuously, follow a grass line that curves with the road, and do so without striking the surface hard enough to damage the tool or lift the machine.

The demands on the manipulator follow from that. The machine studied here, shown in Figure 1.1, carries a heavy rotating tool at the end of a long boom, works in intermittent contact with the ground, and must place that tool accurately while its own base is in motion. Hydraulic actuation is the standard answer at this scale: it supplies the force density and the tolerance to shock loading that electric drives do not, and it does so with components rugged enough for continuous outdoor operation. Precision, however, is not what hydraulics are good at, and that tension is the subject of this thesis.

1.2 Advances in Learning-Based Control for Hydraulic Manipulators

Two developments have changed what is achievable on machines of this kind.

The first is that policy optimisation in simulation has become practical for contact-rich manipulation. GPU-parallel physics engines run thousands of environments simultaneously, which makes reinforcement learning feasible on tasks that would take years of real-machine

experience, and randomising the simulator across a family of plants produces policies that transfer without being tuned to one machine. The second is that the actuator itself can be learned. Rather than deriving valve and cylinder behaviour from first principles and fitting its parameters, a network trained on recorded motion can represent the command-to-motion relationship directly, including the nonlinearities that analytical models approximate away.

Combining the two is attractive and not straightforward. A learned actuator model must be cheap enough to evaluate inside the training loop, which rules out the high-fidelity models used for offline analysis, and it must remain valid across the family of machines the policy will meet. This has motivated a line of work in which a compact code describing the current plant is inferred from recent interaction history and supplied to the controller, so that a single policy can serve a family rather than a single machine. Such methods report substantial gains in legged locomotion and in manipulation, and they are the direct antecedent of the architecture examined here.



Figure 1.1: MULAG FME 500 front mower on a Unimog during roadside verge maintenance. Source: [MULAG Fahrzeugwerk 26].

1.3 Challenges of Precise Tracking with Hydraulic Mowing Arms

A valve command does not produce a proportional cylinder motion. The relationship is nonlinear, depends on load and supply pressure, and is subject to dead time between the command and any measurable response. It also differs between machines: seal condition, fluid temperature, hose compliance and the attachment fitted all change it, so two machines of the same model respond differently and one machine responds differently across a season. A controller that inverts only the geometry therefore leaves the largest source of tracking error unaddressed.

That difficulty admits two responses, and they are routinely conflated. The first is robustness: train across a randomised family so that a single fixed model is adequate everywhere. It costs nothing at run time and gives up the difference between the family and the individual. The second is adaptation: infer which member of the family is present from the interaction history and condition the model or the policy on that estimate. It promises to recover that difference, at the cost of an estimator that must work from observable signals alone, since a deployed machine has no access to its own plant parameters.

The second response is only worth pursuing under conditions that are rarely checked before the work is done. Plant identity must be recoverable from the data the machine actually produces, which depends on how the machine is being driven while that data is collected. And the difference between knowing the plant and not knowing it must be large enough to matter once the estimate reaches a feedback controller. Neither condition is guaranteed, both are specific to the task, and both can be measured before an estimator is built. Establishing those measurements, and applying them to a hydraulic mowing arm, is the problem this thesis addresses.

1.4 Contribution of This Thesis

The difficulty described in Section 1.3 is a modelling problem before it is a control problem. A hydraulic actuator can be simulated accurately, but the models that do so are too slow to sit inside a reinforcement-learning loop that requires millions of environment steps. The alternative usually taken is to discard the hydraulics and command joint positions directly, which removes the load and pressure dependence that makes the machine difficult to control. What is needed is a representation that is cheap enough to train against and faithful enough that the resulting policy is controlling something close to the real actuator.

Learned actuator models and latent-conditioned adaptation are both established techniques, and Section 1.2 described the progress that makes them practical on machines of this class. What remains unclear is how such models should be identified so that they retain the command dependence a controller needs, whether they transfer between simulators without being retrained, and whether online adaptation adds anything once a policy has been trained against them. This thesis therefore treats representation and transfer as the primary questions, and treats online adaptation as the last of them rather than the first. The research statement of the thesis is as follows:

Comparison notation.

For readability, the three policy–actuator stacks carry both a short label and a full name, and the two are used interchangeably throughout this thesis: *R* is R, *IF* is IF, and *AIF* is AIF. The short labels are preferred where several stacks appear in one sentence and the full names where the actuator path itself is the subject. R uses the command-conditioned reduced-order model; IF uses the frozen inverse/forward pair; and AIF augments IF with online adaptation. The labels name the complete stacks, not just their actuator models.

To determine how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation improves on the robustness those learned models already provide.

The statement decomposes into six research questions, which are answered in the order given. The first three concern how the actuator is represented, the fourth concerns control through that representation, and the last two concern what remains for adaptation once the first four are settled. The six questions are:

1. How can data from a high-fidelity Simscape model be used to learn computationally tractable representations of hydraulic actuation while preserving the command dependence a controller requires?
2. To what extent do actuator models identified from Simscape transfer to an IsaacLab manipulator without deployment-specific retraining?
3. Can a physics-structured reduced-order hydraulic model supply dynamics that are accurate and stable enough for GPU-parallel policy training?
4. Can a policy trained from a privileged classical teacher become the sole deployed controller while acting through learned actuator dynamics?
5. How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?
6. Where plant variation survives that robustness, is it identifiable from recent observations, and does online adaptation by latent identification improve closed-loop performance?

The first is answered in Chapters 4 and 5, the second and third in Chapter 5, the fourth in Chapter 6, and the last two in Chapter 7.

RQ6 is the question named in the title, and it is deliberately placed last. It is only worth asking once the preceding questions have established that the learned models transfer, that a policy can control through them, and how much plant variation the resulting closed loop already absorbs unaided. A small answer to RQ6 is then a result about the system rather than a failure of the method, because accurate models and a robust policy leave less for adaptation to recover. The work rests on the following contributions:

Transferable Hydraulic Actuator Learning: A pipeline that identifies hydraulic actuation from high-fidelity Simscape data and transfers it into IsaacLab through two representations, a modular inverse and forward chain and a physics-structured reduced-order model. The inverse model is trained on Simscape data and evaluated on deployment runs drawn entirely outside that training distribution, where it reproduces most of the valve-command variance with an accuracy that is nearly flat across plant domains. It requires no deployment-specific retraining and remains frozen in every downstream stack.

The reduced-order model is transferred differently and the difference is stated wherever it matters: it receives a small bounded calibration against the deployment plant, which is why it serves as a reference condition rather than as an independent competitor to the inverse and forward pair.

Control-Relevant Identification and Validation: A demonstration that predictive accuracy can conceal the loss of exactly the property a controller needs, together with the procedure that detects it. The first forward model reached a near-perfect supervised score while learning to ignore the valve command and reconstruct motion from the state history instead. The validation loss did not show this; ablating the command did. Redesigning the excitation and retraining recovered genuine command sensitivity, and the pairing of excitation design with input ablation is established here as a required test for learned hydraulic models intended for control rather than for prediction.

Reduced-Order Actuator Model: A command-conditioned grey-box actuator in which pressure, force, velocity and extension propagation remain explicit while learning covers valve realisation, unmodelled loading and bounded residual effects. It is accurate enough to serve as the deployment plant, stable in recursive rollout, and cheap enough to evaluate inside GPU-parallel policy training. Zeroing or shuffling its command collapses its extension and velocity accuracy, which establishes that the command authority is real rather than inferred from context.

Policy-Only Control Through Learned Hydraulics: A damped least-squares controller supplies privileged demonstrations during training and is then removed, so that the distilled policy emits desired cylinder motion directly into a learned actuator representation and constitutes the entire deployed controller. Both representations support closed-loop terrain following in this configuration, although two learned models placed in series could have compounded their errors.

A Decision Procedure for Online Adaptation: A matched evaluation protocol that scores the resulting stacks on tracking, control effort, latency, disturbance rejection and sensitivity to the magnitude of plant variation, together with a procedure that decides in advance whether online plant conditioning will repay the effort. The first of its two measurements establishes whether plant identity is present in the data the machine produces. The second supplies the true plant parameters through a privileged encoder and measures what the closed loop does with them. Both precede estimator training, and either can end the effort before it is incurred. This converts an open-ended engineering effort into a measurement, and it is the contribution of this thesis most likely to transfer to other machines and other latent-conditioned designs.

The components on which these contributions rest, namely the causal plant estimator, the bounded adaptation path and the eight-gate evaluation harness, are described where they are used. They are technical enablers rather than separate claims.

1.5 Thesis Structure

This thesis investigates how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, grounding its investigation in the identifiability of

the actuator models and in the robustness of the controllers built on them. The document is organized as follows:

Chapter 2 establishes the background against which the remainder is read. It introduces the machine class and its electro-hydraulic actuation, recalls the kinematic tools underlying the classical controller, and reviews how conventional model-based control handles hydraulic uncertainty. It then treats the learning methods this work builds on, namely data-driven dynamics models and the pitfalls of identifying them from closed-loop data, reduced-order and grey-box modelling, domain randomisation and latent adaptation, and policy optimisation with structured priors. The chapter closes by stating the gap that remains and the position of the three control stacks examined here.

Chapter 3 analyses the platform, using the MULAG FME 500 mounted on a Unimog carrier as the case study. It describes the rigid bodies and the serial chain they form, the coordinate mappings between task space and cylinder space, and the closed kinematic loops introduced by the cylinder mounts. It specifies what the controller can observe, separating measured quantities from quantities produced by a model, and presents the high-fidelity Simscape model from which the actuator data derives. The chapter also states the task, the plant capability limits and the acceptance criteria against which every later result is scored.

Chapter 4 presents the data generation strategy and answers the first research question. It describes the excitation design, the randomised factor set defining the family of plants, and the procedures that keep training, validation and reserved test configurations separate. It then reports the identifiability audit applied to the resulting collection, including the measured finding that excitation gathered under a task controller does not separate plants, and the open-loop collection that repairs this.

Chapter 5 develops the learned models and answers the first three research questions. It presents the inverse model that maps a desired cylinder motion to a valve command, the forward model that predicts the response, and the evidence that supervised accuracy alone does not establish that either model retains command authority. It then presents the command-conditioned reduced-order model that serves as the deployment plant inside GPU-parallel training. The chapter closes with the material required for the final research question, namely the privileged latent and its measured limits, the causal estimator that infers a deployable substitute from observable history, and the bounded adapter through which that estimate reaches the actuator path.

Chapter 6 describes the control framework and its integration with the simulator, and answers the fourth research question. It presents the damped least-squares teacher used for demonstration and data collection, the observation and action interfaces shared by all three stacks, the distillation of the teacher into a student policy, and the reference-anchored fine-tuning that makes further policy optimisation admissible without discarding the distilled behaviour. It then assembles the three deployed control stacks and states the rules under which they may be compared.

Chapter 7 reports the experimental evaluation and answers the last two research questions. It establishes the excitation envelope within which plant identity is recoverable, the sufficiency of the frozen inverse model, and what the inferred latent encodes. It measures latency through the actuation chain and the response of each stack to a commanded change, then compares the three deployed controllers on closed-loop tracking and control effort. It

closes by measuring the disturbance envelope of each controller, an oracle reference for what plant-identity conditioning contributes within this architecture, and the sensitivity of each stack to the magnitude of plant variation.

Chapter 8 concludes the thesis. It summarises what the work establishes and what it does not, states the design implications that follow from the measured results, records the limitations of the evidence, and sets out the future work, which centres on transferring the learned actuator models and the distilled policy to the physical machine and fine-tuning them on data collected from it.

Appendix A provides the system parameters and the coordinate mappings between task space, joint space and cylinder space referred to in Chapter 3. **Appendix B** lists the dataset manifests and the factor ranges that define the family of plants used in Chapters 4 and 7. **Appendix C** records the model architectures, the normalisers and the checkpoint hashes for every learned model reported in Chapter 5. **Appendix D** documents the PPO and IsaacLab configuration used for the training described in Chapter 6. **Appendix E** collects supplementary results supporting Chapter 7.

2. Fundamentals and Related Work

Chapter 1 argued that precise task-space tracking with a hydraulic mowing arm is limited less by kinematics than by the actuator. The path from a valve command to actual cylinder motion is nonlinear and load dependent, and it differs from one machine, season, or attachment to the next. This chapter collects the background needed to judge that argument and to understand the design decisions made later in the thesis. Section 2.1 introduces the machine class and its electro-hydraulic actuation. Section 2.2 recalls the kinematic tools behind the classical controller, and Section 2.3 reviews how conventional model-based control deals with hydraulic uncertainty. The remaining sections cover the learning methods this work builds on. Section 2.4 treats data-driven dynamics models and the pitfalls of identifying them from closed-loop data. Section 2.5 covers reduced-order and grey-box modeling, Section 2.6 domain randomization and latent adaptation, and Section 2.7 policy optimization with structured priors. Section 2.8 then states the gap that remains and how the three control stacks of this thesis (R, IF, and AIF) address it. As defined in Chapter 1, these abbreviate PPO–ROM, PPO–ID–FD, and PPO–(AAM)–ID–FD, respectively. Only material that is needed for the later chapters is included. The concrete system definition and problem formulation follow in Chapter 3.

2.1 Manipulator Actuation and Control Fundamentals

2.1.1 Application Context and Implement Carriers

The MULAG FME 500 mounts a multi-link hydraulic boom and a 1.2 m flail head on a Unimog carrier [MULAG Fahrzeugwerk 26]. It reaches about 6.5 m laterally, below road level and over obstacles, while an operator adjusts cutter height, tilt and lateral position as the vehicle moves. Automating this continuous tracking task is the practical motivation of the thesis.

Hydraulic cylinders suit the required force, shock tolerance and existing vehicle power supply, but their command gain changes strongly with load and configuration. This trade-off between force density and precision is central to hydraulic manipulator control [Mattila 17].

2.1.2 The Electro-Hydraulic Actuation Chain

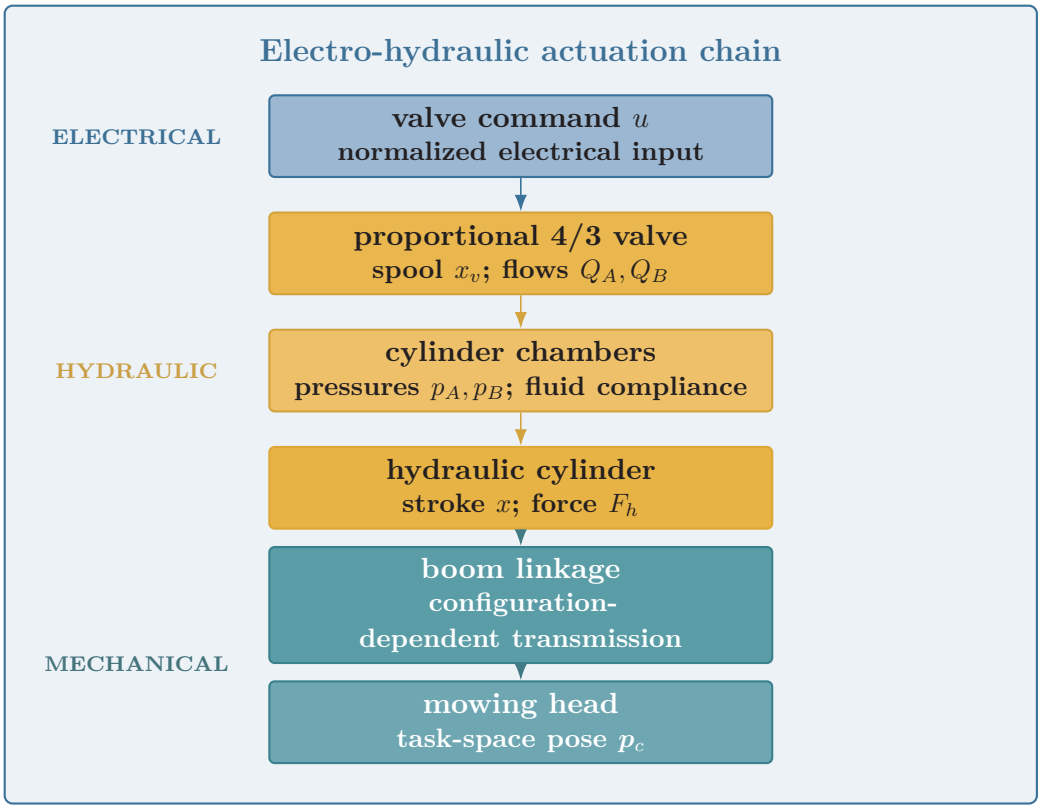


Figure 2.1: The electro-hydraulic actuation chain from valve command to mowing-head motion. The block colors distinguish the electrical, hydraulic and mechanical stages; each subtitle identifies the principal quantity passed to the next stage.

In Figure 2.1, an electrical command moves a proportional valve spool and meters flow to the cylinder chambers. The orifice law [Merritt 67] makes flow depend on spool position and the square root of the pressure margin. Valve authority is therefore load-dependent and directional; overlap creates a dead zone near null and the spool saturates at large commands [Jelali 03].

Compressibility, changing chamber volume, relief limits, leakage and seal friction further separate flow from motion. Several of these effects are non-smooth, while bulk modulus, leakage and friction drift with temperature and wear. The mechanical linkage then converts cylinder translation through configuration-dependent lever arms. Chapter 5 consequently models the hydraulic realization in cylinder space, before kinematic effects are mixed into cutter error.

2.1.3 High-Fidelity Simulation as a Data Source

The component-level Simscape plant [The MathWorks, Inc. 24a] resolves the stiff hydraulic dynamics but runs one machine at small integration steps. Policy optimization instead benefits from thousands of GPU-parallel plants [Makoviychuk 21]. The detailed model is therefore used offline to generate identification data; compact actuator models carry its behavior into parallel training, as summarized in Figure 2.2.

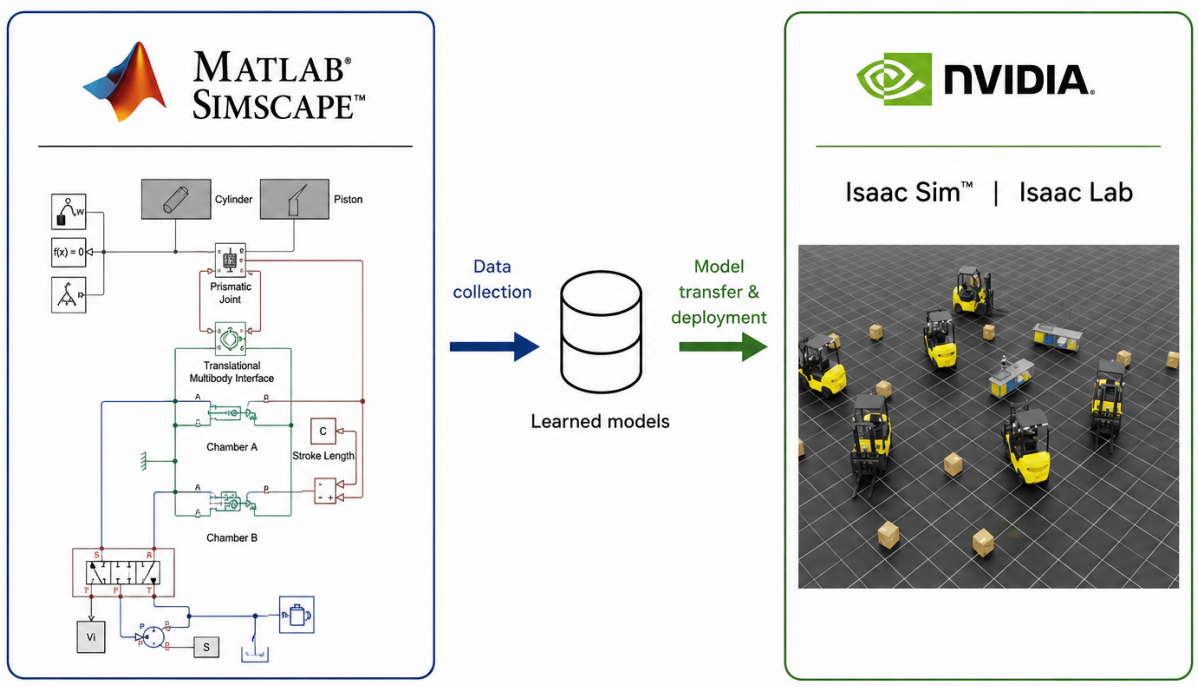


Figure 2.2: Simscape supplies high-fidelity hydraulic data; compact actuator models transfer that behavior to GPU-parallel training in IsaacLab. Product images: MathWorks and NVIDIA.

2.2 Kinematics and Task-Space Tracking

2.2.1 Differential Kinematics

The cutter path is specified in vehicle coordinates, so the manipulator Jacobian maps joint rates to the required tool velocity [Siciliano 09]. Near singular configurations the ordinary pseudo-inverse produces excessive commands. The damped least-squares inverse of Wampler [Wampler 86] and Nakamura and Hanafusa [Nakamura 86] trades some local accuracy for bounded commands and is retained as the classical controller.

2.2.2 From Joint Space to Cylinder Space

The actuated coordinates are cylinder lengths, and their nonlinear cylinder-to-joint map has a configuration-dependent lever arm. A passive head joint further limits independently controllable directions. The calibrated cutter Jacobian in Chapter 3 composes these mappings. It still assumes that commanded cylinder motion is realized; the learned actuator models address that remaining gap.

2.3 Model-Based and Conventional Control Approaches

Industrial systems commonly map joystick input directly to valve current. Fixed-gain position–velocity or position–pressure cascades improve on this practice [Jelali 03], but tuning across large changes in valve gain and linkage leverage produces either weak response or oscillation.

Nonlinear model-based control instead cancels known rigid-body dynamics [Slotine 91], estimates uncertain parameters online [Åström 95], or bounds their effect. Yang et al. [Yang 26], for example, combine state constraints with explicit valve dead-band inversion. Such methods can track accurately, but depend on valve, friction and inertia models that are costly to maintain on a commercial implement [Mattila 17].

This thesis therefore retains reliable structure and learns the uncertain part. The DLS kinematic controller acts as teacher and data-collection controller; learning covers the mapping from cylinder command to realized motion and the policy that issues that command (Section 2.7).

2.4 Data-Driven System Identification and Learned Dynamics

2.4.1 Learned Forward and Inverse Dynamics

Robot model learning distinguishes forward models, which predict the next state from state and command, from inverse models, which predict the command for a desired transition [Nguyen-Tuong 11]. Section 5.1 fixes both interfaces. One-step fit is insufficient when a forward model is recursively rolled out, because small errors compound [Nagabandi 18]. The models here are therefore trained with multi-step objectives. Probabilistic ensembles represent growing rollout uncertainty [Chua 18]; recurrent architectures instead retain unobserved pressure and valve history [Hochreiter 97, Cho 14], as used by the reduced-order model.

2.4.2 Closed-Loop Identifiability

Informative identification requires inputs that excite the plant independently of feedback [Ljung 99, Isermann 11]. In closed-loop data, command and state history are correlated; a flexible predictor can score well by ignoring the command, and validation from the same distribution will not reveal the shortcut [Schoukens 19].

That failure occurred here: a closed-loop forward model reached one-step $R^2 \approx 0.999$ yet showed almost no command dependence under ablation. Chapter 4 therefore uses open-loop valve excitation, and every learned model is checked with zeroed and shuffled commands. Prediction score alone is not treated as evidence of control authority.

2.5 Reduced-Order Modeling

Reduced-order models retain dominant behavior at lower cost [Benner 15]. Here the full Simscape plant is reduced to per-cylinder extension, velocity and pressure difference. The construction is grey-box: known structure is embedded so that data fits what remains unknown [Karniadakis 21]. As in the structured dynamics network of Lutter et al. [Lutter 19], explicit physical layers improve interpretability. The hydraulic ROM retains the orifice, pressure and integration structure; learned terms represent valve realization, configuration-dependent load and bounded residuals.

Hwangbo et al. [Hwangbo 19] combined analytic rigid-body mechanics with a learned actuator response, a pattern later scaled in GPU-parallel simulation [Rudin 22]. Yamamori

et al. [Yamamori 26] instead shaped the physical actuator to match the simulation. A commercial valve and pump cannot be redesigned in that way, so this thesis learns the hydraulic realization while PhysX retains the rigid-body mechanics (Figure 2.3).

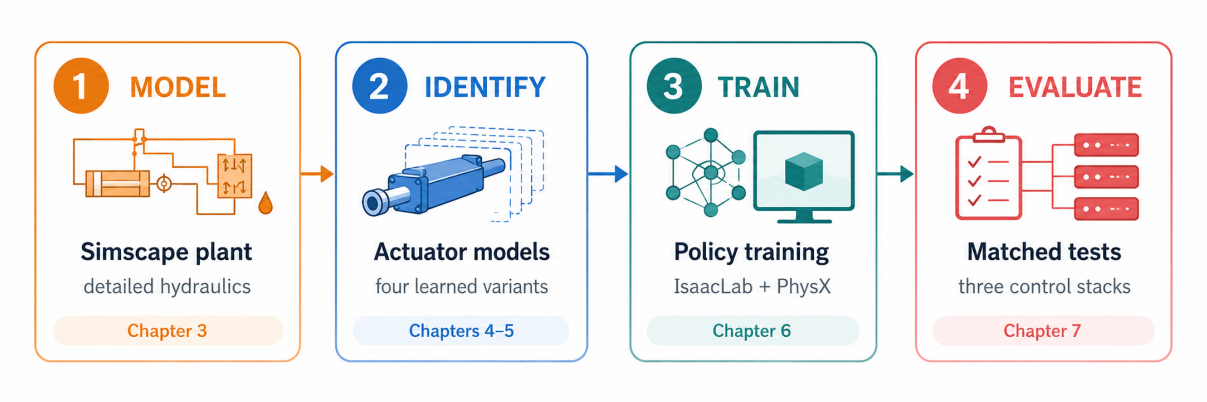


Figure 2.3: Workflow from the hydraulic plant model to matched evaluation. The separation between rigid-body physics and learned actuation follows the actuator-network approach in [Hwangbo 19].

2.6 Domain Randomization and Latent Adaptation

2.6.1 Robustness Through Randomized Training

A model trained on one simulated plant inherits its parameters as assumptions. Domain randomization instead trains across a family, first for visual appearance [Tobin 17] and then for dynamics such as mass, friction, gain and latency [Peng 18]. The hydraulic family in Chapter 4 varies bulk modulus, valve gain, leakage, friction and payload, producing robustness to physically plausible plant variation.

2.6.2 From Robustness to Adaptation

Figure 2.4 contrasts a policy trained for one robust response across the plant family with an adaptive stack that conditions its response on the current plant.

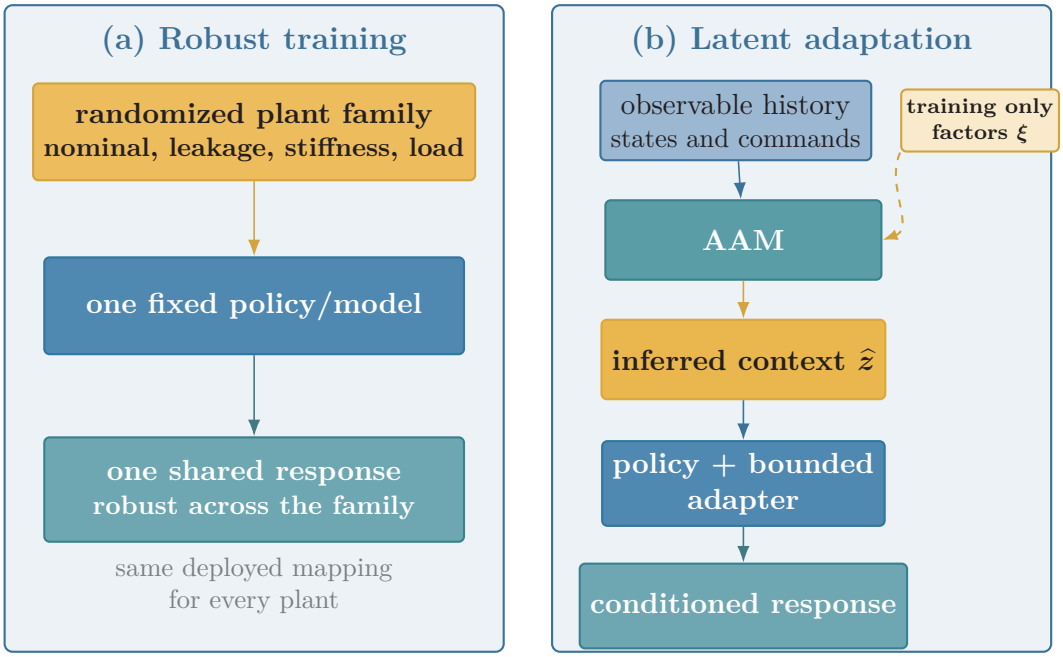


Figure 2.4: Comparison of robust training and latent adaptation. Robust training applies one fixed response across the randomized plant family. The adaptive stack uses privileged factors only to supervise training; during deployment, the AAM infers context from observable history and conditions the downstream response.

Teacher–student methods encode known simulation factors into a privileged latent and train an estimator to recover it from observable history. Lee et al. [Lee 20] used this construction for hidden terrain properties; Kumar et al. [Kumar 21] developed rapid motor adaptation. Unlike classical online parameter estimation [Åström 95], the learned latent need not have a direct physical interpretation. It can also condition a dynamics model rather than a policy, as in the contextual neural ODE of Yu et al. [Yu 26].

The AAM in Chapter 5 estimates a two-dimensional latent from one second of causal observable history. In AIF, that estimate enters the actor observation and a bounded adapter around frozen inverse dynamics; it does not condition the ROM, forward model or PhysX plant. The experiments keep two distinctions explicit: randomized training is robust but not automatically adaptive, and an informative privileged latent does not prove that an inferred latent improves closed-loop control.

2.7 Reinforcement Learning and Structured Priors

2.7.1 Policy Optimization

Reinforcement learning optimizes a policy from observations to actions for expected cumulative reward [Sutton 18]. This thesis uses PPO [Schulman 17] with generalized

advantage estimation [Schulman 16]; Section 6.2.5 gives the exact objective. GPU-parallel simulation supplies the required throughput [Makoviychuk 21, Rudin 22], which is why compact actuator surrogates replace the full hydraulic plant during training.

2.7.2 Structured Priors for Policy Learning

Learning from scratch would discard reliable kinematics. Residual RL instead adds a bounded learned correction to a controller at runtime [Johannink 19, Silver 18]. Policy distillation moves the prior to training: a privileged DLS teacher supplies demonstrations, then the student acts alone [Ross 11]. This thesis distils the teacher and then anchors PPO updates to the frozen student (Section 6.4). The deployed actor remains the sole controller.

The idea parallels real-world fine-tuning that retains a simulation-trained value function [Yin 25], but here the retained object is the reference policy. Anchoring was introduced because unconstrained PPO improved reward while degrading the physical tracking criterion (Section 6.3).

2.7.3 Reinforcement Learning on Hydraulic Machines

Deep RL has reached full-size hydraulic excavators in simulation-to-machine tracking [Egli 20] and general automation [Egli 22]. Spinelli et al. [Spinelli 24] extended it to commercial material handlers with underactuated tools. Maitan et al. [Maitan 26] learned force control on a physical cylinder, using a contraction-metric filter to bound exploration. These studies establish scale and feasibility, but do not combine a randomized plant family, online actuator adaptation, a privileged kinematic teacher and a matched comparison of actuator representations.

A policy can also compensate for a poor actuator model. Chapter 7 therefore reports rollout error, command magnitude and smoothness, and cylinder-level residuals alongside task tracking. Equal task error alone is not evidence of equal model quality.

2.8 Research Gap and Thesis Positioning

The literature leaves a specific combination unresolved. Hydraulic control provides structure but requires maintained plant models; learned dynamics replace derivation with data but depend on identifiable excitation; actuator surrogates and latent adaptation are established mainly for electric legged robots; and hydraulic RL studies do not isolate actuator representation under a matched protocol. Table 2.1 summarizes this position.

The five criteria are: (I) learned hydraulic actuation, (II) data audited for causal input identifiability, (III) online latent adaptation, (IV) a structured controller retained as a bounded prior, and (V) a matched actuator comparison. Parenthesized marks denote partial or implicit coverage.

Table 2.1: Positioning against the reviewed lines of work.

Line of work	I	II	III	IV	V
Model-based hydraulic control [Mattila 17, Jelali 03]	—	(✓)	(✓)	✓	—
Actuator networks for legged robots [Hwangbo 19, Rudin 22, Yamamori 26]	—	✓	—	—	—
RL for hydraulic machines [Egli 20, Egli 22, Spinelli 24, Maitan 26]	✓	(✓)	—	—	—
Latent online adaptation [Lee 20, Kumar 21, Yu 26, Liu 25, Hu 26]	—	—	✓	—	—
Residual RL and policy distillation [Johannink 19, Silver 18, Ross 11, Yin 25]	—	—	—	✓	—
This thesis (R / IF / AIF)	✓	✓	(✓)	✓	✓

The partial mark in column III reflects the results: the latent is recovered causally, but its closed-loop benefit is less conclusive. The contribution is therefore the complete experimental chain—identifiable cylinder data, physics-structured models fast enough for parallel training, online latent estimation, and a common evaluation protocol—rather than an unqualified claim that adaptation always improves control. The three tested stacks are:

R. A command-conditioned ROM realizes the policy's desired cylinder motion. It is the reference and the only actuator calibrated to the deployment plant (Section 6.9).

IF. The same policy interface drives the frozen inverse model followed by the strict forward model, forming the robust learned baseline.

AIF. The AAM infers a dynamics latent from causal observable history. It conditions the policy observation and a bounded adapter around the frozen inverse model, ahead of the same forward model and PhysX.

All stacks share the task, action contract, terrain, trajectories, deployment factors and acceptance gates. Their training runs are not fully matched, and the ROM is calibrated to the deployment plant; causal architectural claims therefore rely on within-checkpoint ablations rather than the across-stack contrast (Section 6.9). Chapter 3 now grounds this comparison in the concrete machine.

3. System Fundamentals and Problem Formulation

Chapter 2 established the control and modeling concepts used in this thesis. This chapter converts that background into a fixed system definition for the remaining work. It specifies the physical mechanism, coordinate mappings, available measurements, actuator models, task geometry, and evaluation interfaces before the controllers are compared. Keeping these definitions together prevents later chapters from changing the plant, observation set, or success criteria when interpreting results.

The physical machine and its two simulation representations are distinguished explicitly. Simscape is the high-fidelity hydraulic source used for actuator identification. IsaacLab/PhysX is the parallel environment used for policy training, adaptation-data collection, and closed-loop evaluation. Both use the cylinder ordering, kinematics, action contract, and cutter-tip task variables defined here, but they are not assumed to be identical plants.

Sections 3.1–3.3 define the mechanism, coordinate spaces, and signals. Sections 3.4 and 3.5 describe the two simulation models. The remaining sections define the verge-following task, the R, IF, and AIF control stacks, the control objective, and the acceptance gates used in the experiments.

3.1 Manipulator Platform

The MULAG FME 500 is modeled as a serial-link mechanism with five revolute joints. Four hydraulic cylinders actuate the slew, two boom, and head-pivot joints. The *Messerkopf* rotation has no independent actuator and is locked at the mowing configuration; it is therefore not a controlled degree of freedom. The cylinder order used throughout the thesis is slew (*Drehzapfen*), boom I (*Ausleger I*), boom II (*Ausleger II*), and head pivot (*Messerkopf-Schwenk*).

Table 3.1 gives the cylinder stroke ranges used in the training data and the fixed limit of the *Messerkopf* joint. Task-space errors are measured at the cutter tip, which lies 0.79 m below the *Messerkopf* body origin.

Table 3.1: Cylinder stroke ranges in the training data (Chapter 4) and the fixed *Messerkopf* joint limit used for the mowing task.

i	Joint	Function	Limit
1	*Drehzapfen*	slew	0.018–0.493 m
2	*Ausleger I*	boom I	0.275–0.791 m
3	*Ausleger II*	boom II	0.309–0.857 m
4	*Messerkopf-Schwenk*	head pivot	0.269–0.689 m
5	*Messerkopf*	head rotation (locked)	$q_{5,\min} = q_{5,\max} = q_{5,\text{lock}}$

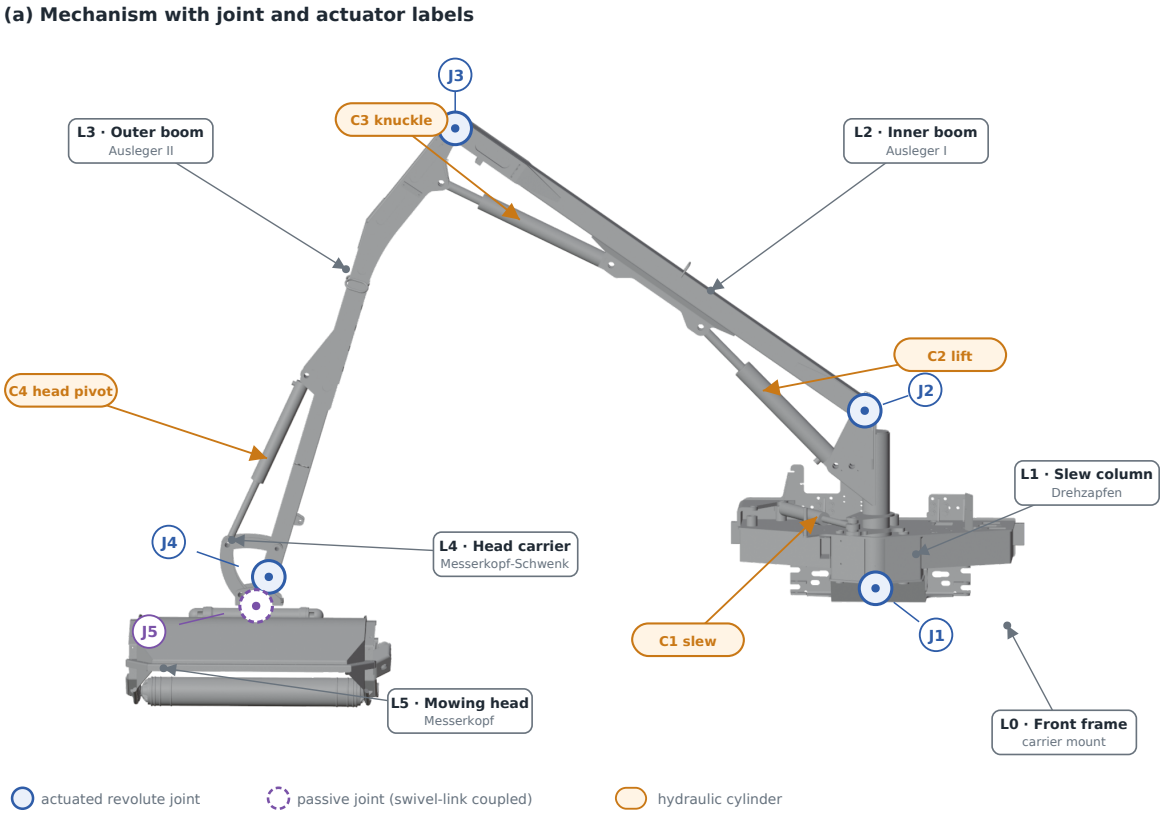


Figure 3.1: Annotated CAD view of the MULAG FME 500. Links L0–L5, joints J1–J5, and the four hydraulic cylinders C1–C4 are identified; J5 denotes the *Messerkopf* rotation, which is locked in the mowing configuration used in this thesis.

Figure 3.1 identifies the links, joints, and four actuator channels used throughout the model.

Figure 3.2 separates the rigid bodies that form the assembled mechanism. The catalog makes the L0–L5 body assignment explicit and shows the representative barrel–rod geometry used for cylinders C1–C4.

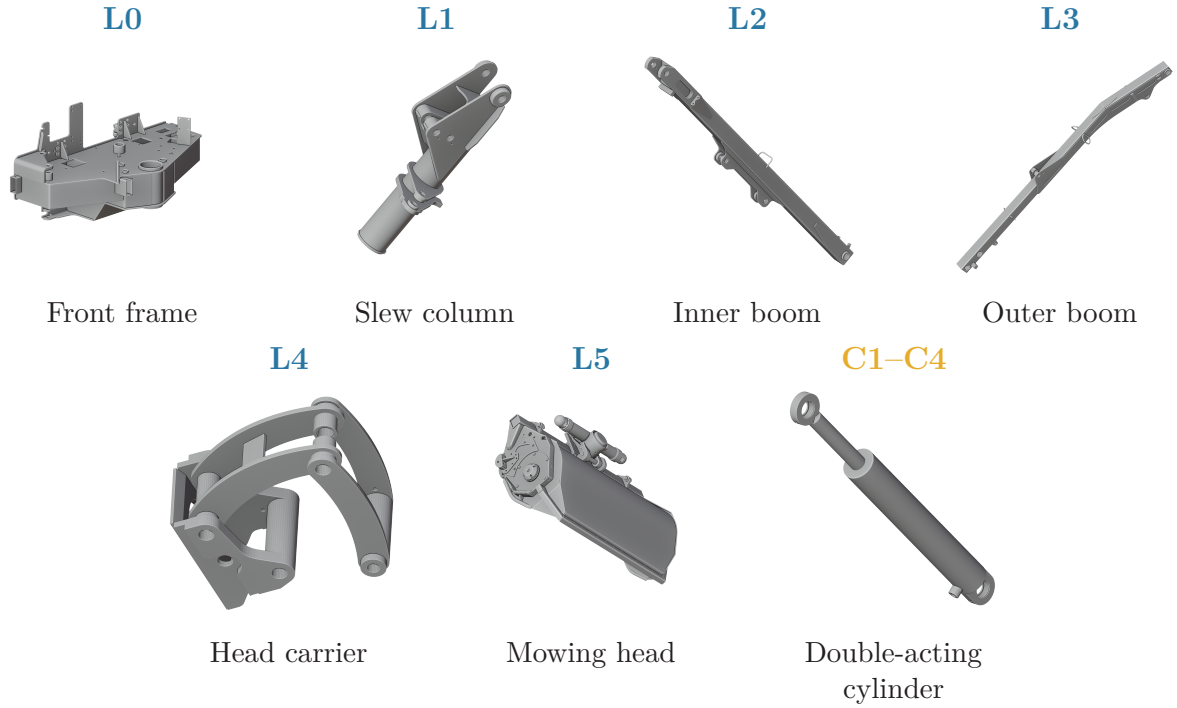


Figure 3.2: Individual CAD-derived bodies of the MULAG FME 500 shown at a common orthographic scale. L0–L5 are the front frame, slew column, inner boom, outer boom, head carrier, and mowing head. The C1–C4 panel shows the representative double-acting cylinder barrel–rod pair.

Manufacturer CAD supplies the link geometry, joint locations, masses, centers of mass, and inertia tensors through Phobos [von Szadkowski 20]. The same CAD-derived bodies and joint frames form the mechanical basis of the Simscape and IsaacLab models, keeping dimensions, axes, masses, and cylinder lever arms consistent while the hydraulic implementations remain model-specific.

3.2 Kinematic Structure and Coordinate Mappings

3.2.1 Joint Space and Cylinder Space

The configuration is $\boldsymbol{q} \in \mathbb{R}^5$; the actuated coordinates are the four cylinder extensions $\boldsymbol{x} \in \mathbb{R}^4$. Cylinder i joins CAD attachment points $\boldsymbol{p}_A$ and $\boldsymbol{p}_B$ on adjacent links. Its length and extension are

$$\ell_i(\boldsymbol{q}) = \left\| \boldsymbol{p}_B(\boldsymbol{q}) - \boldsymbol{p}_A(\boldsymbol{q}) \right\|, \qquad x_i = \ell_i(\boldsymbol{q}) - \ell_{0,i}, \tag{3.1}$$

with $\ell_{0,i}$ the retracted reference length. Differentiating (3.1) gives the scalar mapping between joint motion and extension rate,

$$\dot{x}_i = J_{\mathrm{cyl},i}(\boldsymbol{q})\, \dot{\boldsymbol{q}}, \qquad J_{\mathrm{cyl},i} = \boldsymbol{e}_i^{\top} \left(\boldsymbol{J}_B - \boldsymbol{J}_A \right), \tag{3.2}$$

where $\boldsymbol{e}_i$ is the cylinder-axis unit vector and $\boldsymbol{J}_A$ and $\boldsymbol{J}_B$ are attachment-point Jacobians. Virtual work gives $\boldsymbol{\tau}_{\text{act}} = \boldsymbol{J}_{\text{cyl}}^{\top}\boldsymbol{F}_h$. The mapping varies with configuration and stroke.

3.2.2 Task Space and the Calibrated Cutter Jacobian

The cutter tip is expressed in the truck chassis frame (`base_link`); using the moving slew-base frame would corrupt the tracking error. The controlled point includes the $-0.79\,\text{m}$ tool-tip offset. Vehicle motion supplies the along-track direction.

The calibrated Jacobian $\boldsymbol{J}_c \in \mathbb{R}^{3\times4}$ maps cylinder extension rates to cutter-tip velocity. It was measured at the head-down mowing pose by stepping each cylinder target and recording the settled cutter displacement. Its singular values are 8.24, 5.69, and 2.79, giving rank 3. The damped least-squares controller in Chapter 6 uses this same Jacobian.

3.3 Actuation and Measured State

Table 3.2 lists the signals and their availability.

Table 3.2: Signal inventory of the modeled system. "S" marks signals present in the Simscape data exports, "I" signals available live in the IsaacLab environment, "M" quantities that exist there only through the learned models of Chapter 5, and "P" *privileged* quantities that exist in simulation but must never enter a deployable policy or estimator input.

Signal	Dimension	Availability	Meaning
$\boldsymbol{u}$	$[-1,1]^4$	S, M	normalized proportional-valve commands
$\boldsymbol{x}$, $\dot{\boldsymbol{x}}$	$\mathbb{R}^4$	S, I	cylinder extensions and velocities
$\Delta\boldsymbol{p}$	$\mathbb{R}^4$	S, M	chamber pressure differences
$\boldsymbol{F}$	$\mathbb{R}^4$	S	logged rod forces
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	$\mathbb{R}^5$	S, I	joint positions and velocities
$\boldsymbol{p}_c$	$\mathbb{R}^3$	S, I	cutter-tip position, truck frame
$\boldsymbol{h}$	$\mathbb{R}^9$	I	forward terrain-height profile ahead of the cutter
$\boldsymbol{c} = [\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*]$	$\mathbb{R}^8$	I	desired cylinder command
$\mathbf{e}$	$\mathbb{R}^6$	P	physical randomization factors of the active plant
$\boldsymbol{z} = \mu(\mathbf{e})$	$\mathbb{R}^{d_z}$	P	privileged dynamics latent, $d_z = 2$
$\hat{\mathbf{z}}$	$\mathbb{R}^{d_z}$	M	AAM estimate of $\boldsymbol{z}$ from observable history

In IsaacLab, valve commands and differential pressures are learned-actuator states, not direct measurements. R produces them internally; IF and AIF obtain them from the inverse/forward pair. $\mathbf{e}$ and $\boldsymbol{z} = \mu(\mathbf{e})$ are privileged; only $\hat{\mathbf{z}}$ may enter AIF.

3.3.1 Sensing and Observability

The deployable observation contains articulation state, terrain rays, contact signals, the desired cylinder command, and actuator-model pressure state. Plant parameters are not observed; AIF infers their effect from causal history.

Figure 3.3 separates deployable measurements from privileged plant information. Articulation state is read from the physics engine without encoder noise or quantisation.

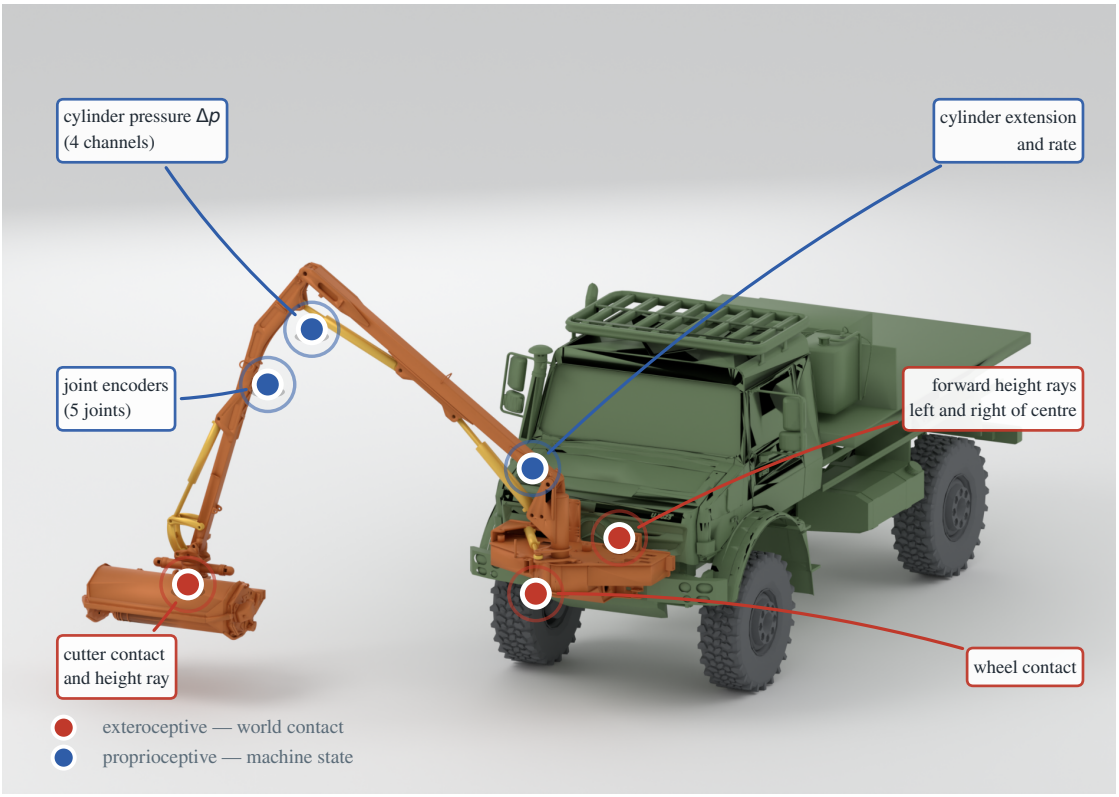


Figure 3.3: Sensor and observability map for the deployed controller. Blue callouts identify proprioceptive inputs: joint encoders, cylinder pressure differences, and cylinder extension and rate. Red callouts identify exteroceptive inputs: forward terrain-height rays and cutter and wheel contact.

3.4 High-Fidelity Simscape Model

The actuator models use a multi-domain Simscape model of the FME 500 developed in the preceding master project. Table 3.3 distinguishes the data source of each frozen component.

Table 3.3: Provenance of each frozen actuator component. The command ROM is the only one that has been calibrated against the deployment plant; the inverse/forward pair is Simscape-identified throughout. This asymmetry is a property of the comparison and is carried into the discussion of Chapter 7, not glossed over.

Component	Identified on	Calibrated on
Command ROM	Simscape closed-loop export (grey-box, full model)	IsaacLab/PhysX closed-loop rollouts, via a bounded command-gated correction head only
Control ID	Simscape closed-loop export	—
Strict FD	Simscape closed-loop export, retrained on the strict open-loop excitation export	—
SD_0, $\mu(\mathbf{e})$, SD_z, AAM	IsaacLab/PhysX adaptation collection (from scratch)	—

Simscape supplies the frozen ID and strict FD checkpoints. The ROM receives an additional bounded IsaacLab calibration (Section 5.4.1). The adaptation models are trained from IsaacLab data collected through the ROM and PhysX.

3.4.1 The Command ROM as Deployment Actuator

The command ROM plus PhysX is the deployment plant. Simscape is too slow for policy training and bare PhysX has no hydraulics. The ROM is therefore both the plant model and the actuator of R:

- R uses the deployment plant itself and is the reference condition.
- IF replaces it with an independently identified inverse/forward pair.
- AIF adds online adaptation to IF.

The R–IF gap measures the transfer cost of the independently identified pair; it is not a general ranking of the two model classes.

3.4.2 Mechanical Subsystem

The mechanism uses the CAD-derived links, joints, and cylinder attachment frames of Section 3.1. It is simulated in Simscape Multibody [The MathWorks, Inc. 24b]. The joint-space dynamics take the standard manipulator form

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_{\text{fric}}(\dot{\boldsymbol{q}}) = \boldsymbol{\tau}_{\text{act}} - \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{\text{ext}}, \tag{3.3}$$

with inertia $\boldsymbol{M}$ from the imported CAD. Friction combines viscous, Coulomb, and Stribeck terms. The cutting load scales with feed velocity and vegetation density. Each cylinder uses a spherical–prismatic–cylindrical loop and hard stops enforce the limits in Table 3.1.

3.4.3 Hydraulic Subsystem

Figure 3.4 shows four 4/3 valve-driven asymmetric cylinders fed by a variable-speed, fixed-displacement pump ($Q_p = D_p\omega$), accumulator, and pressure-relief valve.

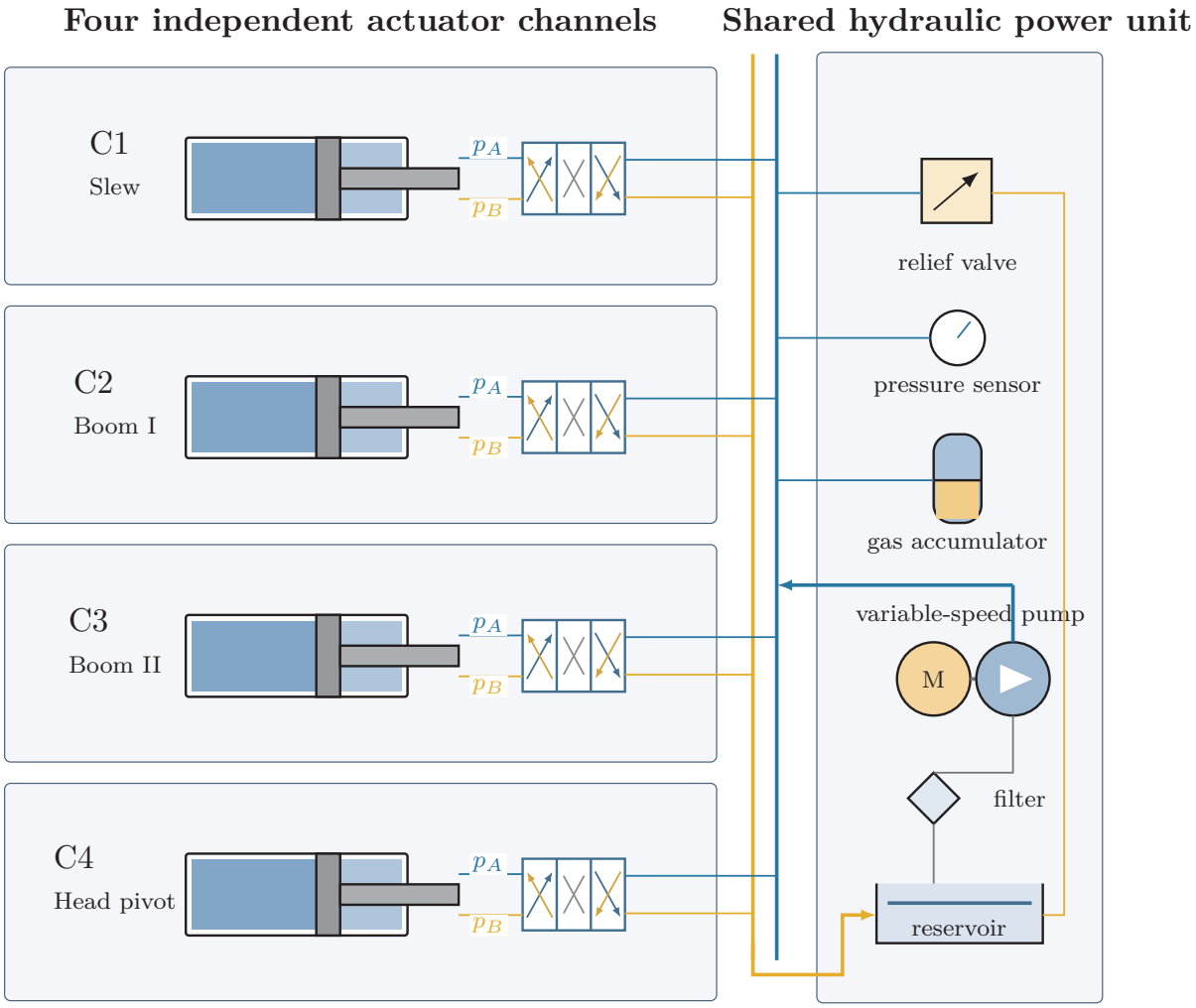


Figure 3.4: Hydraulic system schematic. Four proportional-valve cylinder channels share a variable-speed pump, reservoir, accumulator, sensor, and pressure-relief valve. Blue and orange rails denote supply and return.

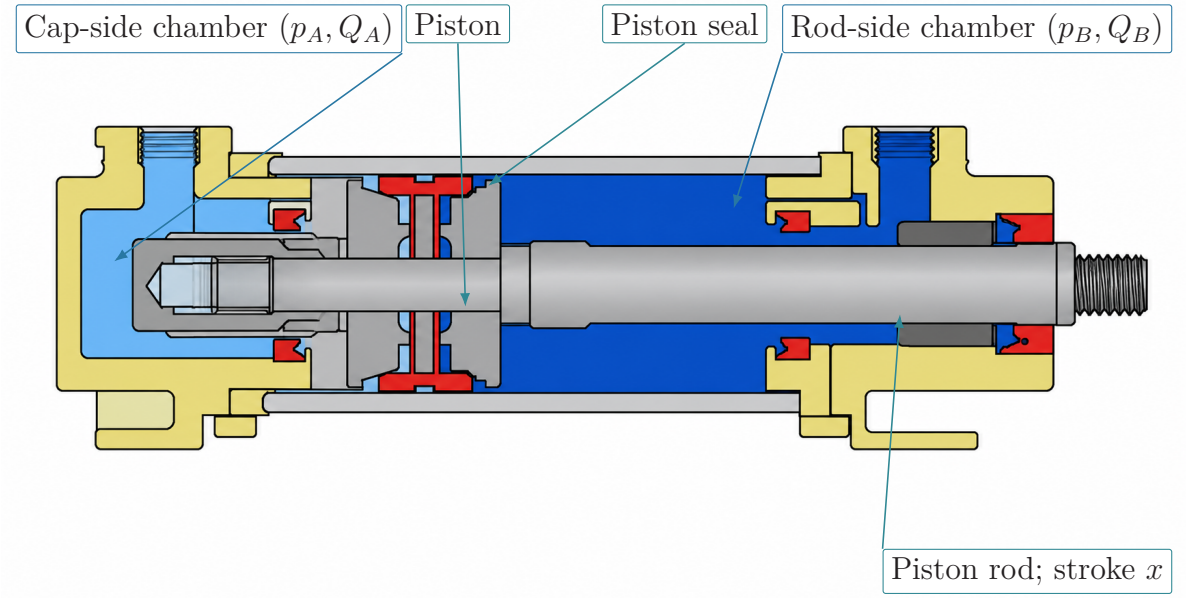


Figure 3.5: Annotated cutaway of an asymmetric double-acting hydraulic cylinder. The cap-side and rod-side chamber pressures generate the piston force and rod stroke.

Figure 3.5 identifies the cylinder chambers, piston, seal, and rod used in the actuator model.

The 4/3 valve in Figure 3.4 meters flow to the cylinder chambers. The chamber pressures follow

$$\dot{p}_1 = \frac{E_{\text{eff}}}{V_1(x)}\big(Q_1 - A_1\dot{x} - C_{\text{leak}}(p_1 - p_2)\big), \qquad \dot{p}_2 = \frac{E_{\text{eff}}}{V_2(x)}\big(-Q_2 + A_2\dot{x} + C_{\text{leak}}(p_1 - p_2)\big), \tag{3.4}$$

where $V_{1,2}(x)$ include dead volumes, $A_1 \neq A_2$, and C_{leak} is internal leakage. Entrained air is represented by

$$E_{\text{eff}}(p) = \left(\frac{1}{E_f} + \frac{\alpha}{p}\right)^{-1}, \tag{3.5}$$

Valve flow follows the orifice law of Section 2.1.2; spool dynamics are $\dot{x}_v = (u - x_v)/\tau_v$. The piston force is $F_h = p_1 A_1 - p_2 A_2 - F_{\text{fric}}(\dot{x})$. The coupled model is stiff and uses a variable-step implicit solver, which prevents RL-time simulation.

3.4.4 Validated Scope of the Model

Validation uses a fixed integral-backstepping controller at 0.01 s while sweeping $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ GPa. Slew-axis delay falls from about 0.85 s to 0.65 s, the system delay index from 0.45 s to 0.35 s, and task-space tracking error by about 11 %. The improvement saturates beyond 1.5 GPa.

The model represents one machine and omits thermal dynamics. The logged rod-force channel is not carried into the learned actuator models: force is generated internally by the PhysX plant at deployment, and the logged channel also fails the piston-area consistency audit (Section 5.4). Chapter 4 introduces variation through parameter randomization.

3.5 IsaacLab Environment

Policy training, adaptation-data collection, and evaluation run in IsaacLab [Mittal 23] on PhysX. The USD articulation contains the Unimog carrier, five arm links, and four prismatic cylinder joints. Learned actuator models supply the hydraulic behavior between the controller and PhysX cylinder targets.

3.5.1 Simulation and Control Rates

Table 3.4 lists the four simulation clocks. PhysX runs at 120 Hz and the policy at 60 Hz. The learned actuator models and AAM update every three policy steps (0.05 s); their prediction is held across the intervening PhysX updates. Data collection uses this same 50 ms grid.

Table 3.4: The four rates of the simulated system. They are reported separately throughout the thesis; "the control frequency" is not a well-defined quantity here.

Layer	Step	Rate
PhysX physics	1/120 s	120 Hz
Policy / control (decimation 2)	1/60 s	60 Hz
Learned actuator model (3 control steps)	0.05 s	20 Hz
AAM latent update	0.05 s	20 Hz

3.5.2 Environment Configuration Details

The arm root is spawned at $z = 1.26$ m; ground-height spawning buries the vehicle. Cylinder targets are integrated in reference space so gravity does not erase commanded increments. Vehicle motion is scripted because the authored tyres and suspension do not provide reliable PhysX traction; a planar fixture holds lateral position and heading.

3.6 The Verge-Following Task

3.6.1 Task Definition

The task is moving verge following. The truck drives a 26 m strip at a speed sampled from 2 to 7 km/h; development uses 4 km/h. Episodes allow 2808 steps (46.8 s), while scoring stops when the strip is completed. The arm must keep the cutter at the target height, on the verge centre line, in contact with the ground, and within its travel limits. Terrain height is randomized per environment (Figure 3.6).

Eleven reference-path families span straight, periodic, spline, sweep, and lane-change motion. Results use the worst path family and terrain mesh rather than a pooled score. A regression test verifies that all eleven families are distinct.



Figure 3.6: IsaacLab realization of the moving verge-following task. The scripted truck travels on the level road while the four-cylinder arm places the cutter above the procedurally generated verge and follows the white reference path. Randomized grass, bushes, soil patches, twigs, and logs form a non-colliding visual layer; the underlying verge mesh supplies the terrain disturbance and height-sensing geometry used by the controller.

3.6.2 Task Feasibility and Plant Capability

The action contract limits cylinders 1–3 to 4 mm per policy step, which bounds lateral cutter speed at

$$v_{\text{lat}}^{\max} = \frac{4\,\text{mm}}{1/60\,\text{s}} = 0.240\,\text{m/s}. \tag{3.6}$$

This uses the 1/60 s policy interval. Early path settings exceeded this limit; the worst case (`boom_sweep`) required 3.7× the available rate. Path and terrain parameters are now derived from (3.6), and a preflight check rejects infeasible tasks.

3.7 Control Architecture

The vehicle advances at the scripted speed and the reference generator supplies the verge centre line and cut height. The policy maps normalized observations to four desired cylinder increments; it has no runtime baseline controller. The action contract bounds those increments. R uses the command ROM, IF the frozen inverse/forward pair, and AIF adds the bounded adapter conditioned on $\hat{\mathbf{z}}$. All paths set the same PhysX cylinder target. The DLS controller is used only for teaching and identification.

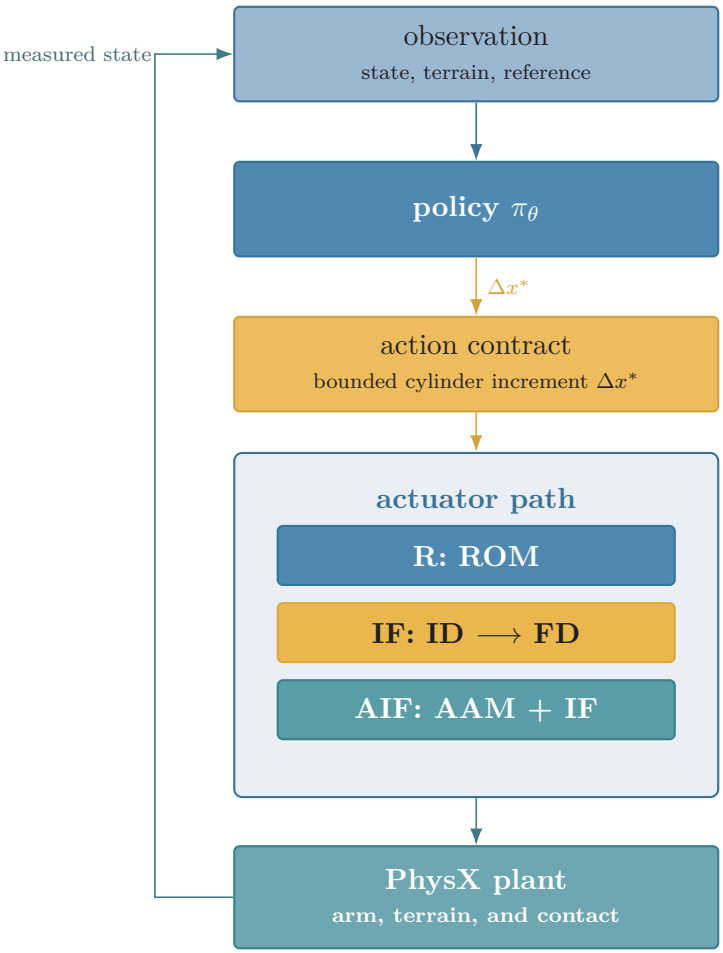


Figure 3.7: Deployed control stack. The policy produces a bounded cylinder increment; R, IF, or AIF realizes it in the shared PhysX plant. Measured state closes the loop.

3.8 Problem Formulation

The reference is the verge centre line $y_{\text{ref}}(t)$ and cut height h_{ref}, expressed in the truck frame. With cutter-tip position $\boldsymbol{p}_c(t)$ and terrain height $g(\cdot)$, the scored errors are

$$e_{\text{lat}}(t) = p_{c,y}(t) - y_{\text{ref}}(t), \qquad e_{\text{cut}}(t) = \Big(p_{c,z}(t) - g(\boldsymbol{p}_c(t))\Big) - h_{\text{ref}}. \tag{3.7}$$

The objective is to minimize the mean and tail of both errors across path families and terrain while maintaining contact, head alignment, and smooth motion. Learned models are evaluated on one-step and recursive cylinder predictions. The stacks are tested across the randomized plant family; AIF is compared with IF on held-out factors.

Constraints are cylinder limits and headroom (Table 3.1), the action contract, checkpoint output limits, and the head-down condition.

3.9 System-Level Evaluation Criteria

Controllers and models are evaluated separately. Controller scores can hide actuator-model errors through policy compensation.

The system-level criteria cover three aspects of operation. Cut-height and lateral errors measure task accuracy. Contact, head alignment, impact rate, and cutter acceleration describe tool–terrain interaction and motion quality. Cylinder headroom and safety terminations check whether the task is completed without driving the mechanism into its limits. The metrics are evaluated over the scored part of each episode and are kept in physical units where possible. This makes a failed gate traceable to a specific limitation rather than to a single combined reward value.

3.9.1 The Eight-Gate Promotion Criterion

Table 3.5 defines the eight physical gates used throughout this thesis. Their role is to supply a criterion that is *not* the reward. A policy is optimised against the scalar return of Section 6.2.5, but it is selected and promoted against this vector of physical quantities, and Section 6.3.2 shows that the two point in measurably different directions. The gates are therefore an instrument for checkpoint selection and for the block-ratchet promotion of Section 6.4, and that is how every result in this thesis uses them. They are used to rank and to promote candidates, not to certify a stack as accepted, and no claim of certified acceptance is made anywhere in this work.

All controller stacks are evaluated with the same path, terrain, and randomization conditions. A candidate is preferred when it satisfies more of the gates at a wider margin; strong mean tracking does not compensate for loss of contact, inadequate cylinder headroom, excessive impacts, or safety terminations. Worst-case aggregation prevents easy terrain or path cases from masking a local failure. Mean absolute errors quantify sustained tracking performance, while the lower and upper percentiles capture short alignment, clearance, or acceleration events that a mean can hide. Contact and termination measures are reported as fractions so that episodes of different completed length remain comparable.

Table 3.5: Eight system-level acceptance gates. Each gate uses at least three terrain meshes and the worst mesh; tracking-critical gates also use the worst path family. †Hard impacts is a wear constraint.

Gate	Metric	Threshold	Aggregation
Cut height	mean $\lvert e_{\mathrm{cut}} \rvert$	$\leq 0.040\,\mathrm{m}$	worst mesh
Lateral tracking	mean $\lvert e_{\mathrm{lat}} \rvert$	$\leq 0.300\,\mathrm{m}$	worst mesh, worst family
Ground contact	contact fraction	≥ 0.700	worst mesh, worst family
Head alignment	p01 head-down alignment	≥ 0.900	worst mesh
Hard impacts†	hard-impact step fraction	≤ 0.350	worst mesh
Tool acceleration	p95 cutter acceleration	$\leq 120\,\mathrm{m/s^2}$	worst mesh
Cylinder headroom	p01 travel margin	$\geq 0.005\,\mathrm{m}$	worst mesh
Safety terminations	termination rate	≤ 0.080	worst mesh

3.9.2 Cylinder and Model Checks

Cylinder checks use fresh $\pm 10\,\mathrm{mm}$ commands, coordinated $80\,\mathrm{mm}$ motions, return error, and saturation. Model checks use one-step and rollout accuracy, input ablation, and Jacobian conditioning (Chapter 5). Chapter 7 reports these with the acceptance contract.

The small command checks local response and deadband; the coordinated motion checks coupling between actuator channels. Return error tests reversibility, and saturation checks whether the learned model respects the command and stroke limits. These diagnostics are interpreted alongside the system gates, not as substitutes for closed-loop task performance.

The system is now specified. The next chapter turns to the data the learned models are built from.

4. Data Generation and Preparation

Chapter 3 defined two plant representations: detailed hydraulics in Simscape and fast rigid-body dynamics in IsaacLab. This chapter documents the datasets that connect them, including collection, auditing, feature construction and splitting. Table 3.3 records their provenance.

Simscape campaigns. Closed-loop runs provide realistic operating data for the inverse and reduced-order models. Separate open-loop valve excitation provides the causal command information needed by the strict forward model (Section 4.4).

ROM calibration. After Simscape training, the command ROM was fine-tuned on five closed-loop IsaacLab reference shapes. Only its bounded correction head remained trainable, with Simscape replay retained (Section 4.6). The ID–FD pair was not calibrated in IsaacLab.

IsaacLab adaptation data. The system-dynamics surrogate, privileged encoder and AAM use a separate 118-run collection from the deployed command ROM–PhysX plant, including dedicated PRBS windows (Section 4.7). No earlier adaptation or system-dynamics model was reused. Figure 4.1 summarizes the common preparation workflow.

4.1 Dataset Objectives and Design Requirements

Four requirements governed collection and preparation.

R1: realistic operating distribution. The learned models are deployed inside a tracking controller, so they must be accurate in the states that controller visits: coordinated multi-axis motion, gravity-loaded poses, moderate speeds. Data collected far from that distribution would be accurate where it does not matter.

R2: causal identifiability of the valve. A forward model is only useful to a policy if it responds to the command. This requires that the valve signal in the training data carry information that the state history does not already carry, which is a property of the *experiment*, not of the network [Ljung 99, Isermann 11].

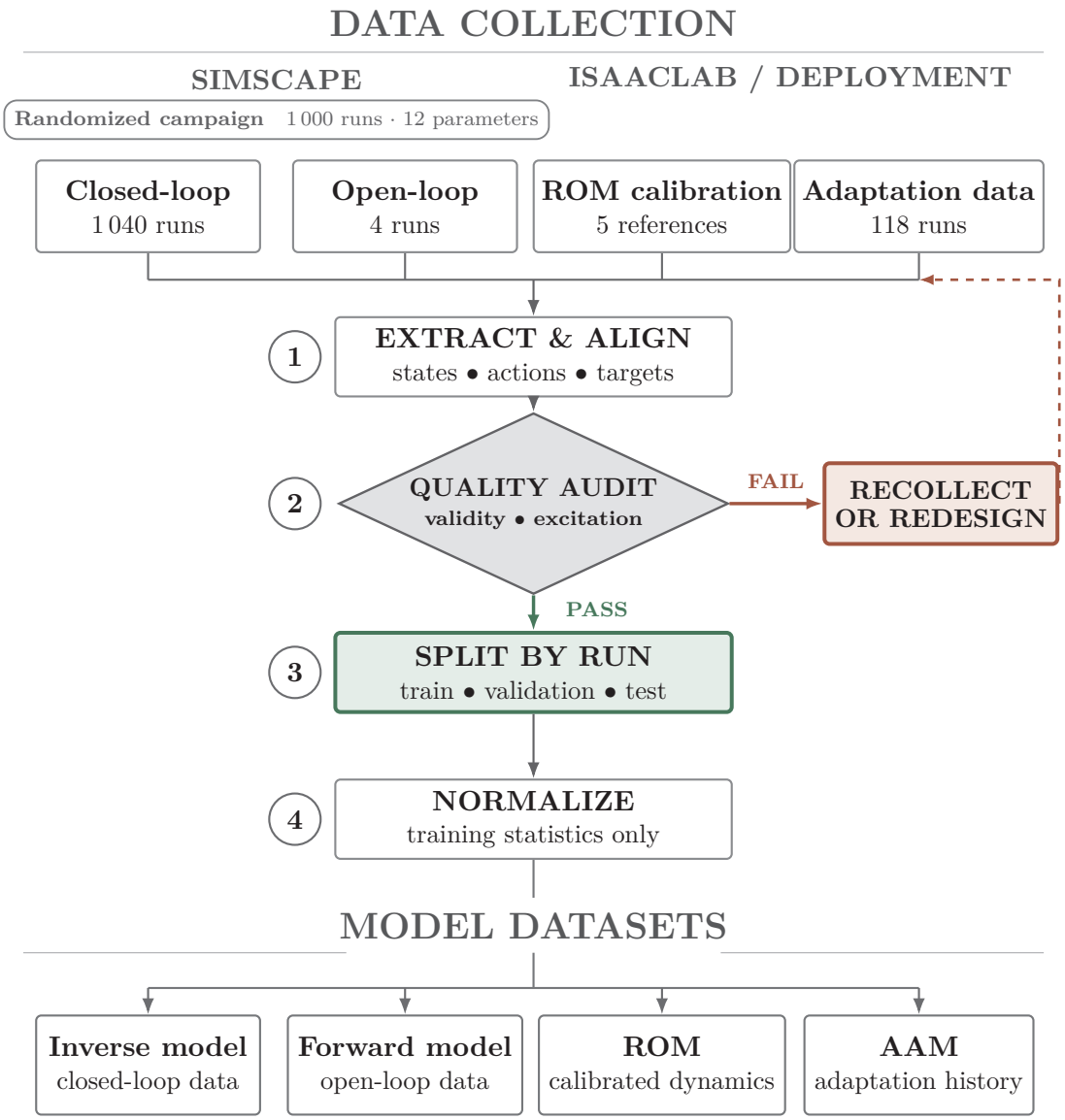


Figure 4.1: Data-preparation workflow. Two Simscape collections and two IsaacLab collections pass through common extraction, audit, run-level splitting and training-statistics normalization stages. The randomized Simscape campaign is provenance metadata rather than a pipeline stage. A failed audit returns the experiment to collection; accepted material forms the inverse, forward, ROM and AAM datasets.

R3: variation across a plant family. Robust training and, later, latent adaptation both need many plants rather than one. The variation has to be physically meaningful, it has to demonstrably change the measured response, and it has to be recorded per run, because the privileged latent of Section 5.5 is computed from those recorded factors.

R4: clean separation. Train, validation, and test material must be separable at the level of complete runs, so that no transition window straddles a split boundary and no run contributes to two splits.

R1 and R2 require different experiments. Closed-loop control gives realistic motion but correlates command with state; open-loop excitation breaks that correlation but is less representative of deployment. The two campaigns are therefore retained and combined as described in Section 4.5.

4.2 Randomized Parameter Space of the Simscape Campaign

This section records the Simscape randomization design. No reported model is trained on this curriculum; it is retained as collection provenance and because its compliance analysis (Section 4.2.3) informed the IsaacLab factor design. Readers concerned only with trained models may continue at Section 4.3.

4.2.1 Sampling Design

The closed-loop campaign draws its plants from a twelve-dimensional parameter space using Latin hypercube sampling [McKay 79], which spreads the draws more evenly over each marginal than independent uniform sampling would at this sample size. The twelve coordinates are fixed once for the whole batch, before any simulation runs, so the parameter table is reproducible and can be audited independently of the data it produced.

The batch is organised as a curriculum rather than a single homogeneous pool. Run 1 is the untouched nominal plant. Runs 2–251 randomize only the hydraulics, runs 252–501 only the mechanics, and runs 502–1000 both together, giving the composition shown in Figure 4.2(a). This structure is deliberate: it makes it possible to attribute a later modelling failure to the hydraulic or the mechanical axis by retraining on one block at a time, which a uniformly mixed batch would not allow.

4.2.2 Hydraulic Parameters

Table B.1 lists the hydraulic axes. They were chosen to correspond to the physical mechanisms that make one machine differ from another in service, in the sense discussed in Section 2.1.2: air works its way into the fluid, spools and seals wear, and relief settings drift.

The seal-wear coordinate deserves comment because it is the one place where a single sampled number is deliberately allowed to move several model parameters together. A worn cylinder does not become leakier without also becoming slower, so $w_{\mathrm{seal},i}$ scales the

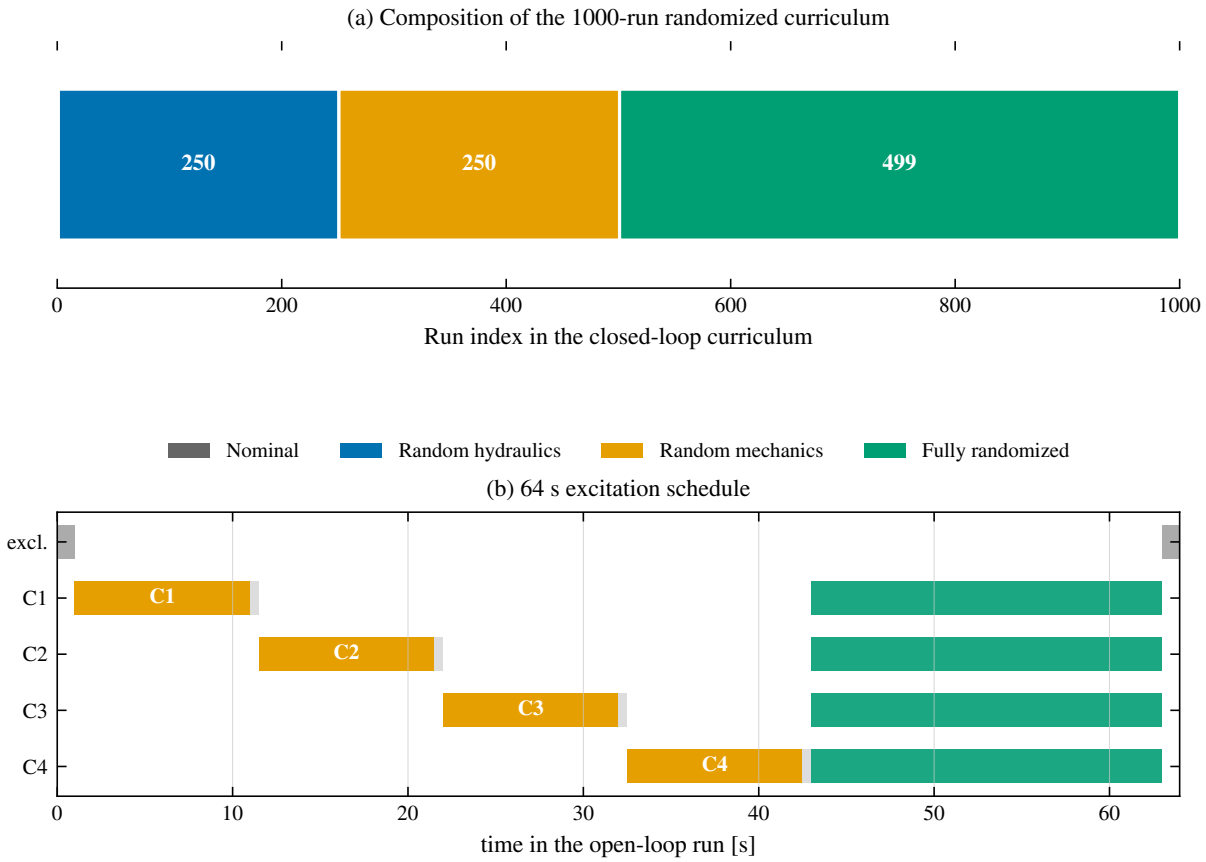


Figure 4.2: Design of the two collection campaigns. (a) Composition of the closed-loop curriculum: one nominal run, then 250 hydraulics-only, 250 mechanics-only and 499 fully randomized draws. (b) Timeline of one open-loop excitation run.

piston clearance linearly, the valve leakage area with the cube of the same factor, and the spool time constant from 8 ms to about 13 ms at the top of the range. Sampling those three independently would have produced physically incoherent machines, some of them very leaky and yet perfectly fast.

4.2.3 The Compressibility Axis

The entrained-air and dead-volume coordinates randomize the chamber dead volume, which spans 505 to 579 mL across the batch. This reaches the pressure dynamics through the continuity equation (3.4), which carries the chamber volume: a larger chamber is more compliant and slower, a smaller one stiffer and faster. The compressibility axis therefore enters the batch as capacitance variation.

4.2.4 Mechanical Parameters

The mechanical axes, listed in Table B.2, are applied to all fourteen rigid bodies of the mechanism: the six links, the four cylinder barrels, and the four pistons. For each body the mass is perturbed by up to ±20 %, the centre of mass by up to ±5 % of its nominal offset, and the inertia tensor together with its products of inertia is perturbed consistently

Algorithm 4.1: Closed-loop curriculum generation.

input : run budget $N = 1000$; joint limits $\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}$; nominal parameters $\boldsymbol{\theta}_0$
output : dataset $\mathcal{D} = \{(\text{trajectory}_i, \boldsymbol{\xi}_i)\}_{i=1}^{N}$

$\mathbf{L} \leftarrow \textsc{LatinHypercube}(N, 12)$ `// one design for the whole batch`
for $i \leftarrow 1$ **to** N **do**
  assign i to a curriculum block (nominal / hydraulic / mechanical / full)
  $\boldsymbol{\xi}_i \leftarrow \textsc{DrawParameters}(\mathbf{L}_i, \text{block})$ `//` $\varepsilon_{\text{air}}, \varepsilon_{\text{vol}}, c_d, w_{\text{seal}}, p_{\text{set}}$ `; masses, CoM, inertia, stiffness`
end
for $i \leftarrow 1$ **to** N **do**
  inject $\boldsymbol{\xi}_i$ into the Simscape model
  $n_w \leftarrow \textsc{RandInt}(12, 20)$; $\boldsymbol{W} \leftarrow \textsc{LatinHypercube}(n_w, \text{JDOF})$ within $[\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}]$
  assign each segment a speed class (slow / medium / fast / variable); **no** dwell
  $\boldsymbol{q}_{\text{ref}}(t) \leftarrow \textsc{Interpolate}(\boldsymbol{W})$ over 22 s
  simulate with `ode23t`; log on the 0.05 s grid
  store $(\boldsymbol{u}, \boldsymbol{x}, \dot{\boldsymbol{x}}, \Delta\boldsymbol{p}, \boldsymbol{F}, \boldsymbol{q}, \dot{\boldsymbol{q}}, \boldsymbol{q}_{\text{ref}}, \boldsymbol{\xi}_i)$
  if $i \bmod 100 = 0$ **then** checkpoint $\mathcal{D}$ to disk
end

with the mass change, so that the body remains physically realisable rather than acquiring an inertia that its mass could not produce.

The joint stiffness and damping terms model compliance in the bearings and linkage rather than in the fluid. They matter here because they set how much of a pressure transient appears as boom oscillation instead of clean motion, which is the coupling that a purely kinematic controller cannot anticipate.

4.3 Closed-Loop Data Collection

Algorithm 4.1 summarizes the procedure. The parameter table is fixed before simulation, and each run receives a new reference path.

4.3.1 The Controller as a Data-Collection Instrument

Every closed-loop run uses the same integral-backstepping controller at 0.005 s. Its gains remain fixed, so response differences are caused by the plant draw rather than controller retuning.

Joint 2 uses PI control with velocity damping and gravity bias because it carries the boom load. A barrier term enforces the URDF limits. Per-axis command limits are [0.95, 0.95, 0.80, 0.80, 0.90]; the first four are also stored in the inverse-dynamics checkpoint (Section 5.2).

4.4 Open-Loop Excitation Experiments

4.4.1 Limitations of the Closed-Loop Batch

The first forward model achieved one-step $R^2 \approx 0.999$ on closed-loop data but was almost insensitive to valve input: zeroing the input changed long-horizon error by only a few

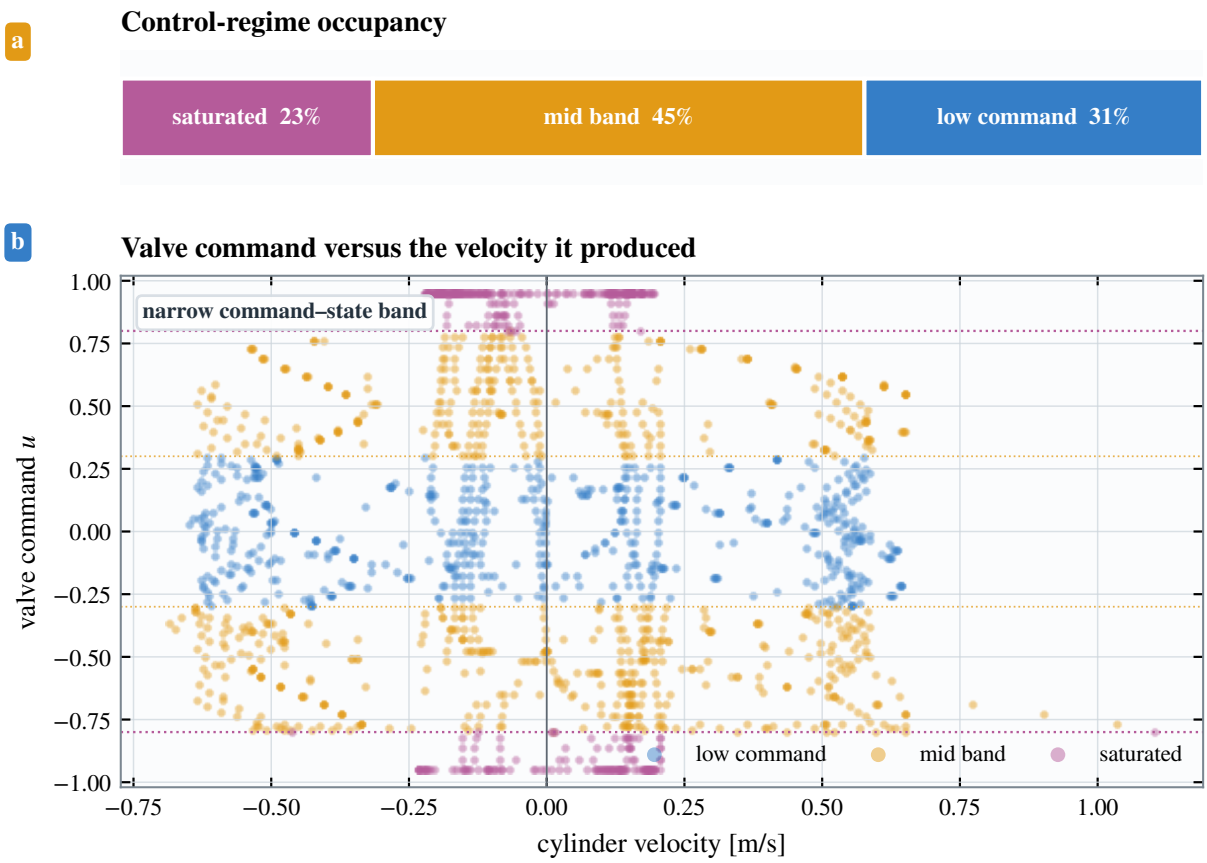


Figure 4.3: The closed-loop batch is diverse in control effort but not identifiable. (a) Fraction of time the valve spends saturated, in the mid band and in low-command creep. (b) The same run in the command–state plane, where the correlation that blocks identification is visible.

percent (Section 5.3.1). In closed loop, command is largely determined by the state history already given to the model, so the data do not separate command causality from state correlation [Ljung 99].

Figure 4.3 confirms that coverage alone is not the issue. The valve is saturated for about one quarter of a pooled run, operates at mid-authority for roughly half, and otherwise remains in low-command creep. Despite this spread, command and cylinder velocity remain concentrated along a narrow band. The data are diverse in control effort but not identifiable, motivating a separate open-loop experiment.

4.4.2 Design of the Excitation Signal

The excitation run lasts 64 s and follows the fixed schedule in Figure 4.2(c). One second of startup is simulated and then discarded, because the plant is settling from its initial condition and those samples describe the initialisation rather than the dynamics. Four sequential slots follow, each giving one cylinder ten seconds of excitation followed by a half-second zero-command gap. The gaps make the boundaries between single-axis segments unambiguous. A twenty-second phase then drives all four channels at once, and a final second of zero command closes the run. Commands are limited to ± 0.90 of full authority.

An initial rapidly switching signal met the formal persistence condition but kept each piston near one stroke position. The final design instead uses long full-authority plateaus in the sequential phase and slow, high-authority motion in the simultaneous phase. A slow near-square cycle is combined with distinct medium-frequency channel components, with raised-cosine entry and exit ramps. This provides both input excitation and useful stroke coverage.

4.4.3 Plant Draws and Acceptance

The same excitation is applied to four plant draws taken from the same parameter space as the closed-loop campaign: nominal, randomized hydraulics, randomized mechanics, and fully randomized. Holding the input fixed and varying only the plant is what makes the four runs directly comparable, since any difference in the response is attributable to the parameters.

Each run is checked for stroke coverage before it is accepted. The collector requires at least 10 % of the available stroke on cylinders 1–3 and 8 % on cylinder 4, and records the achieved coverage per channel in the run summary. The runs retained here cover 33 % of the stroke on cylinder 1, and 12 % on cylinder 4, which is the narrowest of the four. Every channel clears the acceptance floor, and the audit of Section 4.10 confirms that the retained material is informative at these coverages: the instantaneous command matrix has full rank four and the lagged regressor matrix reaches rank eighty.

An earlier export was rejected because half its runs contained duplicated state trajectories. The acceptance checks in Section 4.10 now detect this failure directly.

4.5 Combining Closed-Loop and Open-Loop Datasets

The inverse model uses only closed-loop data because its target is the controller command for a measured state. The forward model is retrained on the strict open-loop export while retaining closed-loop replay at weight 0.25. Without replay, four excitation runs do not provide enough pose coverage.

Figure 4.4 shows the difference: closed-loop data covers the reachable range, while open-loop data concentrates at identifiable valve plateaus. The forward model therefore uses both.

For ROM training, the inverse and forward exports are joined by run identifier, time index and sample time. The join is accepted only when their sixteen-dimensional cylinder states match sample for sample. It contains 452 400 transitions at 0.05 s from 1 040 runs, split into 936/52/52 complete runs. Valve command is used only to supervise the ROM's internal valve head; it is not a runtime input (Section 5.4).

This aligned nominal-plant export trains the inverse, forward and ROM models. It predates the thesis and its collection script is unavailable, so its measurable contents and audits define its reproducibility boundary. Exact alignment, increment consistency and the identifiability failure were checked directly on the export.

The separate thousand-run randomized curriculum was collected for an earlier adaptation plan. No reported result is trained on or evaluated against it; the adaptation chain uses the IsaacLab collection of Section 4.7. The curriculum is retained as provenance and documented in Appendix B.

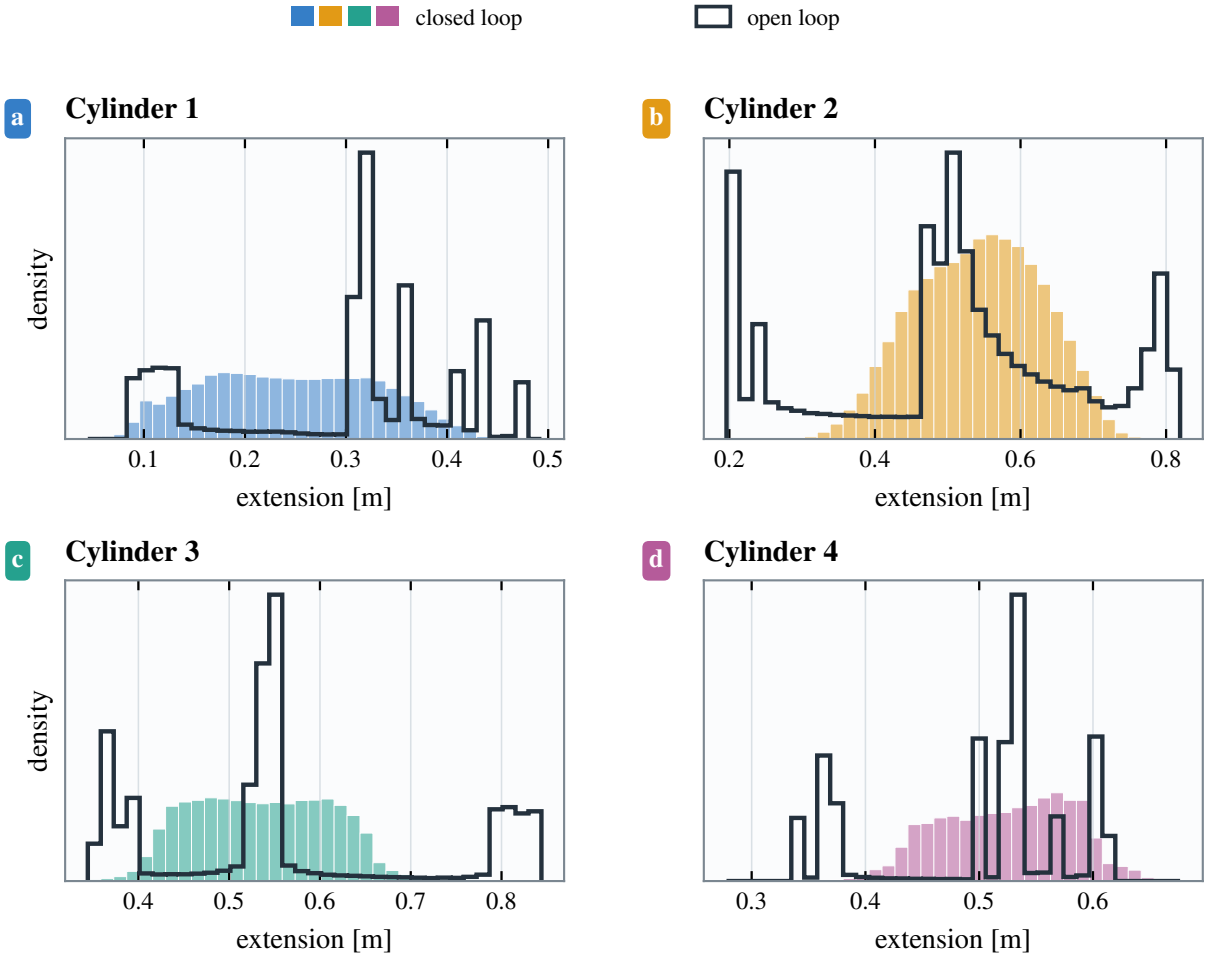


Figure 4.4: Extension coverage of the two campaigns, per cylinder and pooled over runs (closed-loop filled, open-loop outline). The closed-loop data fills the reachable range smoothly; the open-loop data concentrates into the plateaus where the valve is identifiable.

4.6 Closed-Loop Calibration Data for the Command ROM

The ROM drives a PhysX articulation whose compliance and contact response are not present in Simscape. Further Simscape training cannot remove the resulting deployment offset.

The calibration set therefore comes from closed-loop ROM–PhysX operation on the native 0.05 s grid. Its 4 084 transitions store the six-step runtime history and measured PhysX transition.

Only the command-gated correction head is trainable; all other parameters are frozen and the correction is capped at 3 mm. Original-data replay at weight 0.25 preserves wider pose coverage. The gate makes the correction zero at zero command, retaining stationary hold.

The trajectories predate the moving-verge benchmark and exclude cutter–ground contact. The bounded head limits, but does not remove, this coverage gap. Section 3.4.1 explains the consequence for the three-way comparison.

4.7 The IsaacLab Adaptation Collection

4.7.1 Provenance of the Adaptation Data

The models that carry the adaptation argument (the system-dynamics surrogate, the privileged factor encoder, and the AAM) describe the plant that the adaptive controller is closed around. That plant is not the Simscape model. It is the command ROM plus PhysX, the same four-dimensional cylinder command port used at deployment. A surrogate identified on Simscape data would be a model of a model, and the latent it defines would describe variation the deployment plant does not necessarily exhibit. The adaptation dataset is therefore collected *in* the deployment plant, and the frozen ROM, ID and FD checkpoints enter this chain only as fixed components, never as things to be retrained.

4.7.2 The Physical Factor Vector

The randomized quantity is a six-dimensional physical factor vector

$$\mathbf{e} = \Big[e_{\text{flow}},\ e_{\text{compliance}},\ e_{\text{seal}},\ e_{\text{pressure}},\ e_{\text{mass}},\ e_{\text{COM}_z}\Big], \tag{4.1}$$

with the ranges of Table B.3. Four coordinates are hydraulic and two mechanical, which is what makes the hydraulic-only, mechanical-only, and combined domains of Section 4.7.3 separable. Each candidate was admitted only after a one-factor sensitivity screen confirmed that moving it alone produces a measurable change in cylinder extension, velocity, pressure, or end-effector motion. Parameters whose effect could not be measured, or which could not be varied safely, were rejected rather than carried as decoration.

One omission is deliberate. Actuation delay is not a randomized coordinate. Delay in this system is an *emergent* property of flow, compliance, and load rather than a free parameter, so it is measured as a response property of each run (Chapter 7) instead of being supplied to any model as a known factor. Handing an estimator a factor that the real machine would never publish is the same mistake as handing it the true latent.

4.7.3 Domains and Factor Design

Runs are organized into five domains (nominal, hydraulic-only, mechanical-only, combined-training, and an *unseen* combined-test domain), and one factor vector is fixed for a complete run, so that no run mixes plants. Factor configurations are drawn with deterministic scrambled Sobol sequences [Sobol' 67] rather than independent uniform sampling, with explicit boundary anchors so that the corners of the space are represented and not merely approached, and with fixed per-domain seeds so the design is reproducible independently of the data it produced. Combined draws that compound into an excessive response gain are rejected at draw time rather than being collected and discarded later.

The resulting design is auditable: 98.3 % of the factor keys are unique, the active-factor matrix is full rank, every factor quartile is populated, and the factor keys of the training, validation, and test splits are disjoint. That last property is the one that matters most for the claims of Chapter 7: a test run does not merely come from a different episode, it comes from a plant the models have never seen.

4.7.4 Collection Procedure

Every run is collected through the same interface the policy uses. The DLS controller closes the loop on measured cylinder state and issues its command through the four-dimensional action port into the command ROM and PhysX. No joint-target bypass is used, and no PPO policy is used to collect identification data, because a policy trained on the same plant would reintroduce the closed-loop correlation that Section 4.4 spent a campaign removing.

Independent excitation is supplied by dedicated four-channel PRBS windows applied through that same action port, with a small amplitude and a hold of ten control steps, so that the excitation is causally independent of the state without ever leaving the safe envelope. Probe actions are logged separately from controller output, so any analysis can distinguish the two. Each run follows a fixed eleven-segment structure (settle, three postures, three PRBS windows, DLS task motion, reach-and-return, and stow), giving 11 304 control steps and 188.4 s of simulated time per run. The separate open-loop excitation collection of Section 7.5.1 uses a different schedule and runs to 11 040 steps. Three postures plus line, coupled-sine, and reach-and-return motion were verified in a pilot to give full rank over all four action channels under varied geometry and load.

Figure 4.5 summarizes the IsaacLab collection configuration and its frozen run-level outputs.



Figure 4.5: IsaacLab adaptation-data collection. Each run uses one fixed six-factor plant draw and combines DLS task motion with independent PRBS probes. State, action and factor keys are logged, audited and frozen into disjoint train, validation and held-out test splits.

4.7.5 The Frozen Dataset

The final collection contains 118 complete runs: 2 nominal, 40 hydraulic-only, 12 mechanical-only, 48 combined-training, and 16 unseen combined-test. Collection ran in four vectorized IsaacLab environments with per-environment factors and logging. Slots that tripped a safety or validity check were discarded and retried individually rather than being silently kept. All 118 runs passed the final collection gate, which checks finite physics, exact model-grid alignment, command rank, factor-response authority, state coverage, and saturation limits.

The dataset is then *frozen*: the raw runs are checksummed and the run-level split is fixed at 82 training, 20 validation, and 16 reserved combined-test runs, with zero factor-key leakage between them. The reserved sixteen do not influence history length, checkpoint, hyperparameter or architecture selection. They are scored only after each model is fixed and are treated as a single-use test set. Test results are reported with validation results, not in place of them.

4.7.6 Learning Timebase and Hydraulic State

The learning timebase is the actuator model grid, not the control clock. Samples are exact 50 ms ROM transitions taken at control indices $0, 3, 6, \ldots$: state row t and command row t predict state row $t + 3$. Nothing is interpolated. This is the same alignment rule as Section 3.5.1, and it is enforced rather than assumed, because a model trained across a mismatched interval is wrong by a fixed factor in the quantity it exists to supply.

The hydraulic state used by the system-dynamics models is

$$\boldsymbol{s}_t = \left[\boldsymbol{x}_t^{1:4},\ \dot{\boldsymbol{x}}_t^{1:4},\ \Delta\boldsymbol{p}_t^{1:4}\right] \in \mathbb{R}^{12}, \tag{4.2}$$

with joint position and velocity carried as *context* only, and the prediction target the next 50 ms state delta. Differential pressure is not a measured signal. At runtime it is the actuator model's recursive state, generated by the command ROM or frozen forward model (Section 3.3). The system-dynamics model and AAM may use it because it exists inside the deployed loop, but it inherits actuator-model error. Causal history is therefore retained to represent pressure transients that one sample cannot summarize.

4.8 Feature and Target Construction

4.8.1 The Actuator Models

The three learned actuator models take cylinder-space features, in the ordering and notation fixed in Section 5.1. Their input contracts differ because their questions differ. The inverse model receives the measured cylinder state, the logged rod forces, the desired increments, and the previously applied command, twenty-eight numbers in all. The forward model receives the applied valve command and the measured state, twenty numbers. The reduced-order model receives a six-sample history of state, previously predicted valve, and desired command, twenty-four numbers per step.

Three conventions apply throughout, and they are the same ones the models enforce at inference time.

Increment Targets

Every dynamic target is a within-run one-step increment, $\Delta \boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never an absolute state. This removes the large static component of the signals and, more importantly, makes the trivial persistence solution an explicit baseline that a model can be measured against rather than a hiding place it can settle into.

Causal Alignment

Features at step k contain only quantities available at step k. The extraction stage verifies this by recomputing every logged target from the within-run difference of the states and requiring exact agreement, a check reported in Section 4.10.

Time-Base Conversion

The open-loop campaign is logged at 0.01 s but the models run at 0.05 s, so five consecutive raw samples are combined into a single transition. The valve input attributed to that transition is the command held at the start of the window, which is the causally correct choice. The mean command over the window is also recorded, but only for analysis. Cylinder extension velocity is not logged directly in the closed-loop archive and is obtained by differentiating extension on the output grid, which is consistent with the trapezoidal identity that the strict data satisfies to $R^2 = 0.994$–0.9997 (Equation 5.6).

4.8.2 Observable History of the Adaptation Models

The adaptation models use the frozen IsaacLab collection. A history window carries only quantities produced by the deployed stack at runtime: the five joint positions, the five joint velocities, the four cylinder pressure differences and the four previously applied valve commands, giving the eighteen-value row of Section 5.6. The pressure entry is the learned actuator's own recursive state rather than a measurement (Section 3.3, Table 3.2); it is admitted because the model that generates it runs inside the deployed loop, and the consequence of that choice is examined where the estimator is evaluated. Four classes of signal are excluded by construction, and the exclusion is enforced by an explicit input mask rather than by convention:

- the physical factors $\mathbf{e}$ and the true latent $\boldsymbol{z}$, because recovering them is the estimator's job;
- any future observation, because the estimator is causal;
- the hidden internal state of the command ROM, because the deployment interface does not expose it;
- the desired task reference, so that the latent describes the plant rather than the trajectory being asked of it.

Windows are contained within a single run and never cross a run boundary, and each window carries the stopped-gradient target $\boldsymbol{z} = \mu(\mathbf{e})$ of its own run together with the run identifier, domain, split, and valid history bounds. Applied to the 118 frozen runs this yields 444 624 causal history ticks. What is built from them, how the history length is chosen, and how the result is validated downstream belong to Chapter 5.

4.9 Preprocessing and Data Splits

Splitting is done at the level of complete runs, never of individual samples. A random sample-level split would place transitions from the same trajectory on both sides of the boundary and report a validation score that mostly measures interpolation within a run. For the command-conditioned dataset the 1 040 runs are divided into 936 training, 52 validation, and 52 test runs, giving 407 160, 22 620, and 22 620 transitions respectively, with no run identifier appearing in more than one split. The IsaacLab adaptation collection is split by run *and* by factor configuration (82/20/16, Section 4.7), which is the stronger condition: separating runs alone would still let two runs of the same plant fall on opposite sides of the boundary and turn a claim about unseen plants into a claim about unseen episodes.

The strict open-loop export needs a different treatment because it contains only four runs, and holding one out would remove an entire plant draw from training. Instead each run is divided into twenty temporal blocks, every fifth block is assigned to validation, and a five-sample guard interval is discarded at each block boundary so that no 0.05 s transition window spans the split. A simple last-twenty-percent split was tried first and rejected: it changed the spread of the targets between training and validation by a factor of 1.3 to 2.3, because the final fifth of the run is the simultaneous excitation phase and is dynamically unlike everything before it. That split would have measured extrapolation to a different regime and called it generalisation.

Normalization statistics are computed on the training split only and stored inside each checkpoint, so that a model always normalizes with the statistics it was fitted with. Mixing statistics between checkpoints silently invalidates any comparison, and the loader enforces the stored contract rather than recomputing it.

Finally, samples are carried with an explicit validity flag rather than being silently dropped. In the strict export 93.7 % of samples are marked valid. The remainder are the excluded startup and shutdown seconds and the zero-command gaps. Keeping them in the archive with a flag, instead of deleting them, means the time base stays contiguous and the within-run difference checks remain meaningful.

4.10 Dataset Diagnostics

No dataset in this thesis is used before it passes an explicit audit, for the reason given at the end of Section 4.4. The audit of the strict open-loop export is reproduced in Table B.4.

The two rank figures are the ones that speak directly to requirement R2. The instantaneous command matrix has rank four, so no channel is a linear combination of the others, and the lagged regressor matrix built from the command history has rank eighty, so the excitation is persistently exciting to the order the models need. The condition number of that lagged matrix is about 4.6×10^3, which is finite and workable rather than comfortable. It is a consequence of using long plateaus, and it is the price paid for the stroke coverage those plateaus buy.

The exactness of the three increment checks deserves a word, since a maximum error of exactly zero invites suspicion. It is expected here: the targets are computed from the logged states by the same within-run difference that the check recomputes, so the

check verifies that the extraction stage did what it claims, not that the simulation is accurate. It catches misalignment, off-by-one indexing, and rows that have been reordered or concatenated across run boundaries, which are the failure modes that occur. Physical fidelity is a separate question, addressed by the validation study in Section 3.4.4.

Two properties of the Simscape material are recorded here because later chapters refer to them. Stroke coverage is 12 % on the head-pivot cylinder, the narrowest of the four channels, and that cylinder is also the one the strict forward model predicts least well. The rod-force channel carries zero supervised weight in the reduced-order model. That exclusion is a design choice rather than a data defect: force is generated internally by the PhysX plant at deployment, so the actuator model has no need to reproduce it, and the logged channel additionally fails a piston-area consistency check on this export, which confirms the choice. No result rests on it.

The IsaacLab collection is audited against different failure modes. Design adequacy is checked before use (unique factor keys, full active-factor rank, disjoint keys across splits), physical validity per run (finite physics, exact model-grid alignment, full command rank, bounded saturation), and leakage structurally, with privileged channels removed by an explicit input mask rather than by trusting the feature list. Its limitation is scale: 118 runs over six factors is a sparse design, which is why the adaptation claims of Chapter 7 are reported with paired bootstrap intervals and per-run detail rather than pooled point estimates.

With the data described and audited, the next chapter turns to what was learned from it: the inverse and forward models that form the IF actuator path, the command-conditioned reduced-order model that replaces them in R, and the system-dynamics, encoder, and adaptation models built on the IsaacLab collection.

5. Learned Models and Online Adaptation

Chapter 4 established how the datasets were generated and why both closed-loop operation and open-loop valve excitation were required. This chapter develops the learned representations that consume them: the inverse-dynamics (ID) and forward-dynamics (FD) networks of the IF actuator path, the command-conditioned reduced-order model (ROM) that replaces them in R, and the system-dynamics surrogate, privileged factor encoder and Arm Adaptation Module (AAM) that together carry the adaptation argument.

Two commitments organise the chapter. The first is that the surrogate is kept strictly separate from the deployed actuator: the system-dynamics model is an instrument for establishing whether a compact dynamics latent exists and is useful, never a component of any control stack. The second is that no model is accepted on a supervised score, because a recurring result of this work is that such scores can be not merely optimistic but uninformative: a forward model reached a one-step R^2 near 0.999 while having learned almost nothing about valve authority (Section 5.3). Every model is therefore presented with its contract, training objective, and the offline evidence that accepted or refused it. All models operate on cylinder-space signals; the mapping to task space belongs to Chapter 3, the control integration to Chapter 6, and deployment-level results to Chapter 7.

5.1 Modelling Overview and Common Notation

All learned models act on the four hydraulic cylinders of the MULAG arm, indexed $i \in \{1, \dots, 4\}$ in the fixed order slew cylinder (*Drehzapfen*), boom cylinder I (*Ausleger I*), boom cylinder II (*Ausleger II*) and head pivot cylinder (*Messerkopf-Schwenk*). The passive fifth joint is not actuated and enters no model input. At the learned-model sample time $T = 0.05\,\mathrm{s}$ the discrete index k refers to the cylinder extension $\boldsymbol{x}_k$, the extension velocity $\dot{\boldsymbol{x}}_k$, the chamber pressure difference $\Delta\boldsymbol{p}_k$ and the logged rod force $\boldsymbol{F}_k$, each in $\mathbb{R}^4$. The valve command $\boldsymbol{u}_k \in [-1, 1]^4$ is the normalised spool command of the four proportional valves. The desired cylinder command produced by the task controller is the pair

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8, \tag{5.1}$$

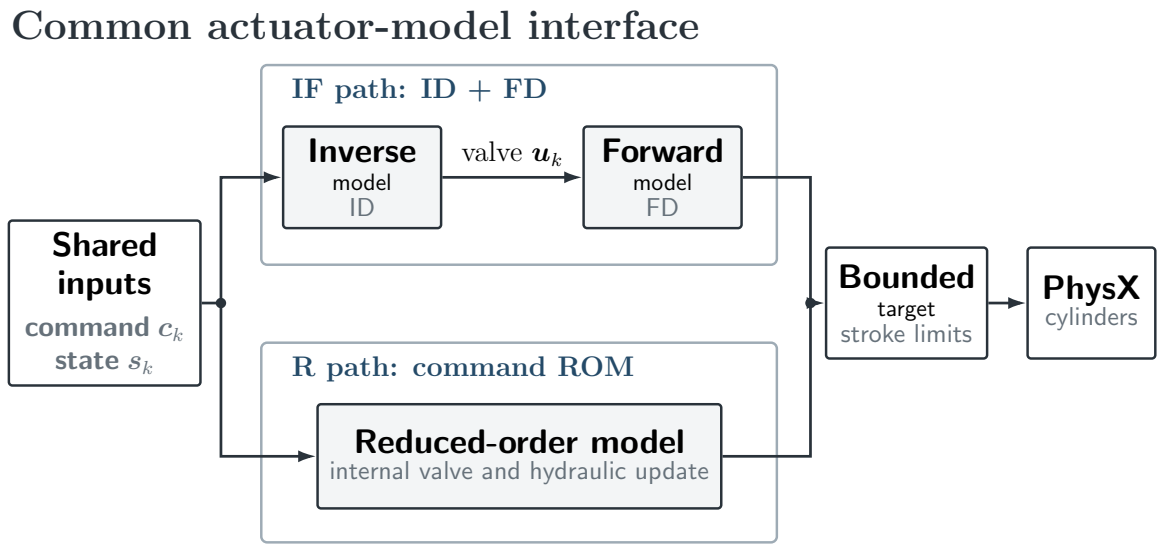


Figure 5.1: Common interface of the two alternative actuator paths. Both receive the desired cylinder command and measured hydraulic state, and both produce the same bounded PhysX cylinder target. The IF path composes the inverse and forward models; the R path uses one command-conditioned reduced-order model.

the extension and extension-velocity increments to be realised over the next sample. Predictions carry a hat, and $\Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k$ denotes a within-run one-step increment throughout.

Figure 5.1 compares the actuator paths. In IF, the ID maps the state and desired increment to a valve command, and the FD predicts the resulting increment. In R, one command-conditioned model predicts the valve internally and propagates pressure, velocity and extension. Both paths produce the same bounded PhysX cylinder target in IsaacLab [Mittal 23] under the same policy action contract [Schulman 17].

Three conventions apply throughout. Dynamic targets are within-run increments, with $\Delta\hat{\boldsymbol{x}} = \mathbf{0}$ as the persistence baseline. Each checkpoint uses its own training-set normalisation. Inputs at step k are causal, and targets are measured transitions to step $k+1$.

Simscape supplies the identification data described in Chapter 4. The exported arrays are audited, frozen and used to train the models in PyTorch [Paszke 19]; Simscape is not part of training or deployment.

The closed-loop Simscape export [The MathWorks, Inc. 24a] contains 452 400 transitions at $T = 0.05\,\mathrm{s}$ from 1 040 runs, split by run into 936/52/52 training, validation and test runs. It supplies the supervised ID, FD and command-ROM data. The FD repair uses four open-loop 64-s runs at 0.01 s with rank-4 valve excitation on four plant draws.

5.2 Inverse Dynamics Model

The ID network maps the current hydraulic state and desired cylinder increment to a valve-command change:

$$\Delta \boldsymbol{u}_k = f_{\mathrm{ID}}\left(\boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta \boldsymbol{p}_k,\, \boldsymbol{F}_k,\, \Delta \boldsymbol{x}_k^*,\, \Delta \dot{\boldsymbol{x}}_k^*,\, \boldsymbol{u}_{k-1}\right) \in \mathbb{R}^4, \tag{5.2}$$

a 28-dimensional input. The deployed command is $\boldsymbol{u}_k = \mathrm{clip}(\boldsymbol{u}_{k-1} + \Delta \boldsymbol{u}_k,\, -\boldsymbol{u}_{\max},\, \boldsymbol{u}_{\max})$ with per-axis limits $\boldsymbol{u}_{\max} = [0.95,\, 0.95,\, 0.8,\, 0.8]$ stored in the checkpoint. The change formulation retains the previous valve operating point near the direction-dependent dead zone (Section 2.1.2). Logged achieved transitions serve as desired increments because no independent cylinder-space reference was recorded.

The 512-wide residual MLP [He 16] uses SiLU activations, layer normalisation and dropout [Elfwing 18, Ba 16, Srivastava 14]. Adam minimises a smooth-L_1 loss on normalised targets, with saturated samples up-weighted [Kingma 15, Huber 64].

Figure 5.2 separates the three input groups and shows the four residual blocks explicitly. The network predicts a four-axis valve-command increment, which is accumulated and clipped before use.

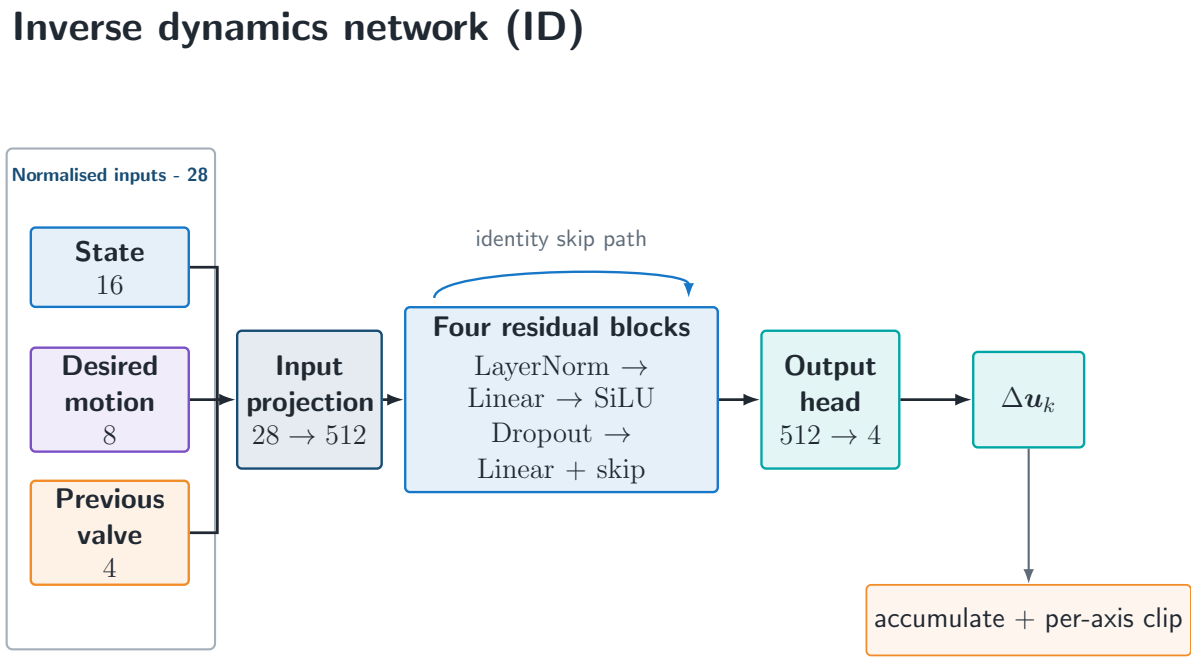


Figure 5.2: Inverse-model architecture. Current hydraulic state (16), desired motion (8) and the previous valve command (4) form the 28 inputs. A 512-wide, four-block residual MLP predicts $\Delta \boldsymbol{u}_k$, which is accumulated and clipped per axis to form the bounded valve command.

Per-valve validation R^2 is 0.9973–0.9996. This shows reproduction of logged commands, not plant authority: in closed-loop data, achieved transitions and logged commands are correlated (Section 2.4.2).

5.2.1 Paired Control Fine-Tuning

The composite is assessed by the *paired differential response*: the FD prediction under the ID command for the requested target minus the prediction under its command for a zero target,

$$\begin{aligned} \boldsymbol{u}_k^{\mathrm{tgt}} &= f_{\mathrm{ID}}(\cdot,\, \boldsymbol{c}_k,\, \boldsymbol{u}_{k-1}), \qquad \boldsymbol{u}_k^{0} = f_{\mathrm{ID}}(\cdot,\, \mathbf{0},\, \boldsymbol{u}_{k-1}), \\ \Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}} &= f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{\mathrm{tgt}}, \cdot\Big) - f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{0}, \cdot\Big). \end{aligned} \tag{5.3}$$

Before fine-tuning, the composite normalised control error was 1.00, equal to the zero-response baseline, and all regression slopes were below 0.004 (Table 5.1). The networks were fine-tuned jointly on $\mathcal{L}_{\mathrm{ctl}} = \|\Delta\hat{\boldsymbol{x}}^{\mathrm{pair}} - \Delta\boldsymbol{x}^{*}\|^2_{\mathrm{norm}} + \lambda_{\mathrm{ID}}\mathcal{L}_{\mathrm{ID}} + \lambda_{\mathrm{FD}}\mathcal{L}_{\mathrm{FD}}$, with the supervised terms retained. Target blocks were shuffled across valid states to break the logged state–target pairing. After 30 epochs, control error fell to 0.109; all slopes exceeded 0.8 and all correlations exceeded 0.92. This frozen checkpoint is used in IF and AIF.

Table 5.1: Effect of paired control fine-tuning on the composite differential response (5.3), per cylinder axis on validation data. Before fine-tuning the composite had essentially no control authority despite near-perfect supervised scores of both networks; the composite normalised control error fell from 1.0011 to 0.1086.

	Regression slope		Correlation	
Cylinder axis	before	after	before	after
1 (slew)	0.003	0.818	0.135	0.920
2 (boom I)	0.002	0.928	0.090	0.967
3 (boom II)	0.001	0.849	0.037	0.933
4 (head pivot)	0.004	0.912	0.249	0.958

5.2.2 Transfer of the Frozen Inverse Model

Because the adapter corrects a frozen ID, its transfer was checked on a separate 64-run IsaacLab collection: 2 nominal, 16 hydraulic-only, 12 mechanical-only and 34 combined runs. These runs do not overlap the 118-run adaptation collection. They are also outside the ID training distribution, which contains only Simscape data. Evaluation was teacher-forced at each ROM tick and scored on the predicted valve command.

The recorded valve RMS is 0.4350 and the ID error is 0.1739, corresponding to about 84 % explained command variance. Domain-mean error varies by only a factor of 1.09 (Table 5.2); the nominal domain is not the easiest.

Three limitations remain. As a one-step state predictor, the ID gives 1.002 mm extension error versus 1.143 mm for zero-order hold, while zero-order hold is 8 % better on velocity. Recursive composite rollout diverges in 48 of 64 runs; mean valve-saturation occupancy is 0.989 in diverging runs and 0.223 in stable runs. This does not affect the one-step closed-loop use. Finally, measured pressure input increases extension error from 1.311 mm to 1.907 mm, so deployment uses the training-mean pressure.

Table 5.2: Transfer of the frozen inverse model across deployment domains, 64 runs, teacher-forced at the ROM tick. The error is on the valve command, the model's own output. The recorded valve signal has RMS magnitude 0.4350. The spread across domains is a factor of 1.09; nominal is not the easiest domain.

Domain	runs	valve RMSE	s.d.	worst run
combined	34	0.1739	0.0311	0.2917
hydraulic	16	0.1672	0.0170	0.2174
mechanical	12	0.1828	0.0129	0.2082
nominal	2	0.1829	0.0155	0.1984

5.3 Forward Dynamics Model

The FD network predicts the one-step cylinder response to a valve command,

$$\left[\Delta\hat{\boldsymbol{x}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\widehat{\Delta\boldsymbol{p}}_k\right] = f_{\mathrm{FD}}\left(\boldsymbol{u}_k,\ \boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \boldsymbol{F}_k\right) \in \mathbb{R}^{12}, \tag{5.4}$$

using the same residual trunk as the ID network. Supervised pretraining on the closed-loop export, with an auxiliary recursive rollout loss [Nagabandi 18], reached grouped one-step R^2 near 0.999 for the extension increments.

Figure 5.3 shows the forward model separately because its input and output contracts differ from the inverse model despite the shared residual trunk.

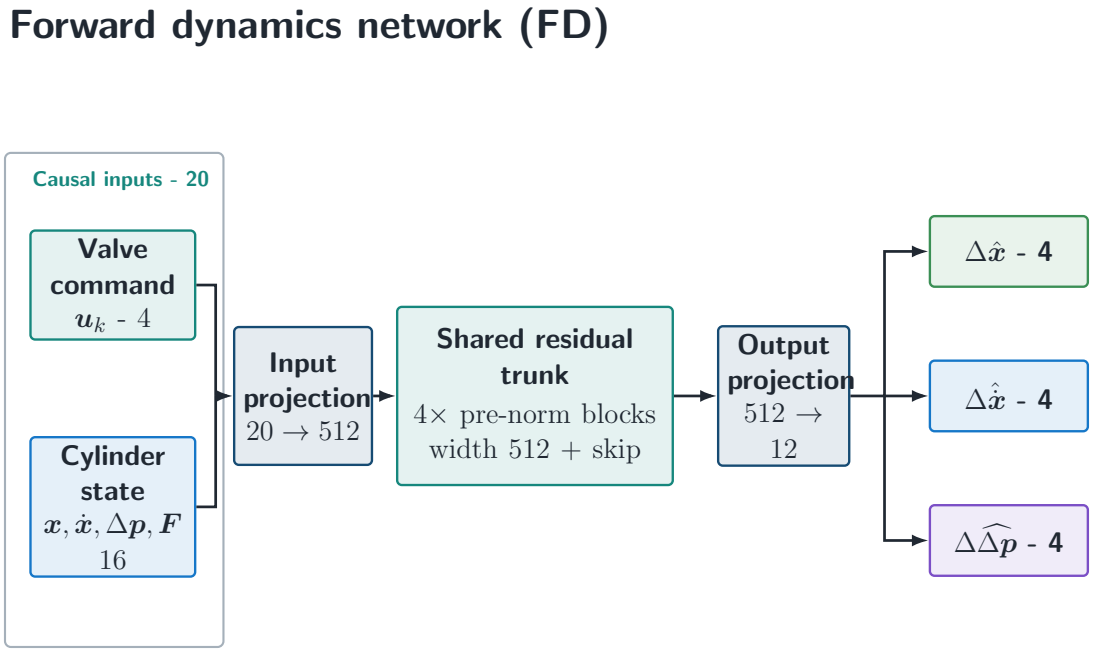


Figure 5.3: Forward-model architecture. The applied valve command and cylinder state condition a one-step hydraulic transition through a 512-wide, four-block residual MLP. Its 12 outputs are extension, velocity and pressure increments. Extension is the integrated controller output; velocity and pressure are diagnostic outputs.

5.3.1 Assessment of the Supervised Score

The supervised score did not establish valve sensitivity. At a 0.5 s horizon, zeroing the valve input changed the rollout-error ratio to only 0.94. The FD had mainly extrapolated the smooth closed-loop kinematics because command and state history were nearly redundant. Closed-loop ablation confirmed that the valve was not identifiable, so repair required new data.

The repair dataset applies rank-4 open-loop valve plateaus to four plant draws. Its trajectories are distinct, valve history predicts pressure with $R^2 \approx 0.80$–0.85, valve-regressor variance-inflation factors are 7–10, and relief saturation occupies 9–22 % of samples. A previous export with duplicated trajectories was rejected [Isermann 11].

Starting from the paired-control checkpoint, strict retraining minimised

$$\mathcal{L}_{\mathrm{FD}} = \mathcal{L}^{\mathrm{strict}}_{\mathrm{Huber}} + 0.25\,\mathcal{L}^{\mathrm{MAT}}_{\mathrm{Huber}} + 2.0\,\mathcal{L}^{(10)}_{\mathrm{roll}} + 10.0\,\mathcal{L}_{\mathrm{kin}}, \tag{5.5}$$

combining the one-step loss on the strict data, a replay term on the original closed-loop data, a 0.5 s recursive rollout loss, and a kinematic consistency term enforcing the trapezoidal identity

$$\Delta \boldsymbol{x}_k = T\,\dot{\boldsymbol{x}}_k + \tfrac{1}{2}\,T\,\Delta\dot{\boldsymbol{x}}_k, \tag{5.6}$$

which holds in the strict data with $R^2 = 0.994$–0.9997. A constant valve Jacobian was rejected because measured authority varies by a factor of 15–75 across pressure quartiles and can change sign. This operating-point dependence motivates the ROM structure [Merritt 67].

Strict retraining reduces extension rollout RMSE from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s. The valve-zero error ratio rises from 0.94 to 1.81, and per-axis one-step extension R^2 improves from $[-5.7, 0.60]$ to $[0.76, 0.98]$. Zero-order hold remains competitive below 0.25 s, and velocity increments remain weak on two axes. Deployment therefore integrates only $\Delta\hat{\boldsymbol{x}}$ and retains velocity as a diagnostic output.

5.4 Reduced-Order Model

The R path uses one command-conditioned grey-box model instead of the ID–FD composition. Physical equations propagate pressure, force, velocity and extension. Neural heads estimate valve realisation, load and bounded residuals [Karniadakis 21, Lutter 19]. Valve authority is pressure dependent by construction through the orifice-flow and continuity equations [Merritt 67]. A GRU [Cho 14] reads six samples (0.25 s) of cylinder state, previous predicted valve command and desired command. The measured valve is only an internal-head training target [Hwangbo 19].

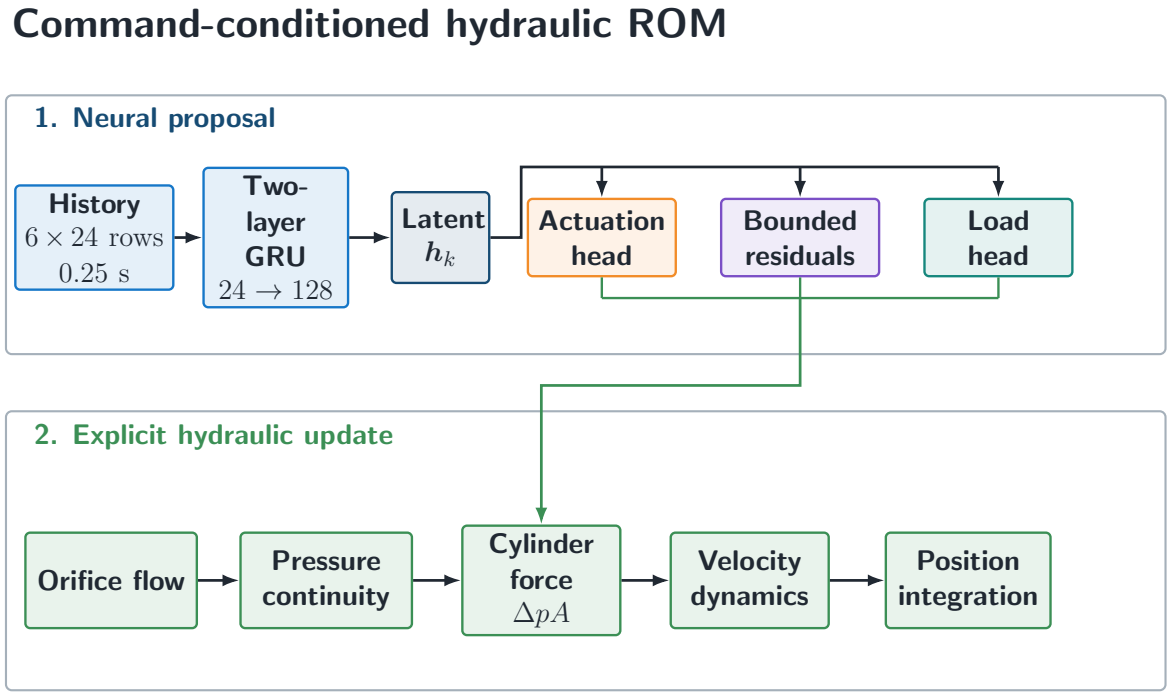


Figure 5.4: Command-conditioned ROM. A recurrent neural proposal encodes a six-step history. Its actuation, bounded-residual and load heads condition the explicit hydraulic update: orifice flow, pressure, force, velocity and position. The explicit chain constrains the predicted transition.

The actuation head uses a learnable smooth dead zone with per-cylinder width and asymmetric gains [Yang 26]. Its output $\hat{a}_i \in [-1, 1]$ drives the orifice-flow and continuity update

$$\begin{aligned} q_i &= \hat{a}_i \, k_{q,i} \sqrt{\tfrac{1}{2}\big(p_{s,i} - \operatorname{sgn}(\hat{a}_i)\, \Delta p_i\big)_+}, \\ \Delta\hat{p}_{i,k+1} - \Delta p_{i,k} &= \frac{4\beta_i}{V_i(x_i)}\big(q_i - A_i\dot{x}_i - k_{\ell,i}\,\Delta p_i\big)\,T \;+\; r_{p,i}, \end{aligned} \tag{5.7}$$

where β_i, $p_{s,i}$, $k_{q,i}$, $k_{\ell,i}$, A_i and $V_i(x_i)$ are the effective bulk modulus, supply pressure, flow gain, leakage, piston area and chamber volume. Pressure then produces force and motion:

$$\begin{aligned} \hat{F}_i &= \Delta\hat{p}_{i,k+1} A_i + r_{F,i}, \qquad \ddot{x}_i = \frac{\hat{F}_i - c_{v,i}\,\dot{x}_i - \ell_i(\boldsymbol{h}_k)}{m_i}, \\ \dot{\hat{x}}_{i,k+1} &= \dot{x}_{i,k} + \operatorname{sat}\big(T\,\ddot{x}_i + r_{v,i}\big), \qquad \hat{x}_{i,k+1} = x_{i,k} + \tfrac{T}{2}\big(\dot{x}_{i,k} + \dot{\hat{x}}_{i,k+1}\big) + r_{x,i}, \end{aligned} \tag{5.8}$$

Here $c_{v,i}$ is viscous friction, m_i is effective modal mass and $\ell_i(\boldsymbol{h}_k)$ is the learned configuration-dependent load. Physical parameters are trained in log-space and clamped to plausible ranges. Pressure and velocity residuals are bounded to 10 % of their step limits, and the position residual to 1 cm, preventing the neural heads from bypassing the physical update.

Training uses rollout horizons $H \in \{1, 5, 20, 50\}$ and scheduled sampling up to 75 % self-fed [Bengio 15]. Residual penalties retain the physical structure. Logged rod force has

zero supervised weight because PhysX generates it at deployment and the export fails the piston-area consistency check.

With the true command, mean one-step R^2 is 0.998 for the internal valve, 0.64 for velocity and 0.97 for extension. Zeroing or shuffling the command reduces velocity R^2 below zero and extension to about 0.5; the extension Jacobian is diagonally dominant with condition number 2.14. At 5 s, terminal extension R^2 is 0.998, 0.998, 0.84 and 0.91 across the cylinders. Pressure-increment R^2 is only 0.06–0.09, so the internal pressure is a dynamic state, not a calibrated pressure estimate (Figure 5.5).

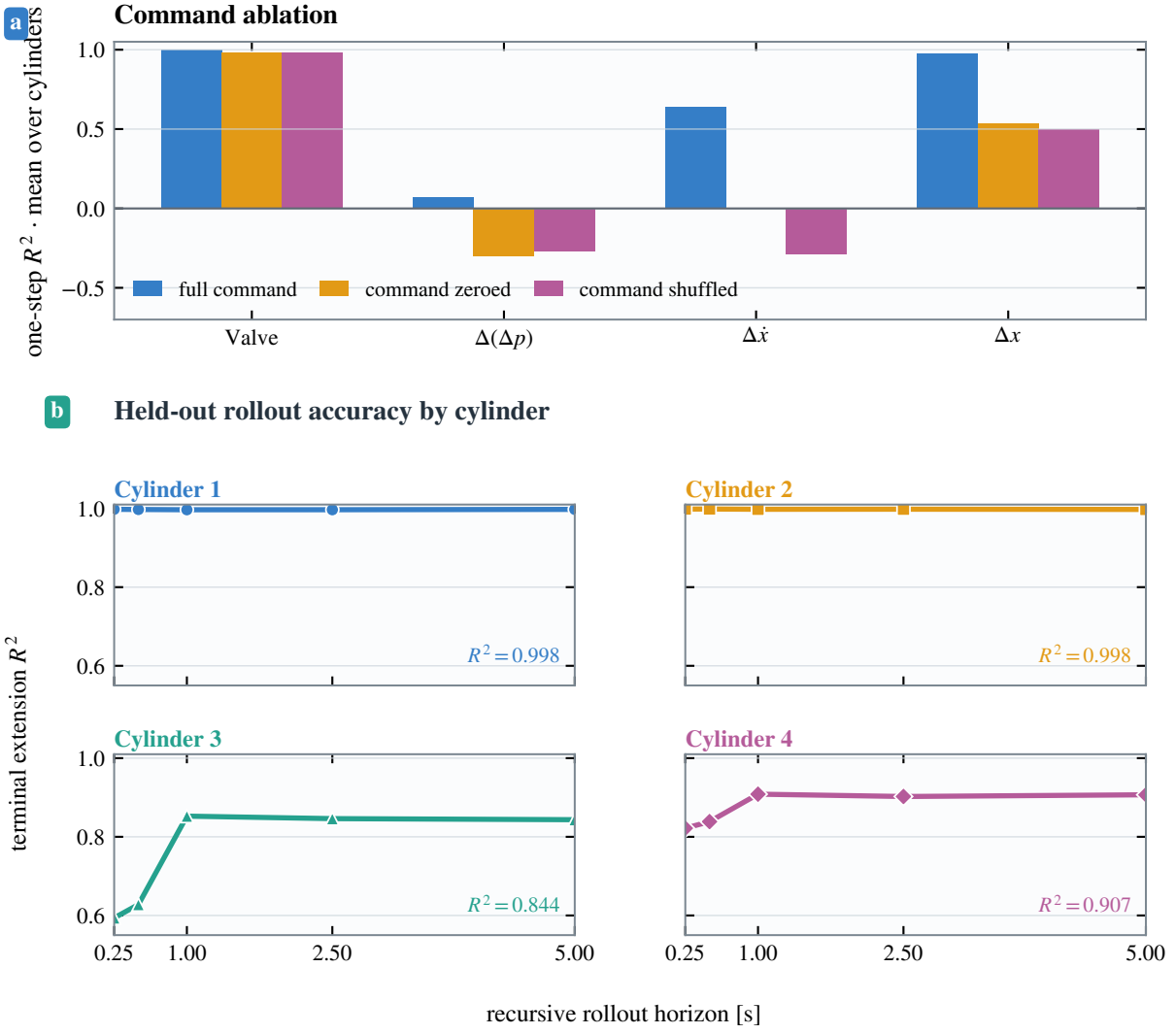


Figure 5.5: Offline validation of the command ROM on held-out runs. (a) One-step R^2 per output group, averaged over the four cylinders, with the full desired command, with the command zeroed, and with the command shuffled across the batch. The collapse of the velocity and extension scores without the true command demonstrates command authority. (b) Terminal extension R^2 of recursive rollouts up to 5 s, shown separately for each cylinder so coincident high-accuracy traces remain visible.

5.4.1 Closed-Loop Calibration Against the Deployment Plant

The Simscape-trained ROM has a systematic extension offset when driving PhysX. The promoted checkpoint adds a bounded, command-gated correction:

$$\hat{x}_{i,k+1} \leftarrow \hat{x}_{i,k+1} + \delta_i(\boldsymbol{h}_k, \boldsymbol{c}_k), \qquad |\delta_i| \le 3\,\mathrm{mm}, \qquad \delta_i\big|_{\boldsymbol{c}_k=\mathbf{0}} = 0, \tag{5.9}$$

It is trained on 4 084 IsaacLab transitions from five reachable reference shapes, with Simscape replay. All hydraulic and integration parameters remain frozen. The gate sets the correction to zero for a commanded hold.

The calibrated command ROM serves as the deployment plant in Section 3.4.1. Unlike the ID–FD path, it has seen PhysX data. It is frozen and has no latent input; adaptation modifies only the inverse path.

5.5 The Privileged Latent and Its Limits

The system-dynamics model is an analysis model, not an actuator. It approximates the command-ROM–PhysX plant and tests whether a compact factor-dependent context improves prediction. It is trained from scratch on the IsaacLab data of Section 4.7.

At each 50 ms step, the model predicts the next 12-dimensional hydraulic-state increment from the observable state, joint context, consecutive commands $[\boldsymbol{u}_{k-1}, \boldsymbol{u}_k]$ and a six-tick causal belief. Training uses a five-transition unroll. The matched control SD_0 has the same structure but no latent input.

The privileged encoder $\boldsymbol{z} = \mu(\boldsymbol{e})$ maps the six physical factors to two dimensions and is trained jointly with SD_z. At a 20-tick horizon, the true latent is compared with zero, shuffled, random, mean and nominal controls using paired bootstrap intervals on validation and reserved-test runs.

The true latent reduces pooled rollout error by 7.20×10^5 on 20 validation runs (95 % interval $[-1.03 \times 10^6, -4.30 \times 10^5]$) and by 4.41×10^5 on 16 test runs ($[-7.61 \times 10^5, -1.74 \times 10^5]$).

The per-channel result is narrower (Table 5.3). The significant gain is confined to pressure; extension and velocity intervals cross zero on both pools. These are the channels used by the controller.

Figure 5.6 shows the two conditioning paths and the transition model. The privileged factor encoder is used during training; the belief GRU uses only causal state-action history.

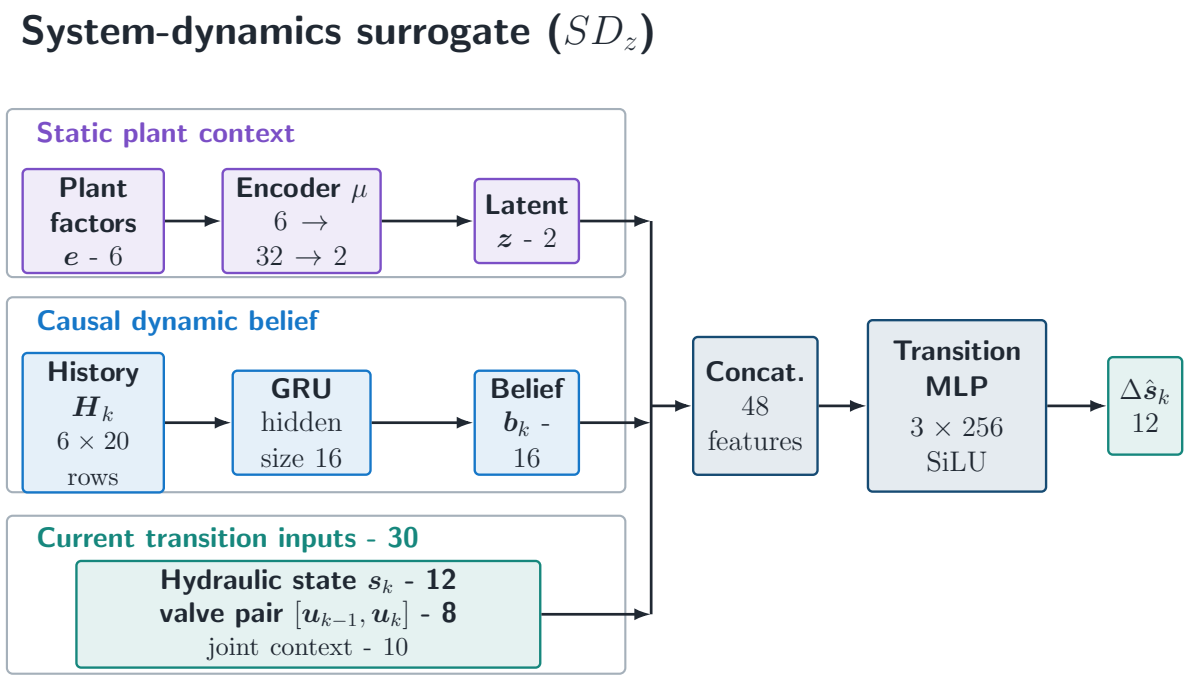


Figure 5.6: System-dynamics surrogate. The transition model combines a two-dimensional static plant context and a 16-dimensional causal dynamic belief with the hydraulic state, consecutive valve commands and joint context. A three-layer, 256-wide MLP predicts the transition; training uses a five-step unroll.

Table 5.3: Per-channel decomposition of the paired latent advantage at a 20-tick horizon, pooled over 36 runs. Values are the mean paired difference between the latent-conditioned model and the no-latent control, with a 95 % bootstrap interval; a negative value favours the latent for the pooled state error, and a positive value favours it for the per-channel errors as signed here. The advantage is confined to the pressure channel.

Channel	mean paired difference	95 % interval	excludes zero
extension	5.2×10^{-5}	$[-3.6 \times 10^{-5},\ 1.4 \times 10^{-4}]$	no
velocity	-2.4×10^{-4}	$[-7.1 \times 10^{-4},\ 2.4 \times 10^{-4}]$	no
pressure	1.75×10^{5}	$[2.1 \times 10^{4},\ 3.3 \times 10^{5}]$	yes

Thus the two-dimensional context contains held-out hydraulic information, but this test does not establish AAM accuracy or control improvement. It compares separately trained models; the within-model latent substitution in Section 7.6.2 isolates latent input and also finds extension and velocity sensitivity. The main limitation is that the strongest signal lies in pressure, which the controller does not integrate.

The encoder, SD_z, belief module, normalisers, input mask, split and rollout protocol are then frozen for downstream use.

5.6 Arm Adaptation Module

The AAM estimates $\hat{\boldsymbol{z}} = \mathrm{AAM}(\boldsymbol{H}_t)$ from deployment history. Each row contains five joint positions, five joint velocities, four cylinder pressure differences and four previous valve commands. A four-layer, width-64 causal convolution processes 20 rows (1.0 s, 360 values).

The pressure values are recursive actuator-model states, not measurements: they come from the command ROM in R and the frozen FD in AIF. The factor vector, true latent, reference, future rows and hidden state are excluded. Using the previous valve command keeps each row causal. Validation selected the shortest competitive history from 5, 10, 20 and 40 ticks. Latent RMSE is 0.802 on validation and 0.734 on reserved test.

Inference runs at 20 Hz inside the 60 Hz control loop. The adapter is bypassed until 20 rows are available. A causal five-prediction rolling mean adds 250 ms lag. Reported results use this online smoother, not the earlier non-causal full-run average.

5.6.1 Information Content of the Estimated Latent

Linear decoding from the per-run mean estimate gives $R^2 = 0.956$ for seal condition, 0.448 for compliance, 0.114 for flow, 0.081 for supply pressure, 0.056 for centre-of-mass offset and -0.008 for mass. The latent therefore represents mainly seal condition. It is stable within a run but does not refine over time.

Low-dimensional adapters can lose information when factors act through different channels [Liu 25]. This differs from the rapid-motor- adaptation assumption that only task-relevant factor directions need to be retained [Kumar 21]: here the dominant seal-condition direction acts through pressure, which the controller does not integrate. Estimating named physical quantities would improve interpretation but requires an explicit plant model [Hu 26].

5.7 Latent-Conditioned Inverse-Model Adapter

The frozen ID produces $\boldsymbol{u}_{\mathrm{base}} = \mathrm{ID}(\boldsymbol{s}, \boldsymbol{r})$. The adapter adds a bounded latent-conditioned residual:

$$\boldsymbol{u} = \mathrm{clip}\Big(\boldsymbol{u}_{\mathrm{base}} + g_\psi(\boldsymbol{s}, \boldsymbol{r}, \boldsymbol{u}_{\mathrm{base}}, \hat{\boldsymbol{z}})\Big). \tag{5.10}$$

Training minimises valve-command MSE at the exact ROM tick, with the residual bounded to 0.15. The ID, FD and policy are frozen during adapter training. The AIF policy is separately latent-conditioned, so disabling the residual restores the baseline actuator path but not the baseline actor. This provides the single-variable adapter ablation used in Chapter 7.

Figure 5.7 shows the causal convolutional stack and the bounded adapter that consumes its estimate. The upper part is the AAM; the lower part shows where adaptation enters the frozen inverse-model path.

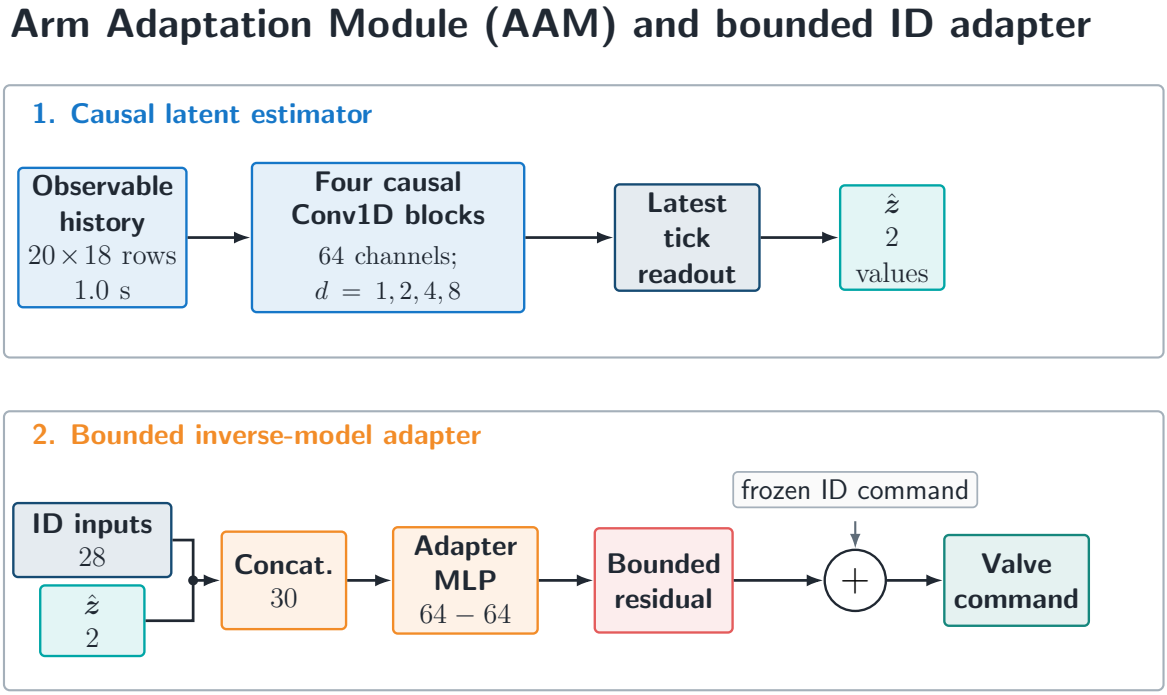


Figure 5.7: AAM and bounded ID adapter. Four causal convolutional blocks map a 20-step history of 18 observable features to the compact latent $\hat{z} \in \mathbb{R}^2$. The adapter combines this estimate with the ID inputs and adds a limited, bounded correction to the frozen valve command.

5.7.1 Open-Loop Evidence

The residual cap is 0.15, and the frozen ID already explains about 84 % of valve-command variance (Section 5.2.2). The adapter can therefore make only small corrections. Section 7.6.2 tests latent information without this ceiling.

Three matched adapter arms use a zeroed, estimated or privileged latent. Across five seeds, both latent arms improve validation loss over the zeroed arm in 5/5 cases. Mean paired improvements are 0.0037 for the privileged latent and 0.0118 for the AAM estimate. The effect is consistent but small.

Masking pressure and force does not strengthen the result: the privileged latent helps in only 2/5 seeds and variance increases by about one order of magnitude. The earlier single-seed improvement is not reproducible.

5.7.2 Closed-Loop Evidence and Its Bound

Closed-loop ablations compare the full stack with zeroed, shuffled, frozen and mean latents, plus a sham plant change. None consistently improves lateral error, cut-height error or impacts.

Valve saturation limits the test: cylinder 1 is saturated for about 92 % of the scored window, cylinder 2 for 67–82 %, and cylinder 4 for 50–57 %. Control-effort ratios before

and after the plant change are 1.005–1.009. A bounded additive correction has little authority under these conditions [Sofrony 24].

Reducing the lead cap lowers cylinder-1 saturation from 0.91 to 0.14 but triples cut-height error. The task is therefore close to the plant capability limit (Section 3.6.2).

5.7.3 Propagation of the Latent Advantage

The chain gate compares $SD_0(\boldsymbol{s}, \boldsymbol{u})$, $SD_z(\boldsymbol{s}, \boldsymbol{u}, \boldsymbol{z})$ and $SD_z(\boldsymbol{s}, \boldsymbol{u}, \hat{\boldsymbol{z}})$ under the frozen protocol with paired bootstrap intervals. Neither the true nor estimated latent improves extension or velocity over SD_0 with an interval excluding zero. The estimate is not worse than the true latent, so estimation is not the limiting stage. The measurable advantage remains in pressure (Table 5.3).

Section 7.6.2 finds the estimate comparable with the privileged latent within one model, confirming that the AAM is not the main loss. The missing link is a path from pressure information to the integrated extension command with sufficient actuator headroom. Chapters 7 and 8 examine this limit.

5.8 Training and Model-Selection Protocol

Adaptation data are split 82/20/16 into training, validation and reserved test by complete run and factor configuration; windows do not cross run boundaries. The 16 test runs are not used for model or hyperparameter selection. SD_z and SD_0 are selected on validation rollout, the AAM on downstream rollout, and the adapter on validation command error. Table 5.4 records the selected checkpoints.

Table 5.4: Provenance of each model in the adaptation chain. Frozen components are loaded unchanged and never receive gradients in this chapter.

Model	Status	Selected checkpoint
Control ID	frozen	paired-control, Section 5.2
Strict FD	frozen	strict-excitation, Section 5.3
Command ROM	frozen	PhysX-calibrated, Section 5.4.1
SD_0 (no latent)	scratch	epoch 58
SD_z + encoder	scratch	epoch 48, frozen after the gate
AAM	scratch	epoch 73, 1.0 s causal CNN
Latent adapter	scratch	epoch 116

The deployed adapter is trained and selected on recorded valve-command error. This avoids the objective mismatch of an earlier next-state-loss variant. Each component is evaluated on its stated training contract.

6. Control Framework and Simulation Integration

Chapter 5 left three actuator representations sitting on a bench, each with its own contract and its own offline evidence. This chapter puts a controller in front of them and turns them into three experiments.

Section 6.1 describes the classical controller, because it is where the competence comes from even though it never runs at deployment. Section 6.2 specifies the policy: what it sees, what it emits, what stops it from emitting something dangerous, and what it is paid for. Section 6.3 then explains how the first becomes the second, and reports a result about that transfer which changed the training protocol of this thesis. The individual stacks appear only after those three are fixed, because they differ in one place only.

6.1 The DLS Teacher and Its Two Roles

6.1.1 Scope and Role of the Controller

The classical controller occupies a narrower position in this architecture than its prominence suggests. There is no damped-least-squares layer underneath the policy at runtime. Nothing is added to the policy's output. If the policy emits zero, the arm does not move.

The DLS controller is nevertheless the reason the policies work at all, and it appears twice, both times at training or collection time and never at deployment (Figure 6.1). It is the *privileged teacher* whose behaviour the policies are distilled from (Section 6.3), and it is the closed-loop controller that drove the identification runs of Section 4.7. Calling it a nominal layer would misdescribe the deployed system and, worse, would make every reported result a property of a controller pair rather than of the policy.

6.1.2 Privileged Information

The teacher is *privileged* rather than better tuned: it reads two quantities that no deployable controller on this machine could obtain. The first is the exact terrain height directly under

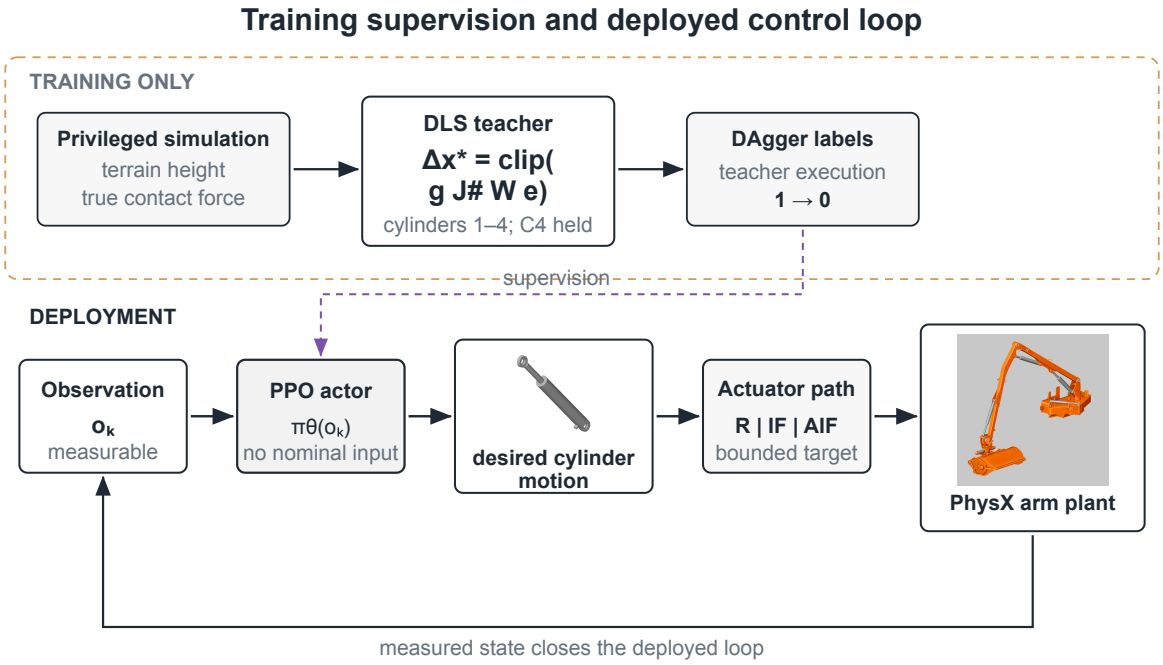


Figure 6.1: Training supervision and deployment. Simulator-only terrain and contact signals drive the DLS teacher, which supplies DAgger labels to the actor. At deployment only the lower feedback loop remains: measured observations pass through the PPO actor, the shared cylinder interface and the selected R, IF or AIF actuator path before PhysX updates the arm. No nominal command is added to the policy output.

the cutter, queried from the simulator rather than inferred from a forward- looking scan. The second is the true cutter–ground contact force. Together these turn the hardest part of the task, knowing where the ground actually is, right now, underneath a 1.2 m flail head that is occluding it, into a lookup.

The teacher uses that privilege in a specific way. Rather than commanding a height, it runs a contact regulator: it maintains a vertical offset relative to the ground and adjusts that offset to hold the measured contact force near 250 N within a 100 N deadband, pressing down below the band, backing off proportionally above it, searching downward when contact is lost and retreating immediately on a force spike. The accumulated offset is clamped to $[-0.30, -0.05]$ m so the regulator can neither drive the head into the ground nor lift it clear.

The teacher's advantage is therefore not kinematic. Its inverse is the same DLS solution available to anything; what it has is contact sensing. That is the quantity the student must infer from a noisy forward scan and its own contact history, and it is a large part of why distillation is a nontrivial transfer rather than a formality.

6.1.3 From Task Error to Cylinder Command

The rest of the teacher is conventional and is stated for completeness. A goal point is maintained in the truck frame: laterally it tracks the verge centre line, vertically it follows the measured ground height plus the contact regulator's offset. The task error is the

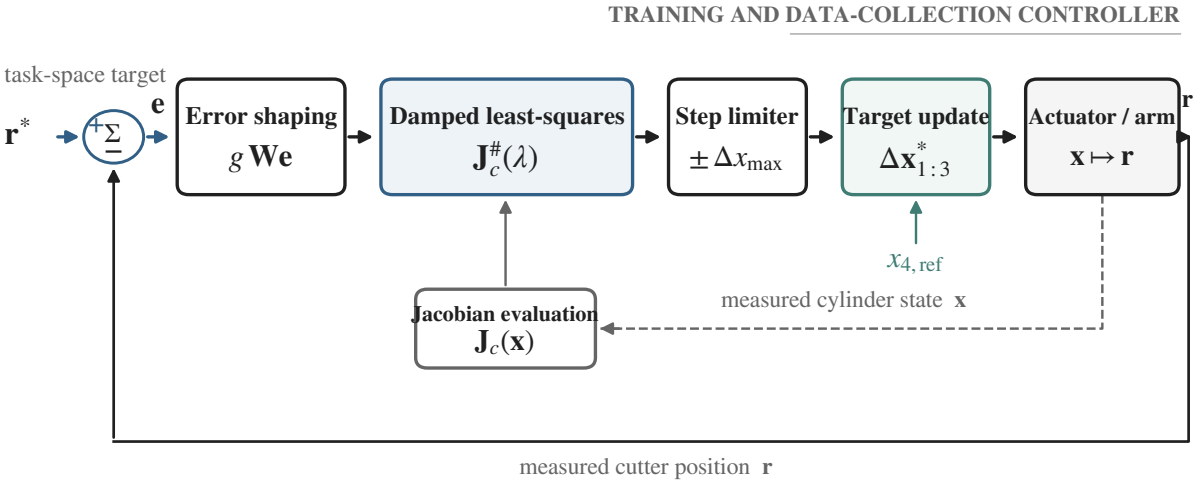


Figure 6.2: Feedback realization of the damped least-squares teacher of (6.1). The weighted task error is scaled by g, mapped to the first three cylinder coordinates by the measured damped inverse $\boldsymbol{J}_c^{\#}(\lambda)$, and clipped to $\pm\Delta x_{\max}$ per tick. Cylinder 4 is held at its head-down reference. Cutter position closes the task-space loop and the measured cylinder state updates $\boldsymbol{J}_c$ at every step. The loop is used only for teaching and data collection, never at deployment.

difference between that goal and the measured cutter position, with the lateral component weighted by a factor of three, because on this machine lateral error is what the acceptance gate measures and vertical error is largely handled by contact.

That error is mapped to cylinder space through the damped least-squares inverse of the measured cutter Jacobian $\boldsymbol{J}_c$ of Section 3.2.2,

$$\begin{aligned} \boldsymbol{J}_c^{\#}(\lambda) &= \boldsymbol{J}_c^{\mathsf{T}} \left(\boldsymbol{J}_c \boldsymbol{J}_c^{\mathsf{T}} + \lambda^2 \boldsymbol{I} \right)^{-1}, \\ \Delta \boldsymbol{x}^*_{1:3} &= \operatorname{clip}\Big(g\, \boldsymbol{J}_c^{\#}(\lambda)\, \boldsymbol{W} \boldsymbol{e},\ \pm \Delta x_{\max} \Big), \quad g = 0.30, \quad \Delta x_{\max} = 3\,\mathrm{mm}. \end{aligned} \tag{6.1}$$

with $\boldsymbol{W}$ the lateral weighting and the per-step clamp bounding how far any cylinder may be asked to move in one tick. Only cylinders 1–3 are solved. Cylinder 4 is held at the reference extension that keeps the head down, for the reason given in Section 3.1. The gain g is small on purpose: this is a stiff, heavy mechanism in contact, and a resolved-rate controller that closes quickly on the task error simply excites it.

6.1.4 Execution Through the Student Interface

One implementation detail determines whether distillation converges to something usable. The teacher's output $\Delta \boldsymbol{x}^*$ is a desired cylinder target, and it is *not* written to PhysX directly. It is converted into the same normalized four-value action the student emits and pushed through the student's entire action chain and into the same plant (Section 6.2), so every label the student is asked to imitate is by construction an action the student can produce. A teacher writing targets directly to the plant would generate labels outside the student's reachable set, giving the imitation error a floor no amount of training could remove. That failure mode was measured here: when an earlier action-rate limit sat below the teacher's own structural bound, 0.43 % of labels became unreachable by construction and acted as exactly such a floor.

Table 6.1: The privileged DLS teacher, measured on the stage-6 verge task through the command-ROM plant (128 environments, 480 steps, all eleven path families, seed 17). It is competent, not perfect, and several acceptance thresholds sit deliberately just outside what it achieves.

Quantity	Teacher	Gate
Mean $\lvert e_{\text{cut}} \rvert$	0.0234 m	≤ 0.040 m
p95 $\lvert e_{\text{cut}} \rvert$	0.1524 m	—
Mean $\lvert e_{\text{lat}} \rvert$	0.1221 m	≤ 0.300 m
Ground-contact fraction	0.7807	≥ 0.700
p01 head alignment	0.9638	≥ 0.900
Minimum head alignment	0.8317	—
p95 tool acceleration	64.9 m/s^2	≤ 120 m/s^2

6.1.5 Teacher Performance

A teacher is not an oracle, and quoting its performance is necessary because distillation cannot exceed it and the gates of Section 3.9 were anchored to it. Table 6.1 reports the privileged teacher measured on the stage-6 verge task through the ROM plant, over 128 environments and 480 control steps.

Two readings follow. First, the teacher is a competent but visibly imperfect controller: it loses ground contact about a fifth of the time and its cut-height tail is six times its mean. Second, and this is why Section 3.9 re-derived several thresholds, a gate set above these numbers would be a gate that the ground-truth controller itself cannot pass, which carries no information about the policy.

6.2 Policy Formulation

6.2.1 Policy Definition

The deployed controller is a memoryless Gaussian multilayer perceptron. It maps a 66-value observation to four normalized actions, one desired motion command per hydraulic cylinder, and that is the entire control law. Actor and critic are separate networks with hidden widths 256–128–64 and ELU activations, over empirically normalized observations. AIF is the one exception: its actor reads 68 values, the same 66 plus the two channels of $\hat{\boldsymbol{z}}$, so the latent conditions the policy as well as the inverse model. That is an architectural difference between the stacks rather than a tuning difference, and Section 6.9 states how the comparison accounts for it.

Three things it is *not* must be stated, because each was a design option that was considered and rejected. It is not a residual: there is no nominal command to correct. It is not recurrent: temporal inference in this architecture belongs to the AAM of Chapter 5, and giving the policy its own memory would confound the two. And it does not command the vehicle: the truck's forward motion is scripted, so the policy cannot solve a tracking problem by driving differently.

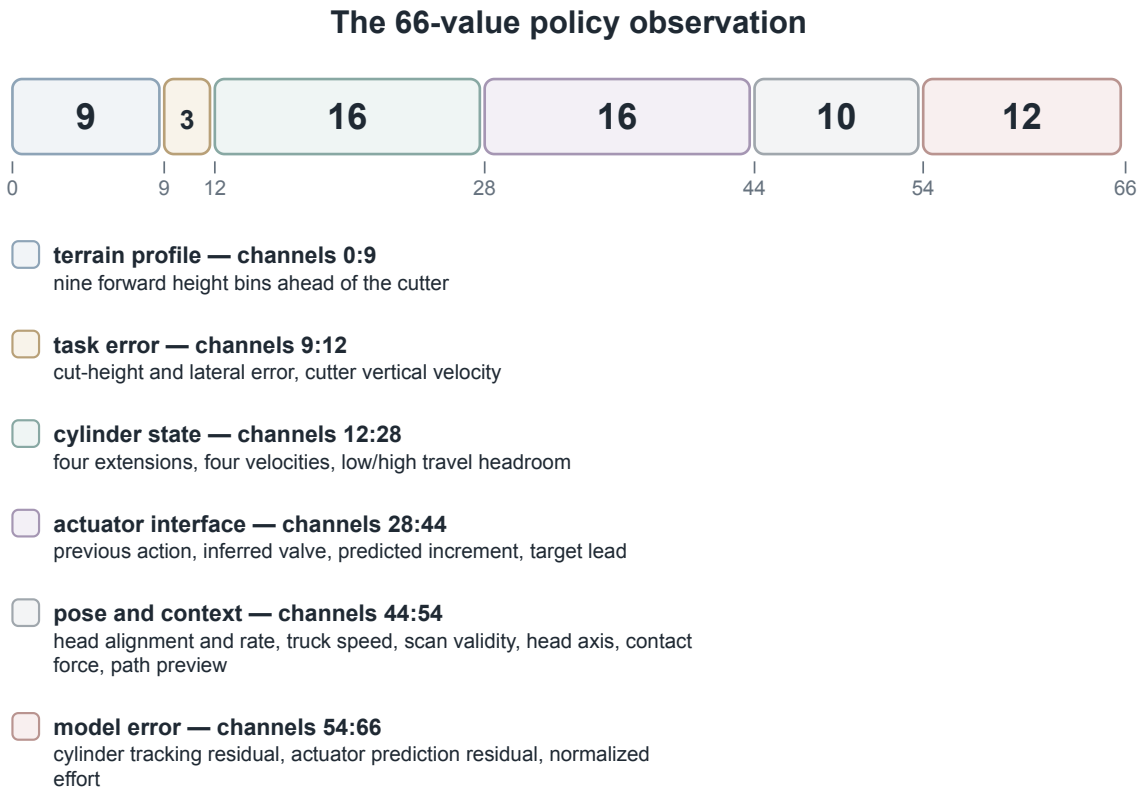


Figure 6.3: The 66-value policy observation, drawn to scale by group. The channels combine measurements, derived quantities and actuator-model estimates. No trajectory-family label, no physical factor vector and no true latent enters the actor. AIF appends the two estimated latent channels to this vector, giving 68, and is reported as the different architecture that it is.

6.2.2 The Observation Vector

Figure 6.3 lays out the 66 channels by group. The grouping tells the story of what this controller has to reason about: nine terrain-height samples ahead of the cutter, three task-error channels, sixteen channels of cylinder state and remaining travel, sixteen describing the actuator interface, ten of pose and context, and twelve of model error.

The last group deserves an argument, because twelve of sixty-six channels is a substantial fraction of the observation spent on something that is not the task. Those channels carry the actuator's standing error: the cylinder tracking residual, the actuator prediction residual, and the normalized cylinder effort. They exist as a direct consequence of the policy being memoryless. Forming an error requires differencing position across two steps and comparing against a prediction issued on the previous step, and a feedforward network can construct neither from a single observation, so without these channels the model error the policy is asked to compensate is not merely hard to see but absent from the input. This was measured rather than reasoned about: across ten training blocks the p01 head alignment sat at 0.79–0.80 and did not respond to changes in the feedforward source or the levelling gain, because the policy had no observable evidence of the error it was being asked to correct.

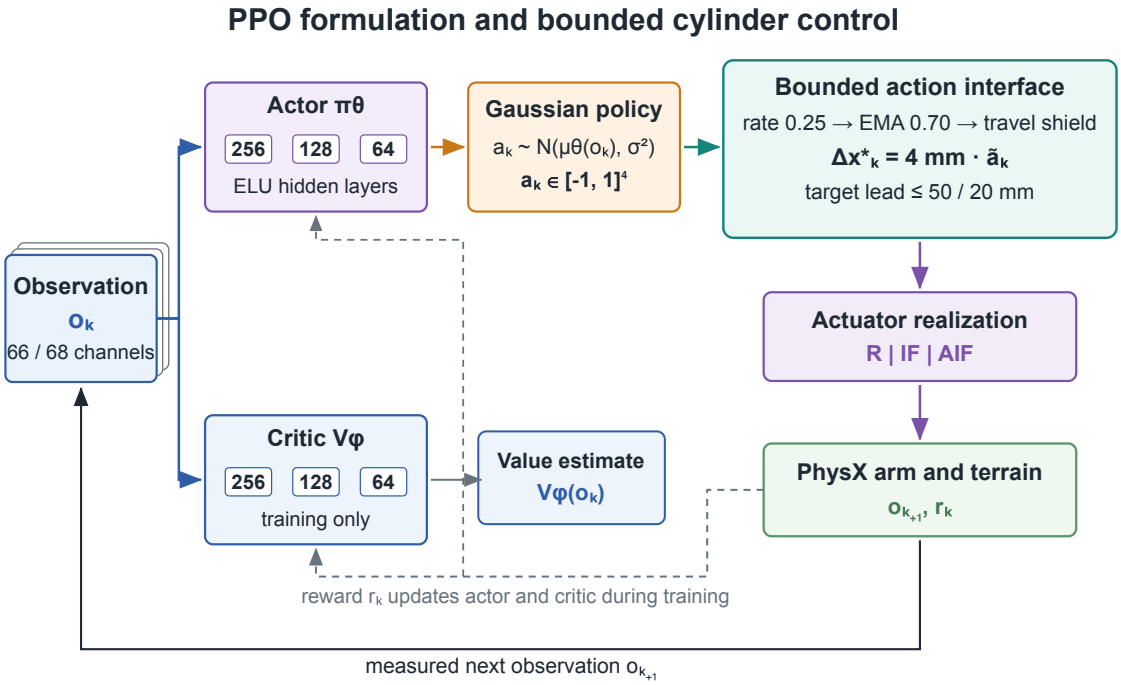


Figure 6.4: PPO actor–critic formulation and deployed control path. The actor maps the observation to a four-dimensional Gaussian action. Rate limiting, exponential filtering, the travel shield, command scaling and the target-lead cap bound that action before the selected R, IF or AIF actuator path drives the PhysX arm. The critic and reward update are training-only; measured state closes the deployed loop.

This group is also what makes the three stacks comparable. Under the ROM the tracking residual is near zero, because the ROM reaches what it is told. Under a learned ID–FD path it is the model's standing error. The same observation contract therefore gives each policy an honest view of its own actuator's shortcomings, which is what a fair comparison requires.

6.2.3 The Action and Its Bounds

The policy emits $\boldsymbol{a} \in \mathbb{R}^4$, sampled from its Gaussian at training time and taken at the mean for evaluation. Between that sample and the PhysX cylinder target sit five bounding stages. Figure 6.4 places them in the complete PPO actor–critic loop. The critic is used only during training; the actor and bounded action interface form the deployed controller. Each bounding stage can only *reduce* authority and cannot introduce motion the policy did not request.

The rate limit bounds $|\Delta \boldsymbol{a}|$ at 0.25 per control step and is held *constant* across every curriculum stage. It is not a curriculum knob, and the reason is the reachability argument of Section 6.1: the teacher's own command stream has a structural per-step bound near 0.20, so a student limit below that makes part of the label set unreachable. The exponential filter ($\beta = 0.70$ at the final stage) smooths the command. The limit shield scales the command down, and only down, and only in the direction that consumes headroom, over the last 20 mm of cylinder travel, which is the soft replacement for the teacher's hard clamp into the learned range. The command scale converts the filtered action into 4 mm

of cylinder travel per unit action. Finally the target integrator accumulates the increments in reference space, with the integrated target permitted to lead the measured extension by at most 18 mm.

That last cap is an anti-windup device and it is load-bearing. Relaxing it from 18 mm to 50 mm halved the lateral error on the two families demanding a sustained standing offset (`lane_change_hold` from 0.4833 to 0.2310, `boom_sweep` from 0.3745 to 0.1602), showing that the clamp was bounding the standing effort those paths need, while the same relaxation on cylinder 4 drove head alignment from 0.9507 to 0.4636. The final task therefore resolves the trade per axis, and this is the configuration every reported R result was produced under: 50 mm on cylinders 1 to 3 and 20 mm on cylinder 4.

Two properties of this chain follow. It is what replaces the residual band, since exploration is bounded by how much motion a single action can command rather than by proximity to a baseline. And it fixes the plant's capability ceiling: four millimetres of cylinder travel per step is the entire authority budget, which produces the 0.240 m/s lateral rate limit of Section 3.6.2 from which the reference paths are derived.

6.2.4 The Reward Function

The reward of the final R task has ten bounded terms, listed in Table 6.2. Three are positive and describe the task. Seven are penalties and describe how the machine must behave while doing it. All are bounded, and all retain a live gradient beyond the acceptance boundary, so that a policy already inside a gate still has a direction to improve in.

It replaces an inherited formulation carrying 53 weighted reward fields. Isaac Lab's own locomotion task uses eleven. The reduction was not cosmetic. At 53 terms the optimum is not something a designer can reason about, and three separate terms turned out to be working against the task without being noticed for thirteen training campaigns. The three rules below were each adopted after a measured failure.

No Gaussian Shaping

The earlier reward priced tracking as $w \exp[-(e/s)^2]$, which is effectively flat beyond about $2s$. Measured on the lateral term, at an error of 0.50 m the gradient is -2.07 at $s = 0.30$ but -0.004 at $s = 0.15$: a factor of 520 weaker where the failing path families live. Every shaping term is therefore linear or Huber in the error. Writing $x = |e|/s$, the two shapes used are

$$B(e; s, c) = \min(x,\, c), \qquad H(e; s, c) = \min\Big(c,\, \begin{cases} \frac{1}{2}x^2, & x \le 1 \\ x - \frac{1}{2}, & x > 1 \end{cases}\Big). \tag{6.2}$$

B has the same gradient at 0.5 m of error as at 0.05 m, so a policy far from the reference is still told which way to move; H buys quadratic precision near zero where cut height needs it, without a vanishing far field.

Bounded Terms

A term that can grow without limit can dominate the sum and destabilize the critic. Each contribution is clipped, which gives the reward a published budget: $6 + 12 + 4 = 22$ for the task terms against -24 for the safety terms. A non-finite value is replaced by -25, just below that worst case, so a numerical fault can never outscore legitimate behaviour.

Table 6.2: The ten reward terms of the final R task, with weights and shaping scales. B and H are the bounded shapes of Equation (6.2). The lateral and head terms each combine a coarse and a fine scale, so far-field correction and near-field precision are priced separately.

Term	Weight	Shape and purpose
Cut-height tracking	+6	H, scale 0.04 m, cap 2.5; the primary mowing-quality term
Lateral tracking	+12	B, coarse 1.50 m and fine 0.60 m mixed 0.6/0.4; the primary line-following term
Ground contact	+4	pays for the cutter being in contact
Head alignment	−12	B on the deficit below 0.98, fine scale 0.02 and coarse scale 1.00 mixed evenly
Contact loss	−3	charges directly for loss of contact
Action rate	−1	B on the mean absolute command change, scale 0.25; penalizes chatter
Tool acceleration	−1	B, scale 300 m/s^2; mechanical-wear proxy on the cutter
Contact force	−1	B on force above 30 kN, scale 150 kN
Cylinder headroom	−2	B on encroachment inside a 0.02 m margin, scale 0.10 m
Hard impact	−4	indicator on the impact event

Fixed Sign Convention

The three task terms are non-negative and the seven safety terms non-positive. This is what makes worst-case (CVaR) weighting safe to add later: scaling a non-positive term by a factor ≥ 1 can only lower the return, whereas up-weighting a *positive* reward for the worst environments pays the policy to be bad. Measured in the earlier stack, weighting the positive contact reward four-fold for the worst quarter of environments made a contact quality of 0.49 pay 7.84 against 2.00 for a quality of 0.50, and collapsed worst-family contact within a single training block.

Three terms of the earlier formulation are absent, and the reasons are results rather than preferences. The *action-magnitude* penalty on mean(a^2) was removed outright: a terrain action is a desired cylinder motion rather than a pose offset, so what the term actually charged for was the sustained offset that `lane_change_hold`, `offset_return` and `boom_sweep` structurally require. The policy correctly learned to return to neutral, which is the one thing "hold an offset" forbids. Removing it produced the first training block in a full session that did not degrade, improving the held-out score from 1.2058 to 0.3678. The *head barrier* was replaced, because in its earlier form the head group dwarfed lateral tracking and line following could never compete for the optimizer's attention. And the *clearance barrier* was disabled, since measured contact, cylinder headroom and cut-height error proved more reliable signals of the same condition.

One apparent inconsistency is deliberate. The tool-acceleration term saturates at 300 m/s^2 while the corresponding gate sits at 120 m/s^2, anchored to the measured physical floor of the stage-6 environment near 100 m/s^2. The reward scale is a saturation scale for a

bounded penalty rather than a target, so the gate remains the binding criterion and the reward's role is only to keep a gradient against gross excursions. The action-rate term is held mild for a related reason: cylinder 4 owns head pitch, so a rate tax falls hardest on the fast corrections that keep the head down as boom attitude shifts, and under a heavier tax the optimizer chose head drift over paying it.

Every term is logged individually and by name during training. The earlier stack logged an aggregate and a partial subset, with the two largest lateral terms not logged at all, which is part of why a reward term opposing the task survived thirteen campaigns. Reward weights on this system are measured trades rather than preferences, and several natural-looking choices are actively harmful. That is also why the reward is *not* the result criterion: promotion uses the eight physical gates of Section 3.9, and a policy may raise total reward while failing the thing the reward was written to encourage.

6.2.5 Optimization

Policies are optimized with clipped-surrogate PPO over generalized advantage estimation, in the form of Section 2.7. The maintained configuration uses clip parameter $\epsilon = 0.2$, discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, five learning epochs over four minibatches per update, an adaptive learning-rate schedule targeting a KL divergence of 0.02 from a base rate of 3×10^{-4}, gradient-norm clipping at 1.0, and a clipped value loss with coefficient 1.0. The terrain task collects 32 steps per environment per update. At 512 environments that is 16 384 transitions per update, chosen because the inherited 128-step horizon exceeded two minutes per update on the available GPU without adding useful batch diversity.

The final fine-tuning blocks depart from that configuration because they run under the reference anchor of Section 6.4 rather than an adaptive learning-rate schedule: clip $\epsilon = 0.1$, discount $\gamma = 0.97$, three learning epochs, fixed learning rates of 10^{-5} for the actor and 10^{-4} for the critic, entropy coefficient 0.001, an exploration standard deviation fixed at 0.15 and excluded from the gradient, and frozen observation normalizers.

Three of those departures are structural rather than tuning. The discount is reduced because the episode is long in simulation steps but locally controlled, so an action should be credited for correcting the terrain state it faces rather than for a return hundreds of steps later that the critic cannot resolve. The actor learning rate sits an order of magnitude below the critic's so that the actor is deliberately the slow variable and trust-region budget is not spent on steps driven by a value function that has not yet converged. And the adaptive schedule is dropped because the anchored formulation already constrains the update through an explicit divergence bound; two competing step-size controllers would make neither attributable.

The exploration settings changed for a reason specific to this architecture. A residual policy explores around a controller that already tracks, whereas here a zero action means the arm does not move at all, so the initial standard deviation was raised from 0.05 to 0.30 and the entropy coefficient with it, to let the policy discover the command directions before refining them. On the terrain task, where distillation supplies a competent starting point, both are reduced again and the standard deviation is *held fixed* rather than initialized, for a further reason taken up in Section 6.4. Exact values for any reported run must be read from that run's frozen manifest, because they changed during development.

6.3 Training Protocol and the Distillation Finding

6.3.1 Distillation by DAgger

Policies are initialized by imitation, not by reward. The privileged teacher of Section 6.1 is rolled out in the student's own environment, and the student is fitted to the teacher's raw action by regression, in the DAgger scheme of Ross et al. [Ross 11]: the teacher's probability of *executing* the action, as opposed to merely labelling it, is annealed from one to zero over the first 70 % of iterations, so the later part of training collects states the student itself visits rather than states the teacher would have visited. Labels are accumulated in a replay buffer of 262 144 transitions and refitted over three epochs per iteration in minibatches of 8 192.

Figure 6.5 shows a complete distillation run for the ROM stack, taken from the stored record. Panel (a) is the handover schedule. Panel (b) is the imitation error: the behaviour-cloning loss falls by more than an order of magnitude while the teacher is still driving, reaches its minimum around the handover, and then *rises* once the student is alone. That rise is not a failure. It is the distribution shift DAgger exists to expose: the student is now visiting states the teacher never chose, and the teacher's advice on those states is new information rather than a repetition of what has already been fitted. Panel (c) shows the corresponding closed-loop task metrics. They remain near the teacher's preflight values without matching them.

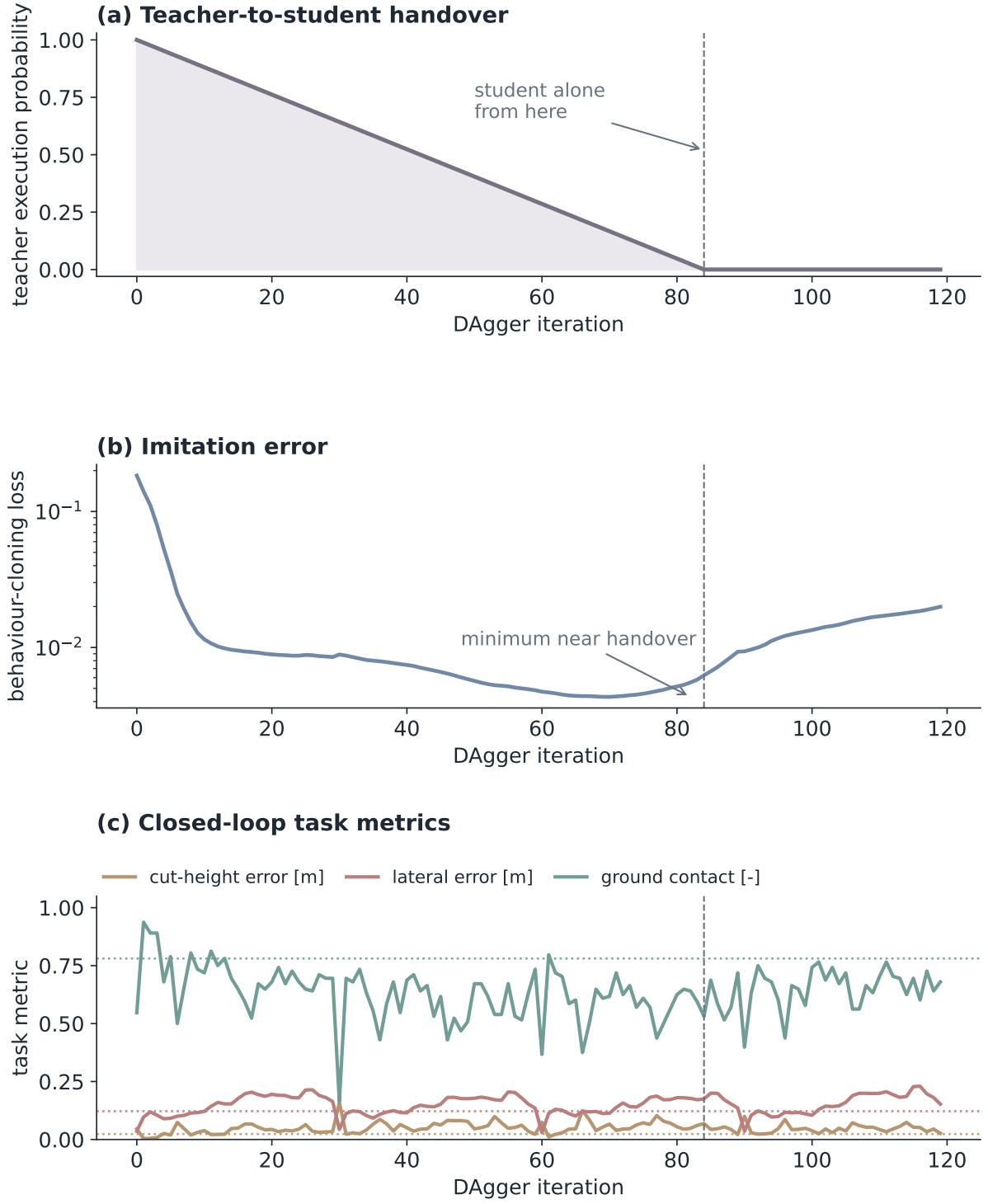


Figure 6.5: A measured distillation run for the R stack, from the stored record. (a) The teacher's execution probability is annealed to zero over the first 70 % of iterations. (b) Behaviour-cloning loss falls by over an order of magnitude under teacher execution, then rises once the student is alone, the distribution shift that DAgger is designed to surface, not a regression. (c) Cut-height error, lateral error and ground-contact fraction; dotted lines mark the teacher's preflight values from Table 6.1.

6.3.2 Objective Mismatch in Unconstrained Fine-Tuning

The natural next step is to treat the distilled actor as an initialization and improve it with PPO. That is what the residual literature does, it is what the plan for this thesis originally assumed, and in unconstrained form, on this task, it does not work. The finding is reported here in full because it is what motivates the anchored formulation of Section 6.4, which is how the final policies are trained.

PPO fine-tuning was run repeatedly from distilled checkpoints, and the outcome was consistent enough to be treated as a property of the problem rather than as bad luck. The critic was healthy throughout, with explained variance in the 0.85–0.95 band, so the failure is not a value-estimation failure. Total reward rose. And the primary tracking objective degraded, in the worst case by a factor of 12.9.

The mechanism follows from placing the reward and the gates side by side. The reward pays for ground contact and cut height as well as lateral tracking, and the optimizer discovers that it can buy the first two by spending the third. That trade increases total reward and moves the policy away from the gate that decides acceptance. The reward's optimum is simply not where the tracking gate is. Four earlier fine-tuning campaigns on a related configuration failed the same way in a different currency: every block degraded head alignment, with p01 falling from 0.9854 to the 0.48–0.78 range while safety terminations rose from 1.3 % to 3.6–7.7 %.

This is a claim about two functionals over the same policy space rather than about the optimizer. Training maximizes the expected return $J(\vartheta)$, while acceptance is decided by the vector of physical gates $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$ of Section 3.9. Writing $\mathcal{F} = \{\vartheta : \boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star\}$ for the feasible set, the measured content of the finding is

$$\arg\max_{\vartheta} J(\vartheta) \notin \mathcal{F}, \qquad \text{while} \qquad \vartheta_{\text{distilled}} \in \mathcal{F}. \tag{6.3}$$

Distillation lands inside $\mathcal{F}$ because the teacher was constructed to satisfy the gates, not to maximize J. Unconstrained ascent on J from that point therefore leaves $\mathcal{F}$ by construction. This is the optimizer working correctly on the wrong functional, and it is why the remedy is a constraint on the ascent rather than a repair of the reward.

Two consequences follow, and both are protocol rather than interpretation. Unconstrained fine-tuning is inadmissible, and both the pre- and post-fine-tune checkpoints are reported in every case; the matching rules that this imposes on the three-stack comparison are stated in Section 6.9. And reward decomposition, critic explained variance, policy entropy and the gate metrics themselves are logged *during* training rather than only at the end, because the whole point is that total reward went up while the system got worse. A training curve reporting only the scalar reward would have shown a success.

The general form of the result applies beyond this machine. When a competent teacher exists and the acceptance criterion is not the reward, reward-based fine-tuning is not automatically an improvement step: PPO optimizes the written objective, that objective is a proxy, and the teacher was constructed to do the task. Distillation transfers the intent; PPO then optimizes the proxy, and here the two point in measurably different directions. Since that divergence grows with distance travelled from a policy where proxy and intent were measured to agree, the remedy is not to abandon reward optimization but to bound the distance explicitly, which is the subject of the next section.

6.4 Reference-Anchored PPO

Equation (6.3) says that reward ascent leaves the feasible set. It does not say that reward ascent is useless, only that it is unbounded. The construction of this section keeps PPO and constrains how far it may travel from a policy already known to satisfy the gates. Writing π_{ref} for a frozen reference policy and J for the expected return, each training block solves

$$\max_{\vartheta} J(\vartheta) \quad \text{s.t.} \quad \mathbb{E}_{\boldsymbol{o}\sim d_{\pi_\vartheta}}\Big[\mathrm{KL}\Big(\pi_{\text{ref}}(\cdot\,|\,\boldsymbol{o})\,\|\,\pi_\vartheta(\cdot\,|\,\boldsymbol{o})\Big)\Big] \le \delta_{\text{ref}}, \tag{6.4}$$

with the expectation taken over the states the policy itself visits.

6.4.1 Critic Warm-Up

A resumed checkpoint carries an actor ϑ and a critic ψ. After distillation the actor already behaves sensibly, but the critic has never been fitted to the long terrain return, so the first PPO updates would move a useful actor using a value estimate that is not yet meaningful. The actor is therefore frozen for 150 iterations while only the value loss is optimized, under the asserted invariant $\vartheta_{\text{after}} = \vartheta_{\text{before}}$. The observation normalizers are frozen at the same point: without that, the same raw observation is normalized differently from one iteration to the next, and the actor's behaviour changes while its weights do not.

6.4.2 The Reference Anchor

Let π_{ref} be a frozen copy of the accepted warm actor. It is queried only on states the live policy visits and never supplies an executed action, so PPO remains the sole controller of the plant. The penalty is the *forward* divergence, which for diagonal Gaussians is available in closed form,

$$D_{\text{ref}} = \mathrm{KL}\Big(\pi_{\text{ref}}\,\|\,\pi_\vartheta\Big) = \sum_i \left[\log\frac{\sigma_{\vartheta,i}}{\sigma_{\text{ref},i}} + \frac{\sigma_{\text{ref},i}^2 + (\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2\sigma_{\vartheta,i}^2} - \frac{1}{2}\right], \tag{6.5}$$

at the cost of one additional forward pass and no sampling. Because both scales are fixed at $\sigma = 0.15$, this collapses to a pure quadratic on the drift of the commanded action,

$$D_{\text{ref}} = \sum_i \frac{(\mu_{\text{ref},i} - \mu_{\vartheta,i})^2}{2(0.15)^2}, \tag{6.6}$$

which is the second reason for fixing σ: with a free standard deviation, entropy collapse shrinks D_{ref} while the commanded action moves, so the constraint would report compliance during the drift it exists to prevent.

The direction is load-bearing. Forward KL is mass-covering: it charges π_ϑ wherever π_{ref} has density and π_ϑ does not, and therefore penalizes the *abandonment* of reference behaviour. The reverse direction is mode-seeking and would permit the policy to collapse onto a subset of reference behaviour at no cost, which is the observed failure, in which one competence was dropped to buy another. It follows that the term is a behaviour-cloning anchor evaluated on the policy's own state distribution, rather than the reverse-KL regularizer of soft reinforcement learning. Its lineage is demonstration-regularized policy gradient [Rajeswaran 18], not the Boltzmann-optimum family.

The actor objective is

$$\mathcal{L}(\vartheta) = \mathcal{L}_{\text{PPO}} + \mathcal{L}_V - \beta\,\mathcal{H}(\pi_\vartheta) + \lambda_{\text{ref}}\,\mathbb{E}_k\Big[\mathrm{KL}(\pi_{\text{ref}}\|\pi_\vartheta)\Big], \qquad \lambda_{\text{ref}} = 0.10. \tag{6.7}$$

6.4.3 Trust Regions and Rollback

A penalty is not a constraint: at a fixed coefficient, a sufficiently large advantage always buys its way past. Feasibility is therefore also verified on the update taken. Two bounds are imposed,

$$D_{\text{update}} = \mathrm{KL}(\pi_{\text{old}} \| \pi_\vartheta) \le 0.01, \qquad D_{\text{ref}} \le 0.05, \tag{6.8}$$

and they do different work. D_{update} is the classical step-size control that keeps the surrogate a valid local model of J. D_{ref} is the new object: because the anchor is *fixed* within a block rather than moving with the policy, it bounds cumulative drift. A sequence of updates each individually satisfying D_{update} can still walk arbitrarily far from feasibility, and that is the trajectory the earlier campaigns took, monotonically, with the reward rising the whole way. Only a constraint referred to a fixed feasible point can observe it.

Enforcement is in two stages. During minibatch optimization the epoch is early-stopped as soon as D_{update} exceeds its bound. After the update both divergences are recomputed over the full rollout. If either bound fails, or any quantity is non-finite, the actor and its optimizer state are restored *atomically*. On a divergence rejection the critic fit is retained when finite, since the bounds constrain the policy and not the value function, and rewinding valid critic fits computed on the same on-policy rollout was wasting most batches near the boundary. On a non-finite update both networks are restored. In the final block, five of one hundred logged updates were rejected, all of them on the reference bound and none on the update bound. That is, the drift caught was of the kind that ordinary PPO clipping does not see.

6.4.4 The Block Ratchet

The reference is not fixed forever. Within a block it is frozen. When a checkpoint is scored and passes the gates, it is promoted to reference,

$$\vartheta_{\text{ref}}^{(m+1)} = \vartheta^\star, \qquad \vartheta^\star \in \mathcal{F}, \quad \vartheta^\star \in B_{\delta_{\text{ref}}}\big(\vartheta_{\text{ref}}^{(m)}\big), \tag{6.9}$$

and the actor's optimizer moments are reset, because momentum accumulated relative to the old anchor is not meaningful relative to the new one. The critic's are retained: it is fitting the same return, and its calibration is expensive to rebuild.

This is the structural point of the construction. The policy is at all times within a bounded divergence of a checkpoint that has been verified against $\boldsymbol{G}$, and the anchor advances only through verified checkpoints. The acceptance criterion therefore enters the training loop instead of appearing only at the end. The optimizer still ascends the proxy. It simply can never travel far from the last point at which proxy and intent were measured to agree. In this construction distillation is no longer the endpoint: it is $\vartheta_{\text{ref}}^{(0)}$.

Algorithm 6.1 summarizes the procedure in five stages. The first two produce the initial verified anchor. Stages 3 and 4 repeat for each PPO block, and Stage 5 returns the last promoted checkpoint.

6.4.5 Theoretical Status

The components are standard and citable. The bound on D_{update} is conservative policy iteration and its trust-region descendants [Kakade 02, Schulman 15]; the atomic rollback

Algorithm 6.1: Five-stage reference-anchored PPO procedure. The teacher is used only in Stage 1. All PPO rollouts are controlled by π_ϑ alone, and the frozen reference moves only when a checkpoint passes every gate.

input : teacher π_T; gate test $\boldsymbol{G}(\vartheta) \preceq \boldsymbol{g}^\star$; M PPO blocks; anchored loss and divergence bounds from Equations (6.7) and (6.8)
output : last promoted actor $\vartheta^\star$ satisfying the gate test

`// Stage 1 - Distil a feasible actor`
$\mathcal{D} \leftarrow$ DAgger states labelled by teacher mean action $\boldsymbol{y}^T$
$\vartheta \leftarrow \arg\min_\vartheta \mathbb{E}_{(\boldsymbol{o},\boldsymbol{y}^T)\sim\mathcal{D}}\left[\|\mu_\vartheta(\boldsymbol{o}) - \boldsymbol{y}^T\|_2^2\right]$
remove π_T from the controller `// all later rollouts are teacher-free`
`// Stage 2 - Warm the critic and set the first anchor`
freeze ϑ and the observation normalizers
for $j \leftarrow 1$ **to** 150 **do**
  roll out π_ϑ and set $\psi \leftarrow \psi - \eta_{\text{critic}} \nabla_\psi \mathcal{L}_V(\psi)$
end
verify $\vartheta_{\text{after}} = \vartheta_{\text{before}}$
$\vartheta_{\text{ref}} \leftarrow \vartheta$; $\vartheta^\star \leftarrow \vartheta$ `// initial verified anchor`
for *block* $m \leftarrow 0$ **to** $M - 1$ **do**
  `// Stage 3 - Train inside the frozen reference bound`
  freeze ϑ_{ref} for the whole block
  for *each PPO iteration in block* m **do**
    collect an on-policy rollout with π_ϑ as sole controller
    compute and normalize GAE; save snapshot $\mathcal{S}$; set $\vartheta_{\text{old}} \leftarrow \vartheta$
    for 3 *epochs over the rollout minibatches* **do**
      if $\text{KL}(\pi_{\text{old}} \| \pi_\vartheta) > \delta_{\text{upd}}$ **then** stop the update epochs
      update actor and critic using Equation (6.7)
    end
    compute $(D_{\text{upd}}, D_{\text{ref}})$ on the full rollout `// full rollout`
    if D_{upd} *or* D_{ref} *is non-finite* **then**
      restore the complete snapshot $\mathcal{S}$
    else if $D_{\text{upd}} > \delta_{\text{upd}}$ ***or*** $D_{\text{ref}} > \delta_{\text{ref}}$ **then**
      restore the actor and its optimizer moments from $\mathcal{S}$; retain the new critic fit
  end
  `// Stage 4 - Evaluate the block and promote a passing checkpoint`
  score every saved checkpoint on at least three held-out terrain meshes
  aggregate each gate to the worst mesh and, for tracking gates, the worst path family
  $\mathcal{C} \leftarrow \{\vartheta_c : \boldsymbol{G}(\vartheta_c) \preceq \boldsymbol{g}^\star\}$
  if $\mathcal{C} \neq \varnothing$ **then**
    choose $\vartheta^\star \in \mathcal{C}$ using the fixed checkpoint selection rule
    load $\vartheta \leftarrow \vartheta^\star$ and set $\vartheta_{\text{ref}} \leftarrow \vartheta^\star$
    reset the actor optimizer moments; retain the critic state
  else
    leave ϑ_{ref} unchanged; do not promote the block
  end
end
`// Stage 5 - Return the deployable policy`
return $\vartheta^\star = \vartheta_{\text{ref}}$

is that literature's backtracking line search reduced to a single trial point; the penalty in Equation (6.7) is a Lagrangian relaxation of the constrained problem (6.4) evaluated at a fixed multiplier, with the rollback supplying the feasibility enforcement a fixed multiplier cannot.

What does not follow is a certificate. Bounding the divergence from a verified policy does bound gate degradation in form: the chain rule of KL over trajectories, Pinsker's inequality and the boundedness of every gate metric give $|\mathbb{E}_{\pi_\vartheta}[g] - \mathbb{E}_{\pi_{\text{ref}}}[g]| \leq 2G_{\max}\sqrt{H\delta_{\text{ref}}/2}$ for a metric g with $\|g\|_\infty \leq G_{\max}$ over a horizon H, a bound available only because the reward of Table 6.2 is bounded term by term and one that an exponential term would destroy. At the production horizon of 2808 steps and $\delta_{\text{ref}} = 0.05$, however, the right-hand side exceeds eight, which for a metric bounded in $[0, 1]$ is vacuous. There is also a measure mismatch, since the constraint is taken under the live policy's state distribution while the chain rule requires the reference's.

The honest claim is therefore narrower. The construction is a principled restriction of reward ascent to a neighbourhood of a policy where proxy and acceptance criterion were measured to agree, assembled from standard trust-region and demonstration-regularized results, with feasibility re-established empirically at every block boundary. The gates, not the theory, certify each promotion.

6.5 R: Reference Actuator Baseline

R is the reference condition of this thesis. The policy emits desired cylinder motion, the frozen command ROM converts it to a cylinder position target, and PhysX closes the loop. There is no online adaptation anywhere in the stack. The truck is scripted over a 26 m strip at one constant speed drawn per episode from 2 to 7 km/h, the eleven path families of Section 3.6 are distributed across environments within a single run, and the terrain mesh is drawn per environment so that no policy can memorize one ground surface.

Its status is that of a reference condition rather than a competitor, as developed in Chapter 7: the command ROM is not a pure Simscape artefact but a Simscape-trained model with a bounded closed-loop calibration against IsaacLab and PhysX, and ROM-plus-PhysX *is* the deployment plant. R's actuator therefore coincides with the plant. Its tracking advantage over IF is consequently not evidence that a reduced order model approximates the actuator better than an identified ID/FD pair: one of the two defines the target.

The policy is produced by the three-layer construction of this chapter: distillation from the privileged DLS teacher to a feasible student, critic warm-up with the actor frozen, and reference-anchored PPO in blocks under Equations (6.7) and (6.8). The final block produced checkpoints 280 to 329. The last iterate was not taken, because PPO can raise mean return while damaging a single path family. Candidates were scored instead on held-out meshes against the eight gates, aggregated to the worst mesh and, for the tracking-critical gates, the worst path family.

The gate table for that selected checkpoint is deliberately not printed here, and the reason is a limitation rather than an omission. The numbers on record were measured on the superseded 16 m rollout, and the checkpoint's own selection record marks them provisional for exactly that reason; that checkpoint's own full-length six-mesh revalidation was never

run. Quoting a certified gate result for R on the strength of a shorter rollout would be the kind of claim this thesis is written to avoid, so R is described here as a reference *condition* whose acceptance evidence is pending, and Chapter 7 reports the revalidated figures for all three stacks under one protocol.

Three separate records of R acceptance exist in the project, and because they disagree it is worth stating plainly which one this thesis relies on. The pipeline's own documentation reports all eight gates passing across the 2 to 7 km/h range on three meshes, which is a selection-time record for a different checkpoint than the one described above. An earlier six-mesh evaluation of the rebuilt pipeline reports seven gates passing and *ground contact failing*, at 0.6486 against the 0.700 threshold and a margin of −0.0514, which was at the time the one genuine performance gap in the stack. Neither record certifies the checkpoint this chapter selects, and neither is the evidence used in Chapter 7: the closed-loop comparison there re-scores a single checkpoint per stack under one protocol, at one speed, over the full path, on the same meshes and the same eleven families. The earlier ground-contact failure is still the right expectation to carry into that chapter. It is measured on a worst-mesh, worst-family aggregation, which is the statistic that the pooled per-cell means of Table 7.4 do not report, and it identifies ground contact on the ROM plant as the gate most likely to bind, a prediction Chapter 7 is able to check rather than assume.

6.6 IF: Non-Adaptive Learned Baseline

IF subclasses the R task and changes exactly one thing. The policy still emits desired cylinder motion, but that command is now realised by the frozen inverse model of Section 5.2, which converts it to a valve command, followed by the frozen strict forward model of Section 5.3, which converts the valve command to a predicted extension increment. PhysX receives the same bounded cylinder target as before. Task, terrain families, meshes, observation contract, action chain, reward, gates and the PPO update are inherited unchanged, so the actuator representation is the only free variable between the two stacks.

The stack matters to the argument twice over. It is the robust learned baseline against which AIF must be measured, since the two differ only by the AAM and the adapter, and it is the honest control for R, whose actuator coincides with the deployment plant.

Robust is used here in the sense of Section 2.6: the stack was trained across the randomised plant family and applies one fixed response to whichever member of that family it meets, performing no online estimation of plant characteristics and conditioning on no plant identity. It does not mean the policy is blind to plant variation. The twelve model-error channels of Section 6.2 carry the actuator's standing residual, and that residual is itself plant-dependent: under a given plant the actuator misses its commanded increment by a characteristic amount, and the policy acts on that miss. What the stack lacks is not information about the consequences of plant variation but any representation of *which* plant produced them.

Two properties of the composed path shape its behaviour, and both were established offline. The valve command is accumulated rather than issued absolutely and is clipped to the per-axis limits $[0.95, 0.95, 0.80, 0.80]$, so the actuator path carries its own saturation boundary independently of the action chain. And only the predicted extension increment

is integrated, since the forward model's velocity output failed validation on two axes (Section 5.3) and is carried as a diagnostic. This stack's standing model error is therefore visible to the policy through the twelve model-error channels of Section 6.2, which is what makes the shared observation contract fair across actuators rather than merely identical.

The promoted policy's run manifest records all eight gates passing across the 2 to 7 km/h speed range on three held-out meshes. That is the selection-time evidence on which the checkpoint was promoted, not a certified result: it was measured on three meshes rather than the six the final claim uses, and it is superseded by the single-protocol revalidation of Chapter 7, where the per-stack tracking numbers belong. One protocol point belongs here: the reported policy was obtained under the reference-anchored construction of Section 6.4 with a lower actor learning rate than R and a fixed exploration standard deviation of 0.05, which is read from the run manifest rather than assumed to match.

6.7 AIF: The Adaptive Stack

AIF adds online adaptation to the IF path and changes nothing else about the plant. Figure 6.6 compares the complete R, IF and AIF signal paths. At every model tick the AAM of Section 5.6 reads the observable history window and produces $\hat{\boldsymbol{z}}$, which enters AIF in two places: it is appended to the policy observation, and it conditions the bounded adapter that corrects the frozen inverse model's valve command. The forward model, the base inverse model and the command ROM are untouched.

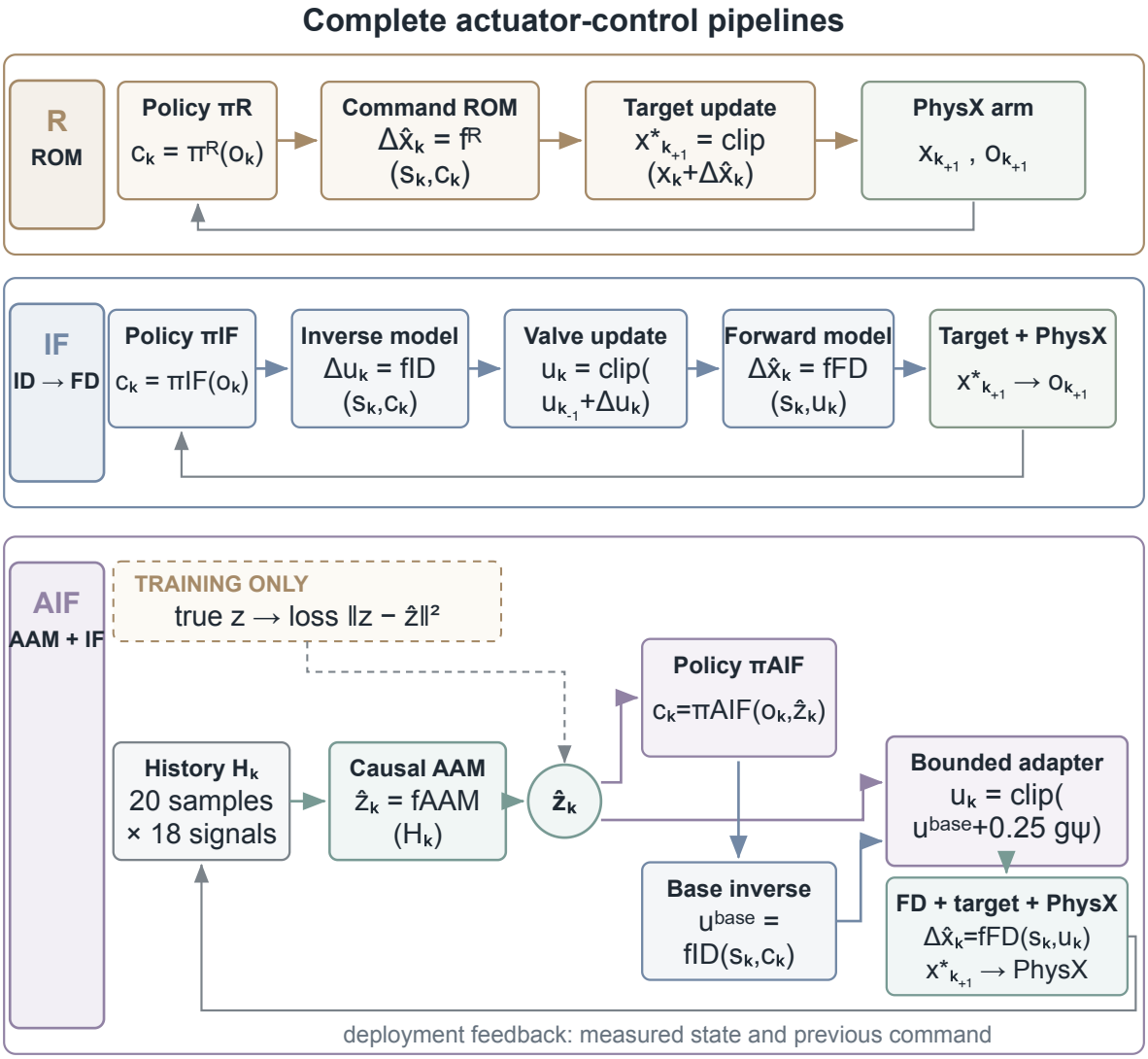


Figure 6.6: Complete R, IF and AIF control pipelines. R maps policy commands through the command ROM. IF composes the inverse and strict forward models. AIF adds a causal history encoder whose estimate $\hat{z}$ conditions both the actor and the bounded inverse-model adapter. The true latent is a simulation-only training target and is absent from deployment. All three paths produce the same bounded cylinder target for the PhysX arm.

Conditioning the policy as well as the adapter is a deliberate design choice and it is the one place where this stack is not a drop-in substitution. The research question is whether an inferred description of the current plant improves closed-loop behaviour, and a latent that reached only the inverse model could change the realised valve command but never the intent behind it. The cost is that the actor is a 68-input network rather than a 66-input one, which is an architectural difference and is treated as one in Section 6.9.

The adapter itself is deliberately weak. It adds a residual to the base valve command, the residual is bounded by construction and further scaled by a declared factor of 0.25, and the result is clipped to the same per-axis valve limits as IF. Three fallbacks make the degradation path exact rather than approximate. Until twenty complete history rows have accumulated the adapter is bypassed and the frozen inverse model's command is

returned unchanged; a non-finite latent triggers the same bypass; and the residual scale can be set to zero at evaluation time without touching any other part of the stack. In all three cases the *actuator path* is IF exactly, which is what makes the residual ablation a single-variable comparison. The actor is not restored by any of the three: it remains the 68-input latent-conditioned network. This stack therefore never reduces to IF as a whole, and the across-stack contrast in Chapter 7 is read with that in mind.

The estimate is not instantaneous, and the latency is part of the contract rather than an implementation detail. The AAM updates on the 20 Hz model-tick grid and its output passes through a past-only rolling mean over five predictions, so a plant change reaches $\hat{\boldsymbol{z}}$ no earlier than 250 ms after it becomes visible in the history window, on top of the 1.0 s window itself. Section 5.6.1 showed the deployed estimate to be a near-static seal-condition fingerprint, so this latency bounds a quantity that does not, on the evidence available, move much within an episode.

What this section does *not* claim is that the added machinery improves the task. The stack is complete, its interfaces are verified, and its manifest records the eight gates passing on six held-out meshes at selection time; whether the adaptation path contributes anything measurable is a separate question, settled in Chapter 7, and Section 5.7 has already reported that the closed-loop contrasts are null and why.

6.8 IsaacLab Integration and Timing

Four rates run concurrently and Table 3.4 lists them: PhysX integrates at 120 Hz, the policy acts at 60 Hz under a decimation of two, the learned actuator models advance on a 20 Hz grid of 0.05 s model ticks, and the AAM updates on that same grid. No single one of these is "the control frequency", and reporting a scalar rate for this system would misdescribe every one of the three stacks.

The relationship between the 60 Hz and 20 Hz layers is where integration errors hide, so it is fixed by construction rather than by interpolation. A learned actuator model is evaluated on every third control step and its predicted increment is held across the two intervening steps. The action presented to the model is the action of the tick on which it fires, not an average, and the target integrator accumulates in reference space so that a held prediction cannot re-anchor to a sagging measured position. Chapter 3 records why that reference-space integration is mandatory: integrating against the measured extension turns a static hold into a slow ratchet.

Order within a tick is fixed and asserted at run time. The AAM consumes the history window ending at the previous tick and produces $\hat{\boldsymbol{z}}$ before the inverse model runs, so no quantity computed at the current tick can influence its own estimate. History buffers are per-environment and cleared on episode boundaries, which matters because the vectorised environments reset asynchronously and a shared buffer would leak one episode into the next. Both properties are checked rather than assumed: a runtime check replays a trace and asserts bit-exact agreement between the online estimate and an offline causal recomputation, and truncating the future of the input leaves the prefix of the output unchanged. Checkpoint loading then compares the stored normalisation statistics, feature contract and history length of every learned component against what the environment expects, and aborts on a mismatch rather than degrading silently.

6.9 Constraints and Fair-Comparison Rules

6.9.1 Limits and Safeguards

Authority is bounded at four independent places, and the redundancy is deliberate because each bound protects against a different failure. The action chain of Section 6.2 limits the rate of change of the command, fades authority as cylinder travel headroom is consumed, and caps how far the integrated target may lead the measured extension. The actuator path clips the valve command to its per-axis limits. The environment terminates an episode on the safety conditions of Section 3.9 rather than letting it accumulate reward in a state an operator would stop. And every learned component falls back to a defined behaviour on a non-finite output.

Preflight runs before any training or evaluation and refuses to start on a violated precondition. The check that does the most work asserts that the commanded reference stays inside the measured actuator authority ceiling of Section 3.6.2: the cylinders move the cutter laterally at most 0.240 m/s, so a path family demanding more is infeasible by construction and no reward, critic or optimiser change could recover it. Terrain is derived from that ceiling, and the preflight assertion keeps the two from drifting apart.

6.9.2 Matching Rules for the Three-Stack Comparison

The comparison is only informative if the three stacks differ where the hypothesis says they differ and nowhere else. The following are held identical across R, IF and AIF: the verge task and its eleven path families, the terrain generator and the held-out mesh set, the reward function of Table 6.2, the action chain and its authority budget, the eight acceptance gates and their aggregation to the worst mesh and worst path family, episode length, actor and critic capacity, and the three-layer training construction of distillation, critic warm-up and reference-anchored PPO. This is the rule the comparison was designed to follow.

Four departures from that rule exist and are declared rather than smoothed over, because each of them weakens a comparison that would otherwise be clean.

The Reference Stack Contains the Plant

R's actuator is the command ROM, and the command ROM plus PhysX *is* the deployment plant (Section 5.4.1). Its tracking advantage over IF is therefore not evidence that a reduced-order model approximates the actuator better than an identified pair, because one of the two defines the target. R is a reference condition, not a competitor.

The Adaptive Stack Has a Larger Observation

AIF's actor reads 68 values against the others' 66. Any difference in its favour is therefore attributable to the latent's content, to the two extra inputs, or to the adapter, and these are separated by evaluation-time ablation rather than by assumption: the zeroed, shuffled, frozen and mean-latent arms of Section 5.7 all keep the architecture fixed and change only what occupies those channels.

Only Two of the Three Runs Are Matched

The rule above is satisfied by R and IF, which agree on every recorded training field and differ only in the actuator path, so that pair is a controlled single-variable comparison and carries the representation claim. It is not satisfied by the adaptive run, which differs in three recorded respects: seed (41 against 17), actor learning rate (3×10^{-6} against 3×10^{-4}) and frozen-actor warm-up (0 against 150 iterations). That stack is therefore reported as a third condition rather than a ranked competitor, and the deviation is recorded in Section 7.1 from the run manifests rather than inferred. The truck-speed schedule of the two reference runs is not recorded at all. Only one PPO training run exists per stack, so training variance is not estimated and the 33 evaluation cells measure scenario variation rather than optimiser variability. The across-stack result is therefore conditional on the selected checkpoints, and causal attribution to the architecture comes from the within-checkpoint latent substitutions instead.

Fine-Tuning Budgets Must Be Matched or Reported

Either all three stacks are distilled with identical teacher data and budget and then evaluated, or all three receive the same bounded fine-tuning budget, with both the distilled and the fine-tuned checkpoints reported. Comparing a distilled baseline against a separately optimised adaptive policy and attributing the difference to the AAM is not permitted, for the reason established in Section 6.3.2: reward ascent moves a policy off the acceptance criterion, so an unmatched budget is itself a treatment.

7. Experimental Evaluation and Discussion

7.1 Experimental Protocol

This chapter evaluates the three control stacks assembled in Chapter 6 and the learned actuator models they act through. It answers the last two research questions of Section 1.4:

RQ5 How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?

RQ6 Where plant variation survives that robustness, is it identifiable from recent observations, and does online adaptation by latent identification improve closed-loop performance?

The two differ in kind. RQ5 asks what a design choice changes and is answered by direct measurement on the matched pair R (R) and IF (IF), which agree on every recorded training field and differ only in the actuator path. RQ6 asks whether a mechanism pays, could fail in ways worth declaring in advance, and is therefore carried by predeclared hypotheses and by within-checkpoint substitutions rather than by the position of AIF (AIF) in a table. The short labels R, IF and AIF of Section 1.4 are used interchangeably with the full names. RQ6 also decomposes into a chain of dependent questions, each worth testing only if the one before it holds: is plant identity present in the data at all; can an estimator recover it; does the estimate follow the plant when the plant moves; does it improve a model that consumes it; does that improvement reach the controller; and if it does not, what stops it? The first four links hold and the fifth does not, which is why the chapter spends its last experiments on the sixth.

7.1.1 Hypotheses

Four hypotheses are declared in advance, the first two settled at model level and the last two in the control loop.

H1 The inverse model trained on randomised Simscape data is sufficiently robust to the gap between Simscape and the IsaacLab reduced-order hydraulic system that it can stay frozen. Support: a state predictor conditioned on an estimated latent, trained without the inverse model in the loop, closes the residual gap and beats a matched no-latent predictor on the same data.

H2 The value of the estimated latent lies in its variation over time, not in a static plant identity. Support: substituting a per-episode constant latent costs materially more than substituting a wrong constant.

H3 The closed-loop effect of plant-identity conditioning, estimated under a common evaluation protocol. H3 is an *estimand* rather than a hypothesis: its product is an interval, informative wherever it is narrower than the baseline mean it is measured against. No practical-equivalence margin was predeclared, so a narrow interval bounds the effect and does not establish equivalence.

H4 The effect estimated in H3 is boundable before an estimator is built, from what a privileged latent read from the true factors achieves.

H1, H2 and H4 carry rejection criteria fixed before the experiment runs: H1 fails without a majority of seeds, H2 if the constant-latent and wrong-latent substitutions cost the same, and H4 if a privileged latent does not outperform an uninformative one. H3 carries an adequacy criterion instead – a matched protocol and a calibrated interval – whether or not that interval excludes zero.

7.1.2 Test Conditions and Experimental Matrix

Every closed-loop experiment scores exactly three rows – R, IF and AIF – under one common condition: three held-out terrain meshes, the eleven trajectory families of the library, a pinned truck speed of 4 km/h, a 26 m scored window, 32 environments and a fixed evaluation seed. Model-level experiments use the open-loop excitation collection `rom_openloop_v1` of Section 7.5.1. Everything scored here is held out: the meshes are unseen, the families are scored individually rather than pooled, and the plant is drawn from sixteen reserved factor configurations whose keys are disjoint from training and validation by construction (Section 4.7), so a scored run is a plant the stack has never seen and not merely an episode it has not seen. How much of that factor draw reaches each stack's plant differs, and is audited in Section 7.4.1. Table 7.1 lists every experiment with its arms and its replication unit.

7.1.3 Metrics, Statistics and Fairness

The closed-loop primary metrics are mean absolute lateral error of the cutter against the commanded line and mean absolute cut-height error, with ground contact, hard impacts, head alignment and tool acceleration reported alongside them and the eight-gate acceptance contract of Section 3.9.1 used as the selection instrument. The shared estimand for every model-level experiment is the free-running accumulated normalised state error over a five-tick horizon, with no teacher forcing: the model integrates its own predicted increments and is charged for where it lands, which is the quantity a controller depends

Table 7.1: The experimental matrix. Closed-loop experiments share the common condition described in the text; model-level experiments use the open-loop excitation collection. The replication unit differs between the two and is stated for each.

Experiment	§	Arms	Unit
Closed loop			
Task performance	7.2	three stacks	33 (mesh, family) cells
Chain latency by stack	7.3.2	three stacks	4 cylinders × 16 envs
Commanded step	7.3.3	three stacks	16 envs
Control effort	7.3.5	three stacks	one mesh
Randomisation sweep	7.4	three stacks × five scales	11 families per point
Mid-episode plant change	7.6.5	online, frozen, shuffled, mean, sham	33 cells
Oracle latent	7.6.6	privileged, estimated, mean, shuffled	33 cells
Latent bandwidth	7.7.4	six time constants	33 cells
Model level			
Identifiability envelope	7.5.1	four excitation tiers	36 held-out runs
Latent content	7.5.3	online, constant, shuffled	five training seeds
Frozen-inverse sufficiency	7.6.1	latent, no latent	five training seeds
Predictive utility	7.6.2	privileged, estimated, mean, shuffled	36 held-out runs
Constructed plant switch	7.6.3	spliced run pairs	eight pairs
Actuation dead time	7.3.1	four excitation tiers	24 runs

on, whereas a one-step error would flatter every arm equally and hide the compounding behaviour that distinguishes them.

Primary metrics and seeds are fixed before each experiment is run. Every comparison is paired and reports the sign count over the paired unit alongside the mean, because a mean can be carried by a single outlying pair while a unanimous sign cannot. Uncertainty is quantified by a paired bootstrap rather than a parametric test, intervals are reported as ranges rather than converted into a verdict, model selection uses the validation split only, and reserved test runs are scored once.

The unit of replication differs between the two kinds of experiment. Model-level experiments replicate over training seeds. The closed-loop evaluation is deterministic – fixed mesh, deterministic policy, no variation in initial conditions – and five evaluation seeds returned bit-identical numbers to nine decimal places, so the seed is *not* a unit of replication and a sign count over seeds would report one observation five times. The closed-loop unit is therefore the (mesh, family) cell, and metrics that exist only per mesh are labelled $n = 3$ so that the weaker evidence stays visible.

Three reporting rules follow from failures met during this work: numbers from different

Table 7.2: Experimental controls used in this chapter, and what each one established. Entries in the third column are outcomes of applying the control, not hypothetical failure modes.

Control	Guards against	Outcome
Input ablation	a model scoring well without command authority	a forward model at one-step $R^2 \approx 0.999$ was insensitive to the valve
Open-loop excitation	command and state made redundant by feedback	the valve was not identifiable from controller-driven data at all
Sham arm	crediting a perturbation for what the schedule alone produces	the mid-episode plant change left the task unmoved
Onset positive control	an estimator manufacturing lag	the linkage contributes zero delay at every amplitude
Determinism check	reporting one observation several times	five evaluation seeds agree bit-exactly, so the seed is not a unit of replication
In-distribution null arm	an out-of-distribution input posing as an ablation	a zeroed latent lies two to three standard deviations outside the distribution, so the mean latent is used instead
Full-path rescoring	a selection window certifying part of an episode	the default window covers 8.9 of 26 m
Retrained baselines	comparing numbers across collections	differing normalisers make such numbers incomparable even for the same estimand
Per-family resolution	a worst-family aggregate hiding a threshold artefact	R and IF exceed the lateral threshold on one family each, and on different families
Matched-pair design	crediting the architecture for what the training run produced	R and IF are controlled; AIF is not
Within-checkpoint substitution	crediting a latent for a training run	carries every adaptation claim in this chapter

collections are never compared, even for the same estimand, because the state distribution and the normaliser differ, so a baseline is retrained instead; every gate is reported with the measured floor of the quantity it constrains and a positive control, since a threshold no run could violate produces a passing verdict carrying no information; and every closed-loop result names its checkpoint and its scoring horizon (Section 7.1.5).

Fairness is enforced by construction, and the guarantee differs by experiment. All model-level and within-checkpoint ablations are *paired*: the arms share the dataset, the split, the objective, the selection criterion and the seed, and differ in exactly one declared respect. The across-stack comparison is matched on evaluation conditions but not on PPO training realisation. Table 7.2 lists the controls that enforce this and what each one established, because several of them changed a conclusion during this work.

7.1.4 What the Three Stacks Can and Cannot Be Compared For

The three stacks do not stand in the same relation to one another, and the distinction governs every figure and table that follows.

The matched pair.

R and IF come from runs that agree on every recorded manifest field – seed 17, terrain seed 17, 256 environments, stage 6, actor learning rate 3×10^{-4}, entropy coefficient 0.001, frozen normaliser and a 150-iteration frozen-actor warm-up – and differ only in the actuator path. A difference between them *is* attributable to the actuator representation, and this is the pair that answers RQ5.

The third condition.

AIF is adaptive in the ordinary sense: its actor is $\pi(\boldsymbol{a} \mid \boldsymbol{o}, \hat{\mathbf{z}})$, conditioned at every model tick on an online estimate of the current plant, which also reaches the actuator through a bounded adapter. That it is adaptive *in operation* and not only in architecture is established rather than assumed: substituting what occupies those two extra observation channels moves closed-loop behaviour by 20.7 % on lateral error and 39.3 % on cut height (Section 7.6.6), which an actor indifferent to the latent could not produce. It is nonetheless a separately trained run at a different seed (41), an actor learning rate two orders of magnitude lower (3×10^{-6}) and no frozen-actor warm-up, so a difference between it and either of the others may be a difference of the adaptation path or of its training run, and this evidence cannot separate the two. It is therefore reported as a deployed controller running end to end under the common protocol and *not* as a ranked competitor; the remaining asymmetries are set out in Section 7.9.

Where the adaptation claims live.

Every causal statement about the adaptation path rests instead on a within-checkpoint ablation: one input is substituted while the policy, the actuator path, the plant, the mesh, the trajectory family and the evaluation seed are held fixed. The four-way latent substitution of Section 7.6.6, the bandwidth sweep of Section 7.7.4, the predictive comparison of Section 7.6.2 and the input ablations of Chapter 5 are all of this form.

7.1.5 Checkpoint Selection and the Scoring Horizon

One protocol decision turned out to be load-bearing and is reported rather than resolved silently. The project's checkpoint-selection utility defaults to a 480-step window, which at the 4 km/h evaluation speed covers 8.9 of the 26 m course, so a candidate can satisfy it on the opening third alone. Every candidate was therefore re-scored over the complete path against the same eight-gate contract.

The adaptive stack is therefore represented throughout this chapter by `headroom_priced_v1` at iteration 299, which passes seven of the eight gates on three meshes and fails only the lateral gate, at 0.326 against a threshold of 0.300. Its name records the mechanism: headroom was priced into that run's reward, and a quantity that is gated but not priced will be traded away. Two measurements fix what the selection is sensitive to. The adapter's

Table 7.3: Candidate adaptive checkpoints re-scored over the full 26 m path, worst-family values on one held-out mesh. The incumbent is among the weakest of the seven. Only `headroom_priced_v1`, whose reward prices cylinder headroom explicitly, clears the ground-contact gate.

Run and iteration	contact	cut height	lateral
`headroom_priced_v1`/299	**0.799**	**0.017**	0.326
`contact_focus_scale025`/625	0.503	0.054	0.092
`boom_focus_scale025_v2`/610	0.403	0.060	0.089
`contact_focus_scale025`/600	0.393	0.061	0.073
`stage31_promotable_v2`/375	0.360	0.068	0.482
`train_final`/599	0.346	0.073	0.107
`speed_range_2_7_contact_safe_v2`/700	0.344	0.076	0.540
gate	≥ 0.700	≤ 0.040	≤ 0.300

residual scale is not the operative variable, since zeroing it moves ground contact by 0.001; and the adaptive stack does not reduce to IF when that residual is disabled, because its policy is separately trained and conditioned on the latent. Each stack is therefore selected on its own terms. The 480-step default was used for R and IF as well, so the caveat was checked for all three rows; those two hold ground contact above 0.95 over the full path and are unaffected.

One earlier checkpoint, `stage31_idfd_aam_adapt_v1`/24, records all eight gates passing on six held-out meshes and appears to beat the one promoted here. It was scored under an easier protocol – 2 km/h rather than 4 km/h, and eight of the eleven families, omitting three of those that Section 7.2 shows carrying the pooled means – and a worst-family gate evaluated over a subset that excludes the hard families is a weaker test by construction. That comparison is therefore not available: no checkpoint of any stack has been shown to pass all eight gates under the protocol used here, and the 7/8 checkpoint scored correctly is reported in preference to the 8/8 checkpoint scored under conditions the rest of the chapter does not use.

7.2 Closed-Loop Task Performance

The chapter begins with the machine doing the task, because everything after it either explains these numbers or asks how far they hold. This is the only unperturbed comparison of the three stacks, it carries the answer to RQ5 on tracking, and it is where the estimand H3 is answered. All three stacks are scored on three held-out terrain meshes and eleven trajectory families under identical conditions: the same pinned truck speed of 4 km/h, the same 26 m scored window, the same 32 environments, distributed over eleven trajectory families, and the same evaluation seed. The unit of analysis is the (mesh, family) cell, which gives 33 paired conditions per stack; Section 7.1.3 explains why the evaluation seed is not a unit of replication here.

Figure 7.1 shows what the aggregate numbers summarise, on the three families the stacks track best. On these the commanded line is followed closely by all three, and the visible differences are oscillations about the reference rather than departures from it. Those

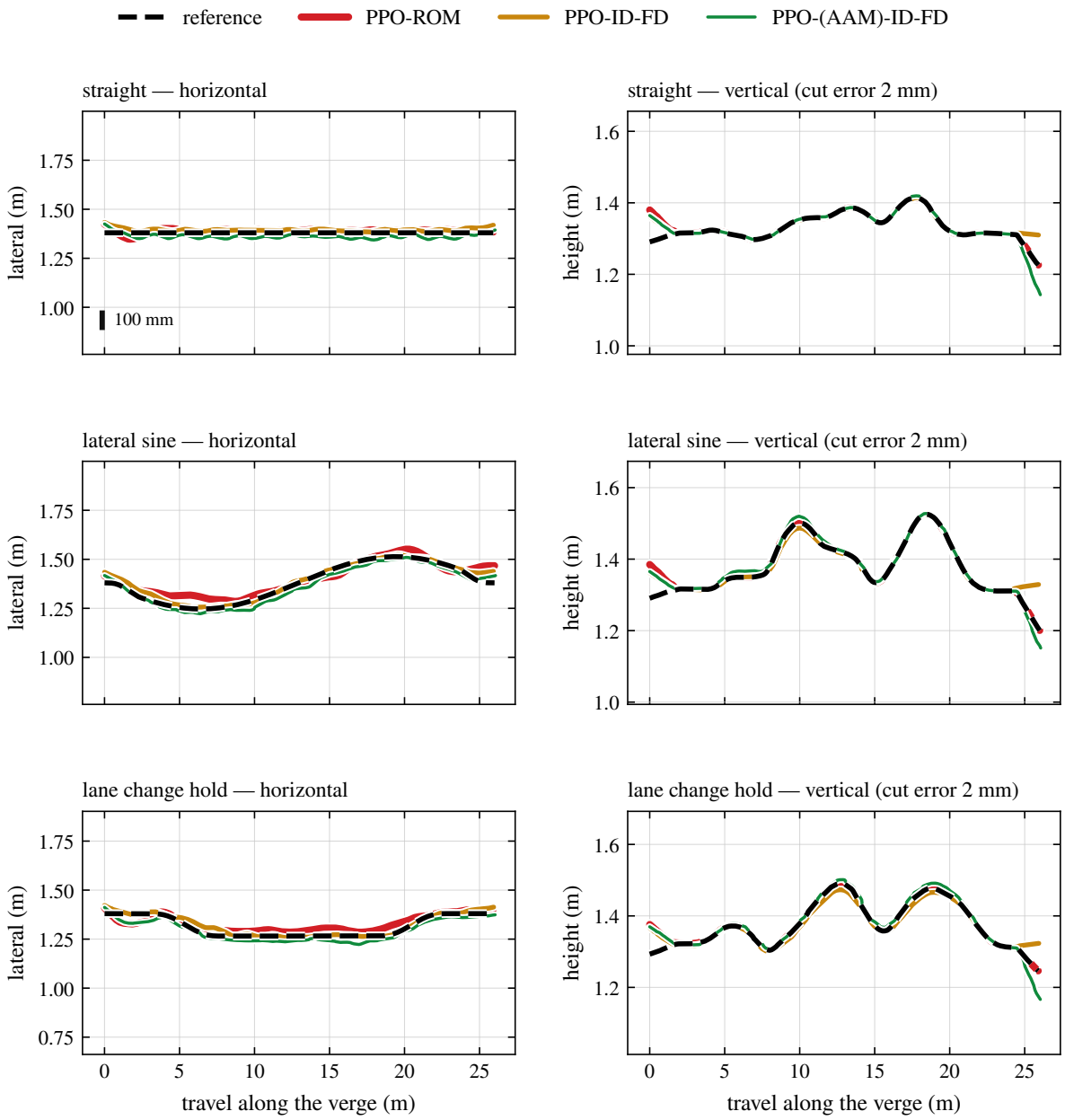


Figure 7.1: Cutter path against the reference on held-out mesh 1337, for the three well-tracked trajectory families that are also structurally different from one another: a flat line, a smooth periodic sweep and a step-and-hold, at 26, 27 and 29 mm mean absolute lateral error averaged over the three stacks. The Gaussian slalom scores marginally better than the lane change but is omitted because its reference is visually a sine and its panel would merely repeat the one above it; the lane change additionally exposes a transient that neither periodic family contains. The left column is the horizontal plane and the right column the vertical plane over the same drive. The vertical reference is not recorded directly and is recovered as $z_{\mathrm{ref}} = z_{\mathrm{cutter}} - e_{\mathrm{cut}}$, which makes both columns like-for-like reference-versus-actual comparisons. In the vertical panels the reference is largely hidden beneath the three traces, which is what a 2 mm mean cut-height error looks like when drawn at scale: the cutter follows the terrain undulation closely on every family, and the stacks separate only in the final metre. The environment-to-family assignment is deterministic and was verified by re-aggregating per-environment errors into per-family means and checking them against the run's own gate report.

Table 7.4: Closed-loop performance over 33 paired (mesh, family) cells. Lateral and cut-height error are the mean over families rather than the worst family, which is a maximum dominated by a single hard trajectory. Impacts, head alignment and tool acceleration exist only per mesh and are therefore $n = 3$. R and IF are the matched pair and differ only in the actuator; AIF is separately trained and its position here is not attributed to the adaptation path (Section 7.1).

	R	IF	AIF
lateral error, mean (m)	0.1026	0.0703	**0.0692**
lateral error, median (m)	0.0392	**0.0281**	0.0342
lateral error, p95 (m)	0.4664	0.2665	**0.2155**
cut-height error, mean (m)	0.0084	**0.0063**	0.0097
cut-height error, median (m)	**0.0013**	0.0031	0.0020
ground contact	**0.9623**	0.9529	0.9490
hard impacts	0.0947	**0.0905**	0.1204
head alignment	0.7438	0.2247	**0.9763**
tool acceleration p95 (m/s^2)	39.90	**39.02**	51.45

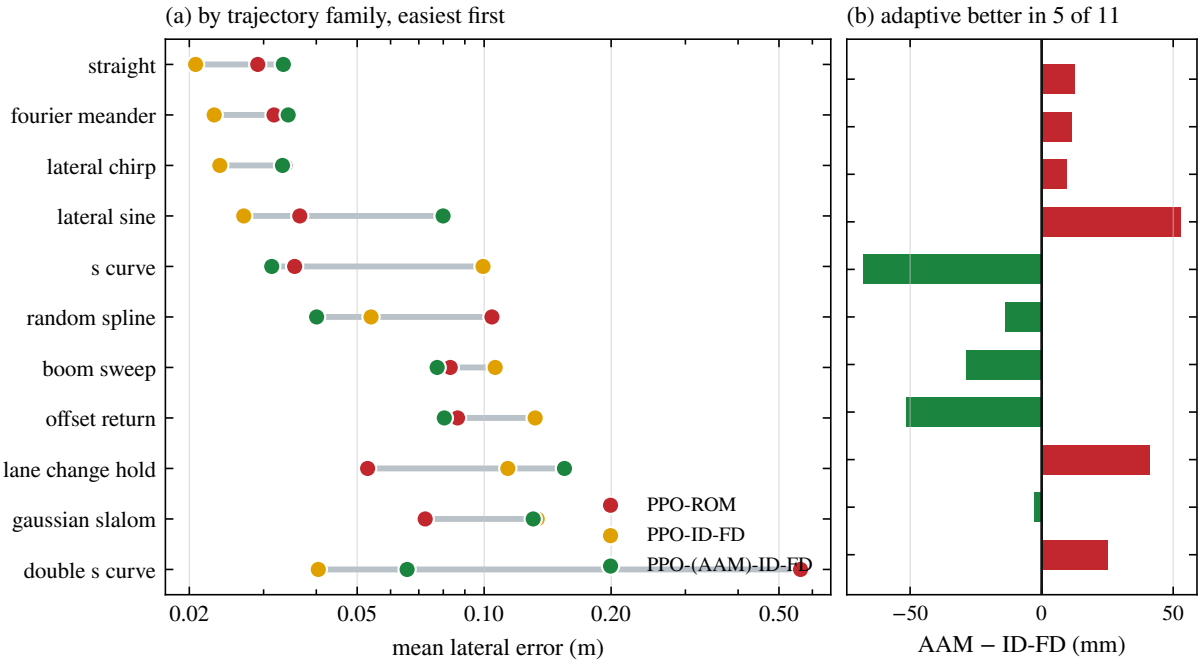


Figure 7.2: Lateral error by trajectory family, which is what a pooled distribution hides. (a) Each row is one family, ordered easiest first, with the bar spanning the three stacks; the suite covers more than an order of magnitude, and the two hardest families are what carry the pooled means in Table 7.4. (b) The paired contrast that isolates adaptation, family by family. No stack dominates: the adaptive stack is better on five of eleven families and worse on six, and the differences are tens of millimetres in both directions.

oscillations are a property of the closed loop and not of the plot: they have a wavelength of one to two metres, which at 4 km/h is a period of one to two seconds, so they are slower than any filter that could be applied without removing the signal itself. Their amplitude is what the 26 to 28 mm mean absolute error records, and reducing it is a control problem rather than a presentation one. That is the honest picture of the typical case, and it is worth stating plainly because the aggregate means are not dominated by it: the suite also contains families such as the double S-curve and the Fourier meander, on which errors are an order of magnitude larger, and those cells carry the means reported below.

The distinction matters for how Table 7.4 should be read. The stacks differ in the *tail* of the error distribution rather than in their typical behaviour, and the worst excursion over all cells is roughly 0.25 m for IF against about 1 m for the adaptive stack and 2 m for R.

The paired contrast closest to isolating adaptation is AIF against IF. The two share the frozen inverse and forward models and the evaluation protocol, but they are not a single-variable pair: the adaptive actor reads two extra observation channels and was trained from a different seed and learning-rate schedule (Section 6.9). What follows is a comparison of two deployed controllers, not of two training procedures.

On the three tracking metrics the two are separated by at most a few per cent. Lateral error differs by -1.6% with the adaptive stack ahead in 9 of 33 cells, cut-height error by $+53.8\%$ ahead in 25 of 33, and ground contact by -0.4% ahead in 25 of 33. All three paired bootstrap intervals span zero, so **the effect of plant-identity conditioning is smaller than this protocol resolves**. The measurement H3 returns is therefore its size, not its sign.

The interval is the product, and it is uneven across the three metrics. The paired intervals are $[-0.044, +0.040]$ m on lateral error against a IF mean of 0.070 m, $[-0.0034, +0.0108]$ m on cut height against 0.0063 m, and $[-0.024, +0.015]$ on ground contact against 0.953. Read as a bound, only the last is informative: ground contact differs by less than about 2.4 percentage points either way. On the two tracking errors the interval is comparable to or wider than the baseline mean itself, so the experiment does not constrain the effect to anything smaller than the effect that was being looked for. Establishing equivalence would require a predeclared margin and a sample large enough to fit inside it; neither is available here.

Mean and median disagree on cut-height error and ground contact: the adaptive stack is ahead in roughly three quarters of the cells yet behind on the mean, the signature of a few poor cells rather than a uniform deficit. Both statistics therefore appear in Table 7.4.

Three differences stand out from the null result above, and two of them do resolve. IF scores 0.2247 on head alignment, including -0.855 on one mesh, the collapse under unconstrained fine-tuning measured over the full path; the adaptive stack is the only one that holds alignment, at 0.9763; and it pays for its ground contact in impacts ($+33\%$, interval $[+0.013, +0.043]$) and tool acceleration ($+32\%$, interval $[+2.8, +21.3]$ m/s^2). Those last two intervals exclude zero, and they are the only closed-loop contrasts in this chapter that do, both of them against the adaptive stack. They are also the weakest evidence in the table: impacts and tool acceleration aggregate per mesh rather than per cell, so the bootstrap resamples three units and the interval should be read as a sign indication rather than a calibrated bound.

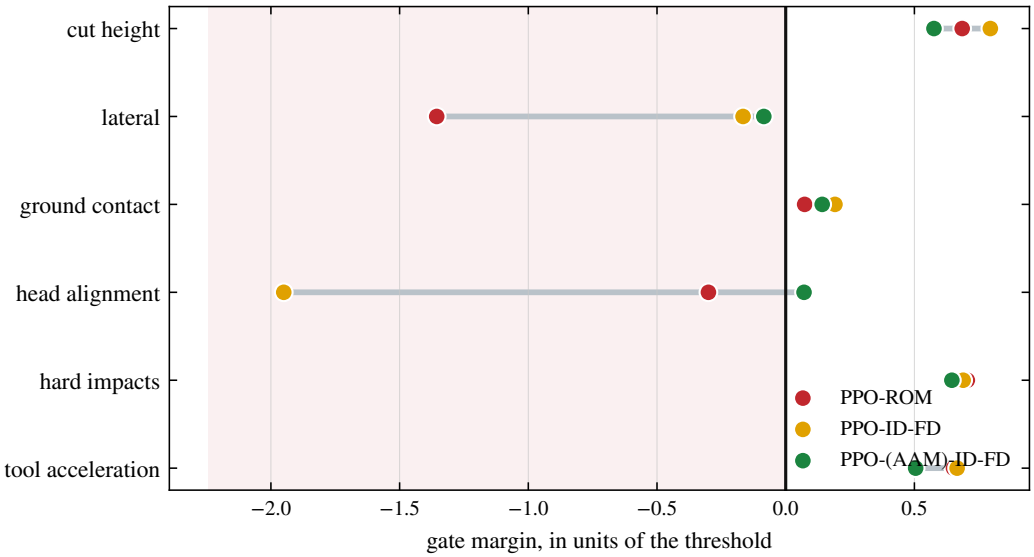


Figure 7.3: Gate margins on worst-mesh statistics, in units of each gate's own threshold, so the six are directly comparable; the shaded half-plane is failure. Two results stand out. The lateral gate is failed by *all three* stacks and most severely by R, so it reflects the difficulty of the worst trajectory family rather than a deficiency peculiar to the adaptive stack. And head alignment is failed badly by IF and passed only by AIF, which is the fine-tuning collapse of Section 7.2 seen against the acceptance contract. The matched pair is R and IF; AIF is separately trained and is shown as a third condition rather than a ranked entry.

Figure 7.3 places these numbers against the eight-gate contract and adds one result that the per-cell table cannot show. On worst-mesh, worst-family statistics the lateral gate is failed by every stack, and by R most severely. A threshold that no configuration satisfies is a statement about the hardest trajectory family in the suite, not a discriminator between architectures, and it should not be read as a defect of the adaptive stack alone.

The worst-family aggregation compresses eleven trajectories into one number, so it is worth resolving per family before it is interpreted. Table 7.5 does that, and the picture differs by stack.

For the two robust stacks the second reading is the supported one, and more strongly than a worst-family gate suggests. IF holds the gate in 32 of 33 cells at a median of 0.0281 m, an order of magnitude inside the threshold, and exceeds it on one family. R holds it in 30 of 33 at a median of 0.0392 m, also on one family. The two failing families are not even the same one, which is what a threshold measuring a specific trajectory rather than a controller deficiency looks like: the 0.300 m figure was anchored before the trajectory library was widened, and it now selects whichever single family the amplitude widening made hardest for a given stack.

That reading does not extend to AIF. It exceeds the threshold on six of eleven families at a median of 0.1412 m, five times the IF median, and the failures span structurally unrelated references. This is a property of that deployed controller rather than of the threshold, and it is consistent with the tail behaviour reported earlier in this section.

One scope note belongs with the table. Only the lateral and ground-contact gates aggregate to the worst family; the remaining six aggregate to the worst mesh, so the per-family reading above applies to those two and not to the others.

Table 7.5: Mean absolute lateral error per trajectory family, worst of the three held-out meshes, against the 0.300 m gate. Families are ordered by IF. An asterisk marks a family over the threshold. R and IF each exceed it on exactly one family, and on different families; AIF exceeds it on six. The first two are the matched pair and differ only in the actuator; the third is separately trained (Section 7.1).

Trajectory family	R	IF	AIF
`straight`	0.032	0.022	0.125
`fourier_meander`	0.032	0.024	0.118
`lateral_chirp`	0.038	0.026	0.155
`lateral_sine`	0.039	0.028	0.567*
`double_s_curve`	0.707*	0.050	0.475*
`random_spline`	0.230	0.109	0.361*
`offset_return`	0.153	0.188	0.366*
`s_curve`	0.037	0.241	0.119
`boom_sweep`	0.165	0.258	0.352*
`lane_change_hold`	0.067	0.279	0.482*
`gaussian_slalom`	0.139	0.350*	0.251
families within gate	10/11	10/11	5/11
cells within gate	30/33	32/33	24/33
median over cells	0.0392	**0.0281**	0.1412

As the selection instrument of Section 3.9.1 the gates do the work asked of them, with one property recorded for reuse: three of the eight do not separate these three stacks. Lateral is exceeded by each of them on a single family, hard impacts is passed by a do-nothing policy, and tool acceleration is passed by all three at less than half its threshold. The remaining five discriminate, head alignment most sharply at 0.9763 for the adaptive stack against 0.2247 for IF, and it is on those that the comparative statements of this chapter rest.

Three questions follow from this table, and the next three sections take them in turn. Why the two representations produce different controllers at all, and what each spends to track, is a question about the actuation chain and is answered in Section 7.3. Whether the performance survives terrain, plant and trajectory it has not seen is answered in Section 7.4. And why the adaptive stack does not convert its model-level advantage into task performance – an advantage established in Section 7.6 and absent from the table above – occupies Sections 7.5 to 7.7.

7.3 Actuator Representation and Dynamic Response

Section 7.2 established that the two actuator representations do not produce the same controller. This section measures what physically differs between them, and it is the section that answers RQ5 on latency and command behaviour: how long a command takes to reach the cylinder, how quickly each stack answers a change in what it is asked to do, and what each spends in actuation to track at all.

Latency enters the system in two quite different places, and separating them comes first. The hydraulics impose a transport delay that no controller can remove; the control stack

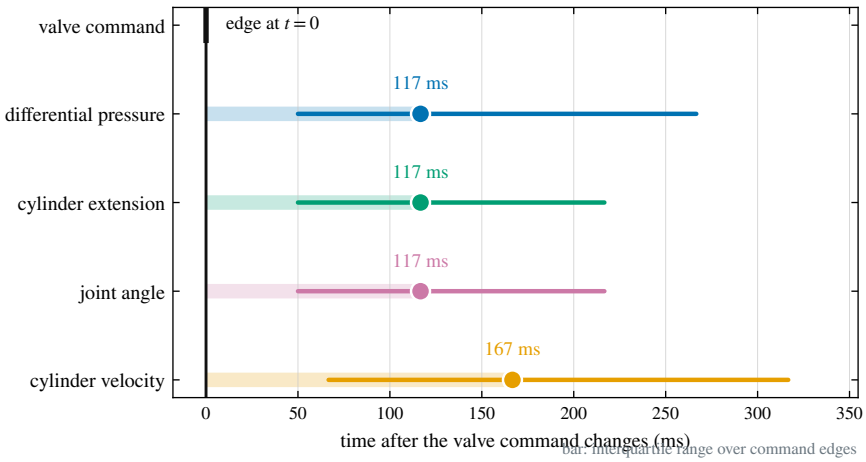


Figure 7.4: How a command propagates through the actuation chain, at excitation amplitude 0.15. The valve command changes at $t = 0$; each row below sits at the delay measured before that stage begins to move, with the shaded lead-in showing the wait and the bar giving the interquartile range over command edges. Pressure, cylinder extension and joint angle all begin at 117 ms and cylinder velocity at 167 ms. That extension and joint angle coincide is the control: the linkage between them is static and must contribute no lag. Each edge is aligned on the command's own onset rather than its steepest sample, since aligning on the slope places the origin after the cause has already acted and the response then appears to precede it.

then adds its own, and only the second is an architectural choice. Section 7.3.1 measures the first on open-loop excitation, where it can be isolated cleanly. Section 7.3.2 asks whether the three stacks differ in how quickly they drive the plant, and resolves the difference into its hydraulic and mechanical halves. Section 7.3.3 asks the question that matters for the task – how long each takes to answer a change in the commanded line – and finds the answer dominated by the control side rather than the hydraulic one. Section 7.3.5 then prices the tracking of Section 7.2 in commanded actuation and in command reversals, which is what wears the machine.

7.3.1 Dead Time in the Plant

Delay is measured here rather than assumed, and it is worth separating from an earlier negative result. Attempts to identify the valve's dynamic *model* from the available datasets did not succeed, but that is a parameter-fitting outcome and says nothing about whether the command-to-response *latency* is observable. It is, and the open-loop collection of Section 7.5.1 is close to an ideal instrument for it: the excitation is a train of zero-mean square waves with known edge times, which is what one would design for a step-response measurement.

The estimator locates each command edge and records the first sample at which the response leaves the pre-edge noise floor by four standard deviations. A cross-correlation peak was rejected because it conflates dead time with rise time. Only the pressure channel is drawn as a trace in Figure 7.4: cylinder and joint velocity are differentiated quantities whose edge-averages leave a transient at the command instant that reads as a response preceding its own cause.

Table 7.6: Transport latency along the actuation chain, median over 24 runs at 60 Hz control, in milliseconds. The last two rows are the control: the two onsets must coincide, and they do at every amplitude. Amplitude-invariance is what distinguishes a transport delay from a threshold effect.

	excitation amplitude			
Link	0.05	0.15	0.35	0.70
command $\rightarrow \Delta p$	117	117	117	150
$\Delta p \rightarrow \dot{\mathbf{s}}$	150	150	133	133
command $\rightarrow \dot{\mathbf{s}}$	183	167	133	117
command $\rightarrow \mathbf{s}$	117	117	100	100
command $\rightarrow \mathbf{q}$	117	117	100	100
linkage contribution	0	0	0	0

The dominant term is the command-to-pressure lag of 117 ms, stable across three of the four amplitude tiers. The kinematic control is exact: the command-to-extension and command-to-joint onsets agree to within 0.0 control steps at every amplitude, so the linkage contributes nothing and the estimator is not manufacturing lag.

The medians do not compose – the first two links sum to 16 control steps while the direct measurement gives 7 to 11 – because the per-edge distributions are wide and the median of a sum is not the sum of medians. Each link is supported individually; the chain decomposition is not.

This matters for the architecture rather than merely describing the plant. A dead time of 117 ms is 2.3 model ticks at the 20 Hz estimator rate and roughly a tenth of the AAM's one-second observation window. A controller reading only the instantaneous state cannot compensate for it, because the information required is not present in a single slice; this is the standard argument for history-based estimation [Ljung 99], and it holds here on measured rather than assumed grounds.

7.3.2 The Chain Compared Across the Three Stacks

The measurement above characterises one plant on open-loop excitation. The three control stacks reach that plant through different actuator paths, and whether those paths differ in latency is a question about the architectures rather than about the hydraulics. It is answered by recording the same chain during a closed-loop drive on each stack.

One methodological difference must be stated before the numbers, because it prevents a false comparison. A closed-loop command varies continuously and has no step edges, so onset detection cannot be used and the estimator is instead a cross-correlation lag, which conflates dead time with rise time. The values in Table 7.7 are therefore valid for comparing stacks, where the bias is identical across the three, and are *not* comparable with the open-loop dead times of Table 7.6. The two tables do not measure the same quantity.

Three readings follow. The adaptive stack reaches cylinder motion in less than half the time of the other two, 133 ms against 283 ms, and the difference is in the hydraulic half of

Table 7.7: Actuation-chain latency during a closed-loop drive, median cross-correlation lag over four cylinders and sixteen environments. The last row is the control: cylinder and joint are coupled by a static linkage, so it must read the resolution floor of one control step, and it does so on every stack. R and IF are the matched pair and differ only in the actuator; AIF is separately trained and its position here is not attributed to the adaptation path (Section 7.1).

Link	R	IF	AIF
command → pressure	450.0	166.7	158.3
pressure → cyl. velocity	0.0	166.7	58.3
command → cyl. velocity	283.3	283.3	**133.3**
cyl. → joint velocity	16.7	16.7	16.7

All entries in milliseconds.

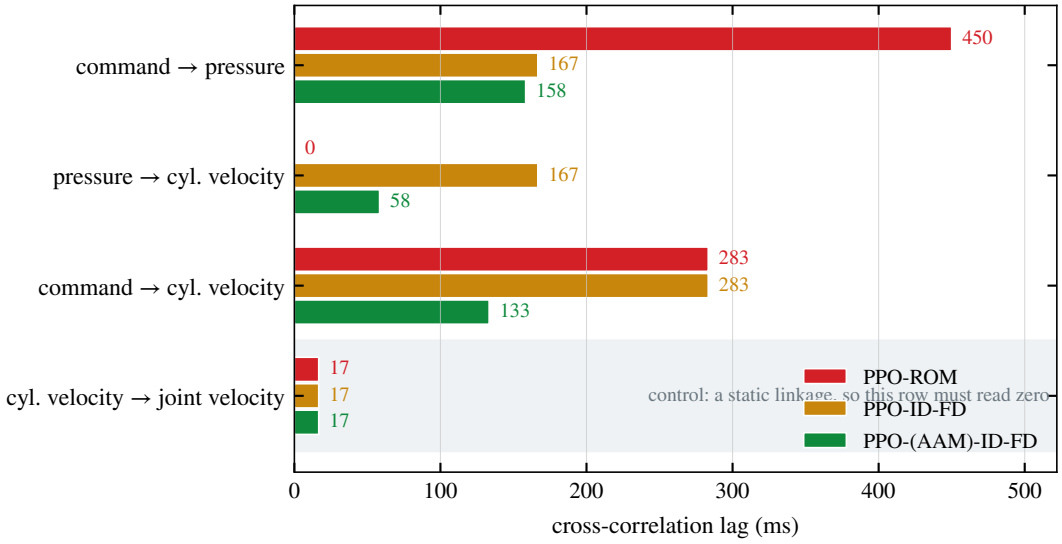


Figure 7.5: The same comparison drawn. The mechanical link at the bottom is the control and reads one control step on all three stacks, which is the resolution floor; the hydraulic links above it separate the architectures.

the chain rather than the mechanical one. R records no lag at all from pressure to cylinder velocity, which is a structural fact rather than a fast plant: the command reduced-order model predicts pressure and velocity together from one internal state, so there is no transport between them to measure, and its long apparent command-to-pressure lag of 450 ms should be read in the same light, since the valve signal on that path is produced inside the model rather than applied to it. The two stacks that drive the plant through the inverse and forward models, by contrast, show a genuine two-stage propagation with comparable lag in each stage.

The mechanical end is identical everywhere, at one control step, which is the resolution floor: the architectures differ in how they drive the hydraulics and not in how motion propagates through the linkage.

Table 7.8: Response to a 100 mm step in the commanded line, median over sixteen environments. The rise time is 10 to 90% of the step and settling is the first time the error stays within 20 mm for half a second. The underlying trajectory is subtracted before measurement, so the only content left is the response to the step itself.

Stack	dead time	rise	settled	overshoot
R	333	617	2633	57%
IF	350	717	**2183**	73%
AIF	558	625	3233	78%
plant dead time, open loop	117	—	—	—
command rate limit	—	417	—	—

All times in milliseconds.

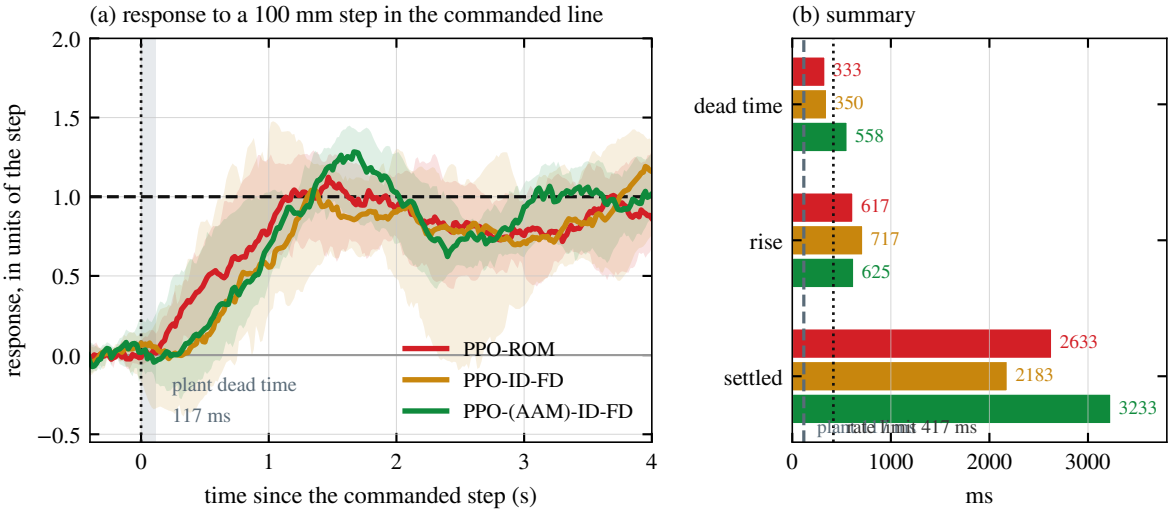


Figure 7.6: (a) Median response of each stack to the commanded step, in units of the step, with the interquartile band over environments; the shaded strip at the origin is the 117 ms the hydraulics alone require. (b) The same summarised, against two references: the plant's dead time and the 417 ms floor the command rate limit imposes on a 100 mm step.

7.3.3 Step Response to a Commanded Change

The dead time measured so far belongs to the hydraulics and is the same for every stack. What differs is how each control stack copes with it, and that cannot be recovered from an ordinary drive: reference and achieved path are coherent only below about 0.3 Hz, where phase reflects the controller's own lag dynamics rather than transport, so no delay extracted there is a dead time (Section 7.9). The delay is therefore injected rather than inferred. A step of 100 mm is added to the *commanded* line partway through the scored window, and each stack is scored on the same mesh, seed and speed. The step is applied at the single point where the tracking line is computed, so it reaches the policy's own preview of the path as well as the error term: a step visible to the metric but not to the controller would measure only the metric's discontinuity.

The result is that **the hydraulics are not what makes these stacks slow**. Every stack

begins to answer the step three to five times later than the 117 ms the plant needs, and none settles inside two seconds. Part of that is a hard bound rather than a control deficiency: at a command rate limit of 0.004 m per control step the fastest possible completion of a 100 mm step is 417 ms, which already exceeds the hydraulic dead time by a factor of three. The remainder is control-side, and settling times of 2.2 to 3.2 s are five to eight times that bound.

Between the stacks the ordering is modest and not uniform. IF settles soonest at 2.18 s, AIF is slowest to begin at 558 ms and slowest to settle at 3.23 s, and R overshoots least. The adaptive stack is therefore not faster to answer a change in the command, which is worth stating plainly alongside the finding of Section 7.3.2 that it reaches cylinder motion in less than half the time of the others: a shorter path from command to actuator does not translate into a quicker response at the tool.

The dead-time column compares when a response becomes *detectable* rather than pure transport, because it is measured against each stack's own pre-step band and the stacks oscillate differently during normal tracking; the rise and settling columns carry the more trustworthy comparison.

7.3.4 The Latency Budget of the Closed Loop

The three results are easy to misread in isolation, and together they say something the chapter needs later. The plant costs 117 ms from valve command to differential pressure, and that figure is amplitude-invariant with an exact kinematic control, so it is a property of the hydraulics rather than of any controller. The stacks differ in how quickly they traverse that chain, and the adaptive stack traverses it fastest, reaching cylinder motion in 133 ms against 283 ms for both others. Yet when the commanded line moves, the adaptive stack is the *slowest* of the three to respond and to settle.

A short path from command to actuator therefore does not produce a quick response at the tool, and the reason is that the dominant term is neither of the two. A 100 mm step cannot be completed faster than 417 ms at the command rate limit, which already exceeds the hydraulic dead time threefold, and the stacks settle in 2.2 to 3.2 s, five to eight times that bound. The actuation chain is not the bottleneck; the commanded rate and the policy's own dynamics are.

This bears directly on the adaptation question. An estimator that updates every 50 ms and a plant that answers in 117 ms are both fast relative to a loop that needs seconds to move the tool to a new line. Whatever the latent contributes, it contributes it to a controller whose response is set elsewhere, which is one candidate explanation for the gap between the model-level effect reported in Section 7.6.1 and the closely matched closed-loop arms of Section 7.2. It is taken up as an explanation in Section 7.7, once the model-level effect itself has been established.

7.3.5 Control Effort and Command Chatter

Tracking error ranks outcomes but not costs, and two stacks with comparable error are not equivalent if one commands twice the actuation to get there. Three measures are taken from the same drives, chosen so that none restates another: the root-mean-square valve command, its rate of change, and the number of times per second the command reverses

Table 7.9: Control effort and the tracking it buys, on held-out mesh 1337. Effort read without the error beside it would rank R best, which is the opposite of the truth. R and IF are the matched pair and differ only in the actuator; AIF is separately trained and its position here is not attributed to the adaptation path (Section 7.1).

Stack	RMS command	rate RMS	reversals/s	lateral error
R	0.586	7.55	40.0	120.1
IF	0.827	10.68	10.6	**26.8**
AIF	0.783	9.41	**9.5**	59.1

Lateral error in millimetres; command in normalised units.

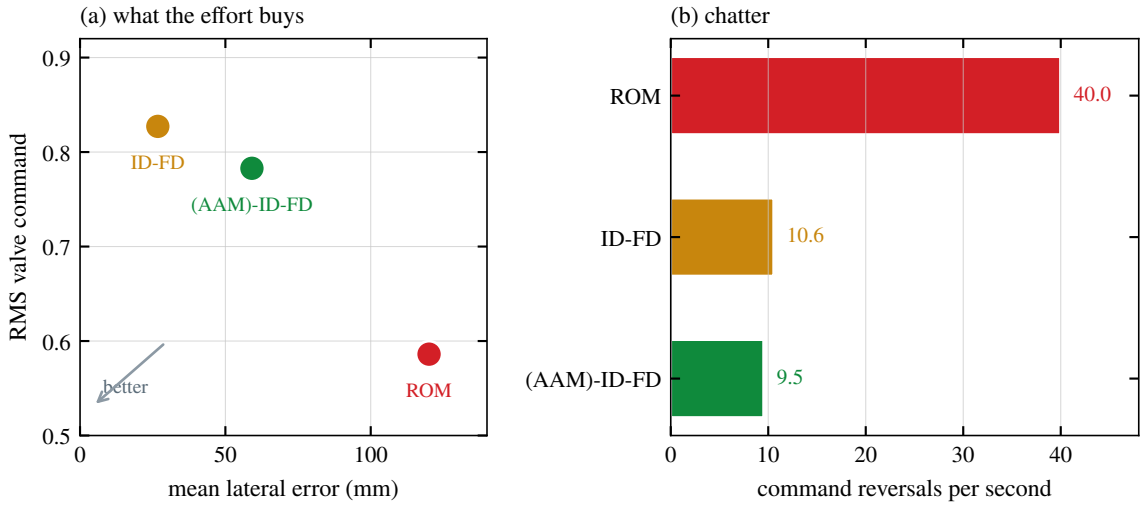


Figure 7.7: (a) Effort against the error it buys; the lower left is better. R spends the least and achieves the least. (b) Command reversals per second, which the root-mean-square does not reveal: R reverses four times as often as either other stack.

the direction of motion. The last is deliberately scale-free, because a stack can command little in total and still chatter, and chatter is what wears the machine.

The measure separates two failure modes that the error column alone conflates. R uses the *least* command of the three and tracks worst, which on its own would read as a conservative controller; its reversal rate of 40 per second, four times either other stack, says instead that its actuation is largely spent fighting itself. That also gives a mechanical account of its tool-acceleration and impact figures in Section 7.2, which otherwise appear as separate observations.

The two stacks driving the plant through the inverse and forward models behave in the opposite way: they command more, reverse a quarter as often, and convert the additional effort into tracking. Between them the adaptive stack commands slightly less and reverses slightly less often than IF while tracking about twice as poorly, so on this evidence it is smoother rather than cheaper.

One limitation bounds what this table can be used for. It compares three distinct policies, so a difference in effort cannot be attributed to any single component of the architecture. In particular it does not test the explanation offered in Section 7.6.6, that the estimated

latent disturbs the policy through its variation: that claim concerns different latents supplied to *one* policy, and the arms in Section 7.6.6 were scored without per-step traces. Measuring effort across those arms is the direct test and remains outstanding.

7.4 Generalisation and Robustness

Before the sweep itself, it is worth stating what the evaluation protocol of this chapter has already established about generalisation, because it is spread across several sections and is easy to miss when each is read on its own. Every closed-loop number reported here is measured on material the policies were not trained on. The terrain meshes are held out, the eleven trajectory families are scored individually rather than pooled, and the plant is drawn from the sixteen reserved factor configurations whose keys are disjoint from the training and validation splits by construction (Section 4.7), so a scored run is a plant the stack has never seen rather than merely an episode it has not seen – with the qualification, established in Section 7.4.1, that on the inverse/forward plant the unseen part of that draw is its mechanical half. On that material IF holds the lateral criterion in 32 of 33 (mesh, family) cells at a median of 0.0281 m (Table 7.5). The learned actuator underneath it generalises across simulators on the same evidence: the frozen inverse model was identified on Simscape alone and scored on 64 IsaacLab runs lying entirely outside its training distribution, where its accuracy is flat to a factor of 1.09 between the best and worst domain (Section 5.2.2). Generalisation across simulator, plant draw, terrain and trajectory is therefore demonstrated, and it is demonstrated without any online adaptation.

The question in this section is the remaining one: how far the plant may be moved before that generalisation stops holding, which a pass or fail on a fixed factor set cannot answer. The magnitude of the deployment-factor randomisation was therefore swept. Each environment keeps the draw it would have received and that draw is stretched away from the nominal plant along the same line, so the requested stress is 0 at the nominal machine, 1 at the manifest as drawn and 2 at twice every deviation. The intention was that only the magnitude should change and never which plants are visited, so that the curve would not be confounded with a change of population.

That is what the sweep requested. Section 7.4.1 reports what the plant received, and the two differ in a way that governs how the rest of this section may be read, so the audit is given before the result.

No stack's tracking moves with the requested stress. Mean lateral error over the sweep goes from 0.1286 to 0.1184 m for R, from 0.0277 to 0.0287 m for IF and from 0.0546 to 0.0673 m for the adaptive stack, and ground contact stays within 2.2% of nominal for R and 2.7% for the adaptive stack, while IF moves furthest at 5.5%. No point on the curve is a breakdown.

That is a flat curve, and a flat curve admits two readings: the controllers absorb the stress, or the stress did not arrive. The audit below establishes that the second reading cannot be excluded, so the first is not asserted here.

The one excursion is instructive precisely because it is not a trend. IF records 0.178 m at scale 1 against roughly 0.03 m everywhere else, including at the *more* extreme scales 1.5 and 2. Resolving it by family shows a single trajectory responsible: the double S-curve reaches 1.352 m, against 0.058 and 0.059 m at the neighbouring scales. One combination

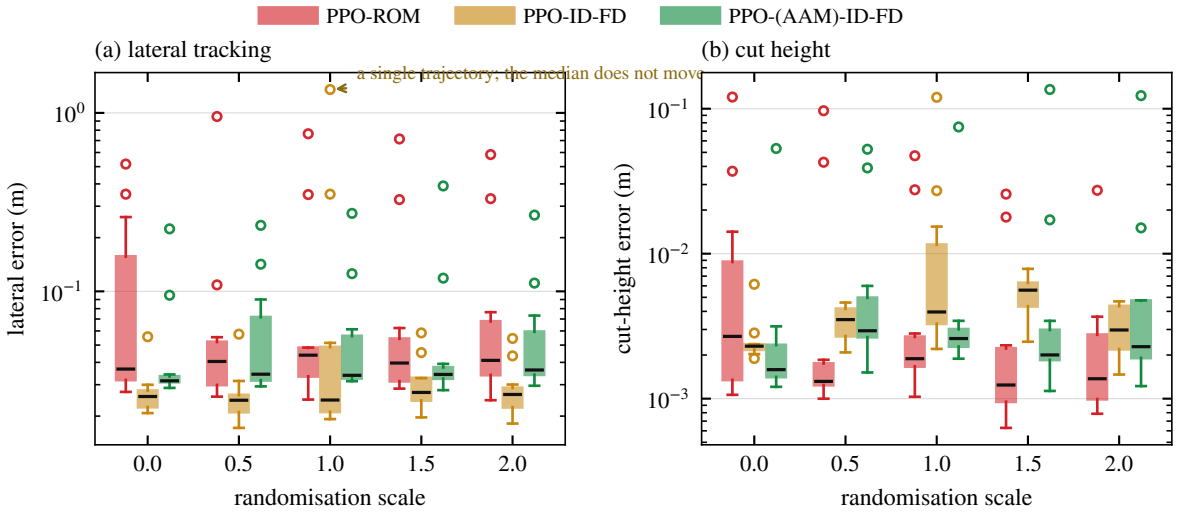


Figure 7.8: Per-trajectory performance at each randomisation scale, over the eleven trajectory families of one held-out mesh. Boxes give the interquartile range and the line the median; points are families outside the whiskers. The medians are flat across the sweep for all three stacks. How much of the requested stress reached the plant is audited in Section 7.4.1, and the flatness above scale 1 must be read against it. R and IF are the matched pair; AIF is separately trained, so its offset from them is not read as an effect of adaptation. IF's excursion at scale 1 is one trajectory and not a shift of the population, which a plot of means would have reported as a breakdown.

of plant and path destabilises that stack, and the combination does not survive being exaggerated. It is a genuine failure on a real manifest draw and it is not a breakdown point, and the distinction matters: a curve summarised only by its worst value would report a breakdown at scale 1 that the neighbouring points contradict.

The sweep uses one mesh and eleven families per point, so it resolves a trend rather than a small difference between stacks, and its absolute values are not comparable with the three-mesh tables earlier in this chapter.

7.4.1 How Much of the Requested Stress Reached the Plant

Two properties of the perturbation path decide what a sweep point delivers, and neither was accounted for when the sweep was designed. They are stated in the order of their severity.

Which factors reach which plant.

The six factors do not all act, and what acts differs by stack. Table 7.10 sets it out. The command reduced-order model takes the four hydraulic factors as an explicit input, so they act on R. The inverse/forward plant does not: its step consumes the valve command, the cylinder state, the differential pressure and the load, and takes no factor argument at all. On IF and AIF the hydraulic factors therefore reach the privileged latent encoder and the logs, and nothing else. What varies across a held-out factor draw on those two stacks is tool mass and centre-of-mass height, applied through PhysX without a clamp.

Table 7.10: Where each deployment factor acts. The inverse/forward plant takes no factor input, so on the two stacks that use it the hydraulic factors are inert and reach only the privileged encoder used by the oracle arm. This was established by inspection of the runtime and is checked empirically by the reachability control described in the text.

Factor	R (command ROM)	IF, AIF (ID/FD)
flow, compliance, seal, pressure	acts, clamped to training range	inert
tool mass, centre-of-mass height	acts, unclamped	acts, unclamped

How far the factors that do act may travel.

The stretch applied to a factor draw is not validated against the audited factor ranges, because a scaled row is constructed directly rather than parsed. The plant enforces those ranges instead: the reduced-order hydraulic model pins every hydraulic factor back into the interval it was identified on – compliance to $[-0.25, 0.10]$, flow to $[-0.25, 0.25]$, supply pressure to $[-0.15, 0.15]$ and seal condition to $[0, 1]$ – and then bounds the realised actuator response gain to $[0.35, 1.50]$.

Table 7.11 evaluates that clamping on the sixteen held-out factor rows the sweep visits. Two distinct quantities appear there and are worth separating, because conflating them overstates the finding. The *delivered fraction* is the Euclidean norm of the applied hydraulic deviation from nominal divided by the norm of the requested one: it is a ratio in factor space and says how much of the requested stress survived clamping, not what it does to the machine. At the requested scale 1.5, thirteen of the sixteen rows have at least one factor pinned; at scale 2, all sixteen do, and 77.8 % of the requested deviation arrives. The label "twice every deviation" is therefore accurate for the mechanical factors and wrong for the hydraulic ones, which sit at the edge of the training envelope while the mechanical factors alone continue to grow.

The *response gain* is the separate quantity, and it is the one that concerns authority. It multiplies both the velocity and the extension increment the reduced-order model emits, so it scales the realised cylinder motion directly. Two bounds apply to it. The explicit limiter clamps it to $[0.35, 1.50]$, which alone permits a reduction of at most 65 %. Within the clamped factor ranges the worst case the six factors can actually compose is 0.376, so **the factor path cannot scale the reduced-order model's realised actuator motion below** 0.376 **of nominal, a reduction of at most** 62 %. That is a statement about this model's realised motion under this perturbation path, and not about hydraulic authority in general; on the inverse/forward plant the same factors scale nothing at all.

Three consequences follow, and they replace the conclusion this experiment was originally read as supporting.

The first is that for two of the three stacks the sweep did not vary the hydraulics at all. A flat curve for IF and AIF across a hydraulic randomisation is the expected reading of an inert input, not evidence that those stacks tolerate hydraulic variation. Their curves record tolerance to a doubling of tool mass and centre-of-mass offset, which is a narrower claim and is the one made here.

The second is that the sweep axis is not an axis of increasing difficulty. Above scale 1 it changes the *kind* of plant rather than the amount of deviation, since the hydraulic factors

Table 7.11: Requested against delivered stress over the sixteen held-out factor rows. "Delivered" is the applied hydraulic deviation from nominal as a fraction of the requested one – a ratio in factor space, not a measure of authority – so 1.000 means the sweep point reached the plant intact. The last column is the separate quantity that does scale realised motion, the reduced-order model's response gain. Above scale 1 the hydraulic half of the sweep is progressively pinned to the training envelope, while tool mass and centre-of-mass height are never clamped.

Requested scale	rows with a pinned factor	delivered	min. response gain
0.5	0/16	1.000	0.671
1.0	0/16	1.000	0.472
1.5	13/16	0.911	0.402
2.0	16/16	0.778	0.376

stop moving while the mechanical ones continue. The single excursion described above is consistent with exactly that: IF's failure occurs at scale 1, the last point at which the full manifest draw is delivered intact, and disappears at 1.5 and 2, where the hydraulic part of that same draw is pinned back inside the envelope. A non-monotonic response is what a curve of this construction should be expected to produce, and it is not evidence that the plant became milder.

The third is a bound on the instrument itself, on the one stack where the hydraulic factors do act. A knockout to zero authority is unreachable by this route: the realised motion cannot be scaled below 0.376, whatever is requested. A robustness boundary is a statement about the point at which control authority becomes insufficient, and this experiment cannot reach that point by construction. The flat curve in Figure 7.8 is therefore a property of a bounded perturbation path as much as of the controllers, and the claim that these stacks are insensitive to plant variation is not supported by it.

7.4.2 What a Robustness Boundary Would Require

The correct experiment is not this sweep extended to scale 3, which Table 7.11 shows would deliver less of what it requests, not more. It is a search for the physical point at which command authority becomes insufficient, and it needs three things this sweep lacks.

It needs, first, a stress axis that *reaches every stack and means the same thing on each.* The deployment factors are not one, for the two reasons above. The axis specified here instead scales the realised cylinder motion the actuator produced, immediately before that motion updates the physics target:

$$\Delta x_k^{\text{applied}} = \alpha \, \Delta x_k^{\text{actuator}}, \qquad \alpha \in (0, 1], \tag{7.1}$$

where $\Delta x^{\text{actuator}}$ comes from the command reduced-order model on R and from the inverse/forward pair on the other two. It bypasses the clamp, it is applied at the same point in all three stacks, and it is named for what it is: *actuator-authority degradation* at the deployment interface, not seal, not flow, and not any hydraulic parameter. Its meaning is unambiguous – the fraction of commanded actuator motion that is actually realised – and unlike a factor distance it has an obvious endpoint, since at $\alpha \to 0$ the moving reference must become untrackable.

That endpoint is what makes the *positive control* exact. Running $\alpha = 1$ against $\alpha = 0.01$ and recording both $\Delta x^{\text{actuator}}$ and $\Delta x^{\text{applied}}$ allows the ratio to be checked numerically rather than inferred from a trajectory plot: it must equal α. Only then is the second question asked, whether the cutter loses the line. A stack that still tracks at one per cent of its actuator authority is a finding about the harness, and the two failures are distinguishable because the ratio separates them.

It needs *one factor at a time* before any combination. A six-factor stretch permits one factor to compensate another, so the direction that makes the task harder has to be measured rather than assumed.

It needs its *thresholds declared in advance*. Two are used: degradation onset, the first stress at which median lateral error reaches one and a half times its own nominal value or ground contact falls by five percentage points; and functional breakdown, ground contact below 0.700 or a safety termination rate above 0.080.

And it needs a *selection rule that cannot favour the adaptive stack*. The degradation curve is measured on IF alone, the non-adaptive baseline; the stress levels are frozen from that curve; and only then are all three stacks scored at those levels. AIF is never inspected while the levels are chosen, so the condition cannot have been selected to flatter it.

That experiment is specified and instrumented but not run: it requires the IsaacLab plant on GPU hardware, and its results are not part of this thesis. What this section establishes is narrower and is stated in place of the robustness claim it replaces. On R, within a perturbation path that cannot remove more than 62 % of actuator authority and over a sweep whose upper half is progressively clamped, no degradation of task performance was observed. On IF and AIF the sweep varied tool mass and centre-of-mass height only, and neither degrades under a doubling of both. The boundary is not located on any stack, and *the absence of an observed boundary is not evidence that one is distant*.

7.5 Identifiability of Plant Variation

Sections 7.2 to 7.4 answer RQ5 and leave RQ6 standing. Both representations support closed-loop control on held-out terrain, plants and trajectories, and neither degrades when the plant is moved as far as this work can move it – all without any online adaptation. That is the setting in which adaptation has to earn its place, and the first question is not whether an estimator is accurate but whether there is anything in the data for it to recover. This section measures that, and then asks what kind of quantity the recovered latent actually is.

7.5.1 Excitation Design and the Identifiability Envelope

What the data makes identifiable determines what any estimator can recover, so it is measured before any estimator is judged, and the headline is that excitation gathered under a task controller does not separate plants at all.

The reason is structural. A task controller computes its command from the measured state, so command and response are close to redundant and little is left for a plant descriptor to explain; the input must be rich enough for the regressor to stay well conditioned [Ljung 99], which makes excitation design a first-class decision [Schoukens 19]. The collection used

Table 7.12: What each excitation tier makes identifiable, and why the higher tiers do not. Decode R^2 is held out over 36 reserved runs; drift is the across-plant spread of cylinder extension at matched ticks, expressed as a fraction of the stroke the probe itself produces. Plant identity survives the first two tiers and is gone by the third, while command saturation never exceeds ten per cent.

Amplitude	seal R^2	mean R^2	drift / stroke	command sat.
0.05	0.703	0.109	0.18	0.11%
0.15	0.505	0.099	0.28	0.35%
0.35	−0.018	−0.040	0.39	3.26%
0.70	0.043	−0.021	0.50	9.98%

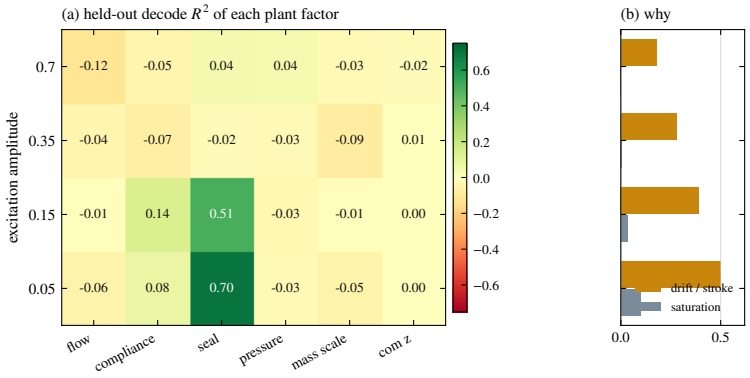


Figure 7.9: The identifiability envelope. (a) Held-out decode R^2 for each of the six plant factors at each excitation level. Only the seal condition is recoverable, and only in the two lowest tiers. (b) The reason: operating-point drift, as a fraction of the stroke the probe produces, rises steadily with amplitude while command saturation stays low throughout and cannot account for the collapse.

throughout this chapter therefore removes the controller entirely and applies open-loop zero-mean square-wave excitation directly, at the cylinder-command interface because the command reduced-order model accepts cylinder extensions and not valve signals by design (Section 5.4). This collection, `rom_openloop_v1`, comprises 118 plant draws of 11 040 control steps each, split 82/20/16 by run into training, validation and reserved test sets, and its Hankel matrix reaches full rank 40 of 40 at a condition number of 17.8.

The excitation sweeps four amplitude tiers. Table 7.12 reports what each tier makes identifiable, decoding the six true plant factors from the per-tier mean latent estimate and scoring on held-out runs. The seal-condition factor is the positive control throughout, since it is the one factor the estimator is known to represent (Section 5.6.1); if it collapses, the decode is broken and no other entry in the row carries information.

The obvious explanation for the collapse is actuator saturation, and it is wrong. Saturation reaches only 9.98% of steps even at the largest amplitude, which cannot account for a decode falling from $R^2 = 0.703$ to zero. The measurement that does account for it is operating-point drift. Every run executes an identical excitation schedule, so at a matched tick the only thing that differs between runs is the plant, and the across-run spread of cylinder extension is a direct reading of how far the plants have desynchronised in position.

That spread grows from 1.5 mm at the start of the episode to 54.2 mm at the end, reaching half of the stroke the probe itself produces. Once runs are that far apart, a one-second observation window cannot separate a response that differs because the seal is leaking from one that differs because the cylinder is sitting five centimetres lower. The confound is an accumulated history effect, and it is not one the estimator can be blamed for.

Two consequences follow, and both constrain later sections. First, all adaptation experiments in this chapter are run below amplitude 0.35, inside the band where plant identity is present at all; an experiment run above it would be testing the estimator on data that contains no answer. Second, the excitation plan ramps amplitude monotonically across the episode, one tier per quarter. That schedule is deliberate: it sweeps the full amplitude range inside every run, so each plant draw contributes to all four tiers and the tiers are compared on identical plants rather than across separate collections. The consequence for interpretation follows from the design and is stated so that the collection is read correctly: amplitude and episode time are the same axis here, so this collection supports statements about amplitude and not about elapsed time within an episode. A pilot analysis that read decode quality as decaying over the episode was reading the amplitude sweep, and is recorded as such.

7.5.2 Closed-Loop Data Against Excitation Data

The envelope above is a statement about data rather than about estimators, and the sharpest demonstration of that is an architecture that changes sign when the data changes. The latent-conditioned state predictor of Section 7.6.1 lowers free-running error by a median of 22.3 % on the open-loop collection. On the earlier collection gathered under a task controller, the same architecture, the same objective and the same selection criterion put the latent *behind* its no-latent baseline: the no-latent predictor scored 0.17278, the end-to-end latent 0.17596 and the privileged latent 0.18515, so plant conditioning appeared worthless or worse.

Nothing about the model changed between those two results. What changed was whether the data contained an answer, exactly as the envelope above predicts: under a task controller the command is computed from the state, so command and response are close to redundant and the plant has little left to explain. Two consequences follow for this chapter. Reporting only the closed-loop collection would have supported a confident negative conclusion about the architecture that the open-loop data contradicts, which is why every model-level experiment here uses `rom_openloop_v1`. And an identifiability check of this kind is cheap, requires no estimator, and belongs before rather than after the decision to build one.

7.5.3 Information Content of the Adaptive Latent

That the latent helps does not establish *why*, and the two available explanations carry different consequences for the thesis. Under the first, the estimator recognises which plant it is operating on and the policy is conditioned on a plant identity. Under the second, the estimator tracks a time-varying operating condition and its value lies in the estimate moving. H2 separates them with a substitution experiment that changes nothing but the contents of the latent slot.

Table 7.13: Where the latent's value lies, five seeds, one trained model each, scored on the reserved validation runs. Freezing the estimate at its correct per-episode value costs 13.6%; making that frozen value wrong costs a further 0.43%. Both effects have the same sign in every seed.

Latent supplied to the state model	mean error	vs. online	seeds
online estimate $\hat{\mathbf{z}}(t)$	0.018842	—	—
per-episode constant, correct identity	0.021402	+13.6%	5/5
per-episode constant, wrong identity	0.021492	+14.1%	5/5

One trained model is scored three ways on the same data. In the *online* condition the latent is read from the observable history at every model tick, as in deployment. In the *constant* condition it is replaced by that run's own mean estimate, which preserves the correct plant identity exactly and destroys all variation. In the *shuffled* condition it is replaced by another run's mean, preserving the distribution of latent values while making the identity wrong. The gap between online and constant measures what the variation is worth; the gap between constant and shuffled measures what the identity is worth.

The decomposition is unambiguous and tight across seeds (Figure 7.10b). Removing the variation costs 13.6%, with a per-seed range of 11.9 to 14.3%. Removing the identity on top of that costs a further 0.43%, with a per-seed range of 0.32 to 0.84%. The second effect is consistently signed in all five seeds and so is real rather than noise, but it is roughly thirty times smaller than the first. Essentially all of the latent's value lies in the estimate moving, and almost none of it in the estimate being correct about which plant this is. H2 is supported.

This licenses one claim and forbids another. The claim it forbids is that the module identifies the plant it is operating on: at 0.43%, that reading is very nearly false, and the thesis does not make it. The claim it supports is that the module tracks a time-varying operating condition, and that the tracking is what earns the performance.

This is the ordinary form of the claim rather than a weak one. The module estimates from a fixed one-second causal window and carries no state between ticks, so it re-estimates rather than accumulates: recursive estimation with a forgetting factor, the standard construction when the estimated parameters are themselves time-varying [Åström 95, Ljung 99]. A converging estimator would lock onto a past operating condition and stop responding [Kumar 21].

Two boundaries are worth stating before the closed-loop chapters. The first is that this analysis concerns the jointly fine-tuned estimator trained on open-loop excitation, which is a different model on different data from the deployed estimator characterised in Section 5.6.1; the two disagree about stability, and the disagreement is informative rather than contradictory. The deployed estimator, trained under a task controller, is static within a run, whereas this one varies within an episode by roughly nine times the spread between plants. Excitation decides which of the two behaviours the estimator exhibits. The second boundary is that everything here is measured on a dynamics model rather than in the control loop; H4 repeats the substitution experiment on the closed-loop stack in Section 7.6.6, and only that result can carry the adaptation claim into the task.

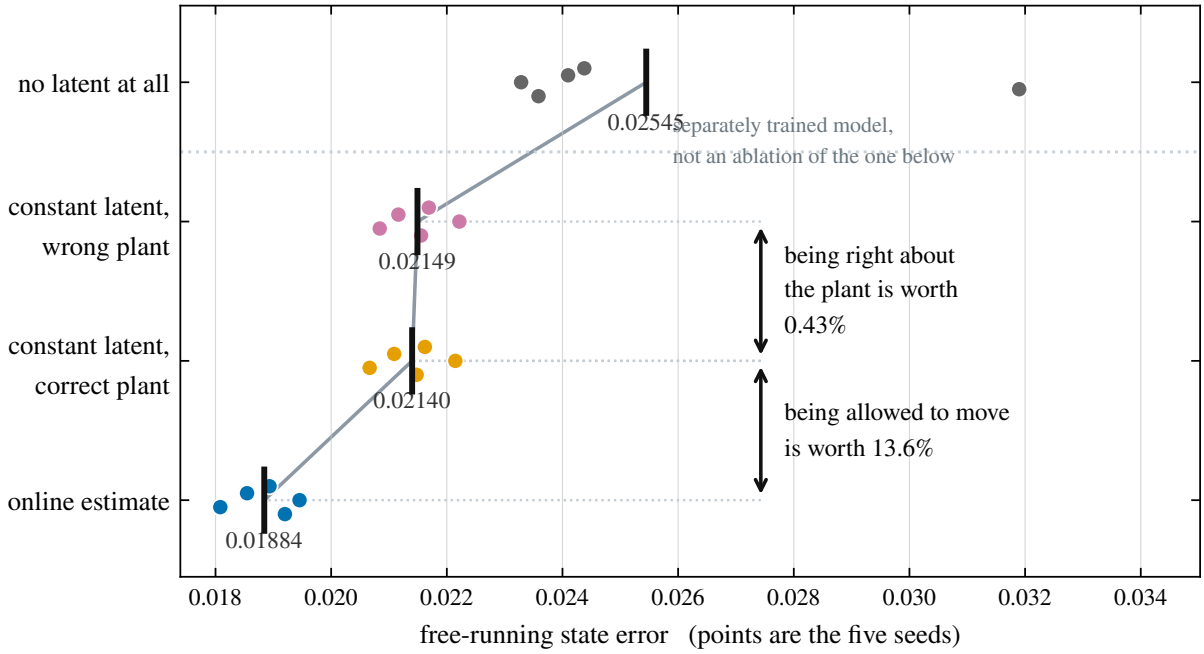


Figure 7.10: The model-level result as a single sequence, worst at the top. Each point is one of the five seeds and the heavy bar is the mean. The *gaps* are the claim: making the constant latent correct about the plant rather than wrong is worth 0.43%, while allowing that same estimate to vary in time is worth 13.6% – a factor of thirty-two. The topmost row is a separately trained no-latent model rather than an ablation of the three below it, so the step down to it is not attributable to the latent alone and is divided off.

7.6 Online Adaptation

Plant identity is present in the data when the excitation is right, and what the latent carries is a moving operating condition rather than a fixed label. This section asks what that buys. It proceeds outward from the model to the machine: first whether the latent improves a model that consumes it and whether the estimate is as good as the truth, then whether the estimate follows the plant when the plant moves, then whether either matters to the closed loop under a plant change and under an oracle that removes estimator error altogether.

7.6.1 The Model-Level Gain and the Frozen Inverse Model

The inverse model is trained on randomised Simscape data and deployed against the IsaacLab reduced-order plant, and the architecture of Chapter 6 depends on it staying frozen. H1 asks whether that is defensible. The direct measurement of inverse-model transfer error is reported in Section 5.2.2; what is tested here is the weaker but more consequential proposition that whatever residual gap remains is learnable *without touching the inverse model at all*.

The test removes the inverse model from the loop entirely. A state predictor consumes the current state, the applied command and an estimated latent inferred end to end from $[\mathbf{q}, \dot{\mathbf{q}}, \Delta p, \mathbf{u}]$, and predicts the next state. The baseline is the same predictor with no latent input, retrained on the identical data, split and objective, at five matched seeds. Figure 7.11 reports the pairs.

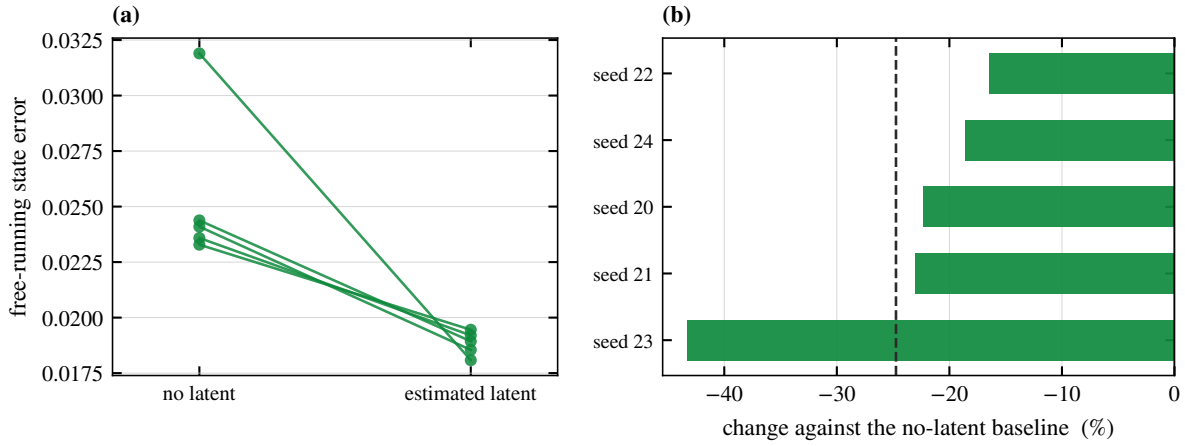


Figure 7.11: Latent conditioning against a matched no-latent baseline, five seeds. (a) Each line is one seed. No pair crosses, which is the result; the mean is not. (b) The same data as paired changes, with the mean dashed. Seed 23 carries the mean because its baseline is an outlier, so the median −22.3 % is the figure to quote.

Table 7.14: Sufficiency of the frozen inverse model, five matched seeds. The estimated latent lowers free-running state error in every seed. Seed 20260823 has an anomalously poor baseline and inflates the mean, so the median of −22.3% is the representative figure.

Seed	no latent	estimated latent	change
20260820	0.02438	0.01893	−22.3%
20260821	0.02410	0.01855	−23.0%
20260822	0.02328	0.01945	−16.5%
20260823	0.03190	0.01808	−43.3%
20260824	0.02359	0.01920	−18.6%
mean	0.02545	0.01884	−24.7%

The estimated latent improves on the no-latent baseline in 5 of 5 seeds, with a mean paired change of −24.7% and a paired bootstrap 95% confidence interval of [−34.2%, −18.2%] that excludes zero (Figure 7.10a). One qualification belongs with the number rather than in a footnote: seed 20260823 produced an unusually poor baseline at 0.03190 against roughly 0.024 elsewhere, and its −43.3% is as much a statement about that baseline as about the latent. The median change of −22.3% is the figure to quote, and the sign, reproduced in every seed of a paired design, is what supports H1.

That the same comparison reverses on controller-driven data is what makes the result interpretable, and it is reported with the identifiability evidence in Section 7.5.2.

H1 is therefore supported, with its scope stated plainly. The experiment does not measure inverse-model error, and it is not evidence that the inverse model is accurate in any absolute sense. It shows that the residual mismatch between the Simscape training distribution and the deployment plant is recoverable by a latent-conditioned state model, which is what licenses the simpler architecture in which the inverse model is never fine-tuned online.

Table 7.15: Free-running rollout error under four latents, pooled over 36 held-out runs. Lower is better. The estimate tracks the privileged latent at every horizon, and both are clearly ahead of an uninformative or a wrong one.

Horizon	privileged $\boldsymbol{z}$	estimated $\hat{\mathbf{z}}$	mean	shuffled
1 tick	2.961×10^5	2.966×10^5	3.388×10^5	3.499×10^5
10 (0.5 s)	1.820×10^6	1.826×10^6	1.952×10^6	1.998×10^6
20 (1.0 s)	2.961×10^6	2.977×10^6	3.196×10^6	3.256×10^6

7.6.2 Estimated Against Privileged Latents

Section 7.5.3 established that the latent's value is temporal and Section 7.6.1 that it lowers free-running error. Neither asked the question a controller cares about: is the *estimated* latent as useful as the true one, and is it better than no plant information at all? That comparison is made here, on the frozen system-dynamics model, by supplying four different latents to one network and changing nothing else.

The four arms are the privileged latent read from the true factors, the causal AAM estimate, an in-distribution mean carrying no plant information, and a shuffled latent with the right distribution and the wrong plant. All four are scored on the same 36 held-out runs, on free-running rollouts at one, ten and twenty ticks, with the model integrating its own predictions.

Table 7.15 gives the ordering, and the paired statistics give its standing. Against the privileged latent the estimate is indistinguishable: the paired difference at twenty ticks is $+1.60 \times 10^4$ with a 95 % interval of $[-1.35 \times 10^4, +4.62 \times 10^4]$, and the estimate is ahead in 18 of 36 runs, which is exactly chance. Against the two uninformative controls it resolves: -2.19×10^5 with the interval excluding zero and 33 of 36 runs against the mean latent, and -2.79×10^5 with 29 of 36 against the shuffled one. On the reserved test split the estimate is marginally ahead of the privileged latent, 2.837×10^6 against 2.850×10^6.

Two things follow. The AAM recovers essentially all of the information the privileged encoder carries, so the estimator is not the limiting component of the adaptation chain. And the advantage resolves on extension and velocity, not only on pressure: against the mean latent the extension difference is -1.87×10^{-4} with 32 of 36 runs and the velocity difference -5.14×10^{-4} with 31 of 36, both intervals excluding zero. This is a different comparison from the one in Section 5.5, which contrasted a latent-conditioned model with a separately trained no-latent model and found the advantage confined to pressure. Substituting latents *within* one model isolates the latent itself, and by that measure the channels the controller integrates do respond.

7.6.3 Response to a Change of Plant

An estimator that identifies a fixed plant is not yet an adaptive one. What remains is whether the estimate follows the plant when the plant moves. No recorded run answers this, because each run holds one factor vector for its whole length. The trajectory is therefore *constructed*: two held-out runs are spliced at a matched tick, which is admissible here because every run executes the identical eleven-segment schedule, so at a matched tick

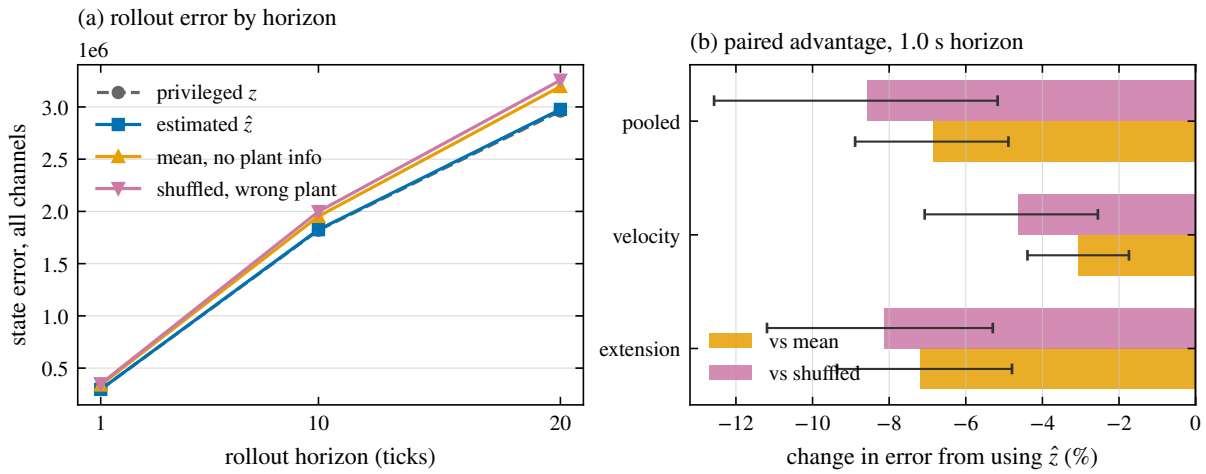


Figure 7.12: Predictive utility of the estimated latent, 36 held-out runs. (a) Free-running rollout error against horizon under the four latents. The estimate tracks the privileged latent at every horizon while both stay clear of an uninformative or a wrong one. (b) Paired advantage of the estimate at the 1.0 s horizon, per channel, as a percentage of the control arm's own error, with 95 % paired bootstrap intervals. Extension and velocity are the channels the deployed controller integrates, and both resolve.

the commanded motion is the same and only the plant differs. The result is a constructed sequence rather than a recorded one and is reported as such.

Eight pairs are used, chosen as the most separated in latent space among the sixteen reserved runs, and each is spliced at its midpoint. In all eight the estimate ends closer to the new plant than to the old one, and in all eight it moves away from the old plant after the splice. The median time to cross, that is for the estimate to become closer to the new plant than the old, is 0.18 s, and the median time to cover ninety per cent of the distance between the two settled values is 0.70 s. Both sit inside the 1.0 s observation window, which is the shortest interval in which a change can become fully visible to a causal estimator reading that window.

The estimator therefore re-identifies rather than merely classifying: it does not carry a plant label fixed at the start of an episode but revises it from history when the evidence changes.

7.6.4 The Benefit Does Not Scale With Plant Deviation

The natural expectation is that plant knowledge is worth more the further the plant sits from nominal, and it is the expectation under which an adaptive architecture is usually justified. It is not supported here. Ranking the 36 held-out runs by the magnitude of their factor deviation and measuring the latent's benefit as the paired gap between the mean-latent and estimated-latent arms, the milder half of the population returns a mean benefit of 2.53×10^5 and the more severe half 1.85×10^5, a ratio of 0.73. The correlation between deviation magnitude and benefit is -0.143.

The gain is therefore uniform across the plant family rather than concentrated where the plant is hardest. This is measured at model level, where the closed loop cannot absorb it, so it is a property of the latent rather than of the controller. It also forecloses one reading

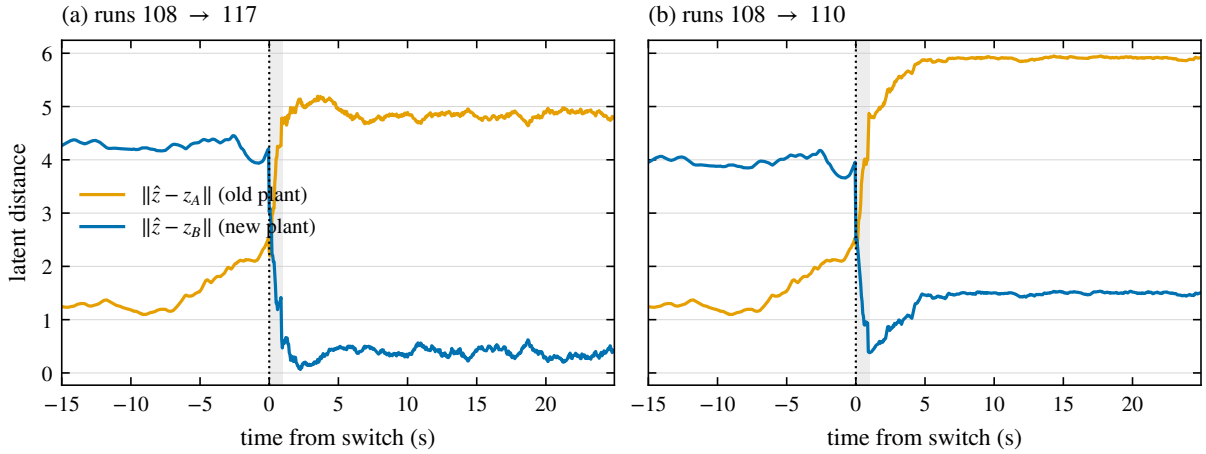


Figure 7.13: Latent response to a constructed plant switch, for the two most separated held-out pairs. The distance from the estimate to the old plant's latent rises and the distance to the new plant's falls, with the crossing occurring inside the estimator's own 1.0 s observation window, shaded. The dotted line marks the splice. The trajectory is constructed by splicing two runs at a matched tick, not recorded.

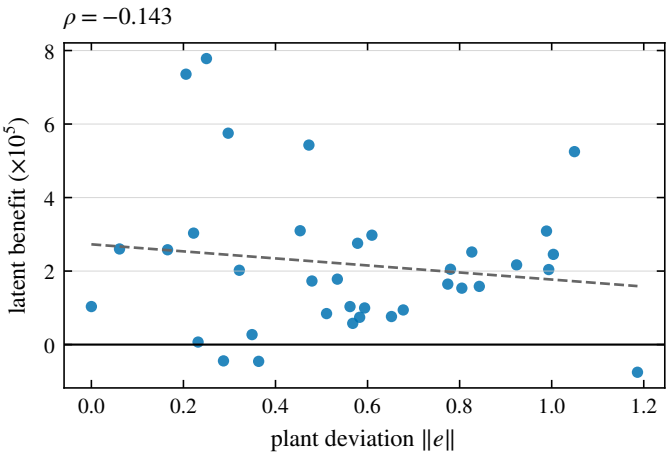


Figure 7.14: Latent benefit against plant deviation over the 36 held-out runs, measured as the paired gap between the mean-latent and estimated-latent arms at the 1.0 s horizon. The dashed line is a least-squares fit. The benefit does not grow with deviation, which forecloses the reading that a closed-loop effect exists but only at large mismatch.

Table 7.16: Response to a mid-episode plant change, over 33 paired (mesh, family) cells. Every paired bootstrap interval straddles zero and no sign count approaches unanimity.

Latent supplied	lateral (m)	cut height (m)	ground contact
online estimate	0.07964	0.01159	0.94012
frozen, pre-change	0.07867	0.01314	0.93742
shuffled	0.06822	0.00934	0.94655
mean	0.07595	0.01156	0.94361
online, plant unchanged	0.07974	0.01382	0.93505
IF reference	0.11537	0.00966	0.93865
online vs. frozen	+1.2%	−11.8%	+0.3%
better in	17/33	17/33	17/33

of the closed-loop result of Section 7.4: the absence of a benefit under large plant deviation is not a case of the controller hiding an effect that grows with mismatch, because no such growth exists to hide.

7.6.5 Response to a Mid-Episode Plant Change

This experiment measures the disturbance envelope of the deployed stack: how large a change in the plant it absorbs without loss of performance. The result is a specification, and it states when a fixed controller suffices.

Design.

The plant's valve effectiveness is scaled by 0.6 at a known instant inside the scored window. All arms share the policy, the mesh, the seed and the schedule, and differ only in what occupies the latent slot.

The *online* arm is the deployed system and the *frozen* arm latches a pre-change value, so it is identical up to the latch and then stops tracking; beating it is the adaptation result. Three controls bound the reading: *shuffled* carries the right latent distribution and the wrong plant, which rules out any benefit from extra input dimensions; *mean* is in distribution and carries no plant information; and *sham* repeats the online arm with the schedule firing and the plant left alone. IF is scored under the same schedule as a robust reference.

A zeroed latent is deliberately not used as the no-information arm. Measured on a real trace both channels sit at means of -1.19 and -2.34 with no positive samples, so zero lies two to three standard deviations outside the distribution; it would be an out-of-distribution arm wearing the label of an ablation. The *mean* arm does that job instead.

Result.

The arms sit within a few per cent of one another on every metric, and the sham arm explains why: the perturbation they were meant to respond to left the task untouched.

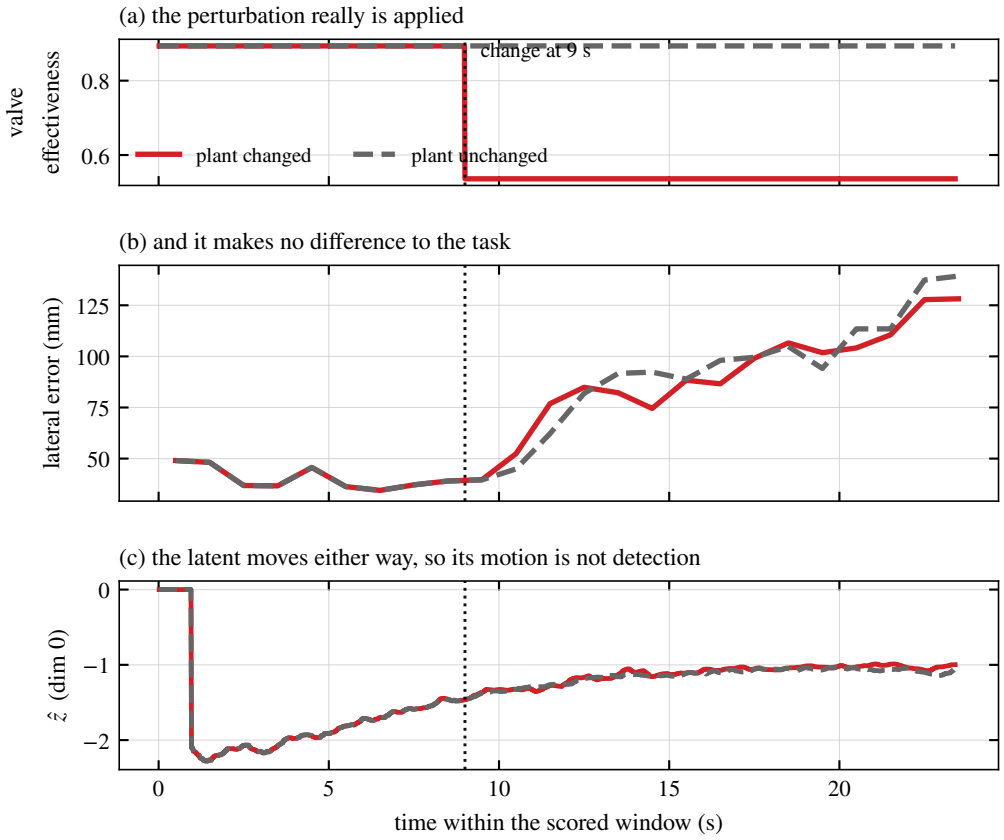


Figure 7.15: Both runs use the same policy, mesh and schedule; in one the valve effectiveness is scaled at 9 s and in the other the schedule fires and the plant is left alone. (a) The perturbation is genuinely applied. (b) The two runs nevertheless track together, and the rise in error after 9 s happens equally without any plant change. (c) The latent moves by much the same amount either way.

Effect of the perturbation.

Three measurements settle it, and they must be read in order.

1. **The perturbation is real.** The recorded plant parameter falls from 0.893 to 0.536, a ratio of 0.600 exactly as configured, at 9.0 s inside a window ending at 23.4 s.

2. **The task does not notice it.** Over the post-change window the perturbed run records 95.3 mm of lateral error against 98.1 mm unperturbed: a difference of −2.9%, in the wrong direction, and well inside the spread between cells.

3. **The estimator does not detect it.** $\hat{\mathbf{z}}$ shifts by 0.496 across the step when the plant changes, and by 0.473 when it does not.

A 40 % reduction in valve effectiveness moves neither the task nor the estimate. Tracking, cut height and ground contact all hold through a change of that size. The tempting conclusion – that no adaptation is required within this envelope – is not the one the experiment supports, because two properties of the design bound what it could have shown.

The first is the size of the perturbation. A perturbation that leaves the task within 2.9 % of its unperturbed value, in the wrong direction, has not created an error for adaptation to recover; measurement (2) above is a statement about the disturbance and not about the controller. A development measurement recorded in the evaluation harness puts a transport-delay step from 2 to 8 control steps at +22 % in lateral error where this valve step costs a fraction of a per cent, so perturbations with a measurable effect on this stack do exist. Injected delay is not the right one: the 117 ms plant dead time is present throughout training, so a policy that already compensates for it is being tested on a novel disturbance rather than on adaptation (Section 7.3.1). The point stands regardless of which knob is chosen – the perturbation has to be sized against a measured degradation curve before an adaptation arm can recover anything.

The second is the choice of factor. Valve effectiveness acts on flow, and the deployed estimator barely represents flow: linear decoding from the latent gives $R^2 = 0.114$ for flow against 0.956 for seal condition (Section 5.6.1). This experiment therefore asks an estimator that encodes seal condition to detect and compensate a sudden change in flow, which is not the strongest test of the adaptation path that the available factors permit.

The obvious correction – perturb seal instead – is not available on this stack, and why not is itself a result. Seal enters the plant only as an input to the command reduced-order model, and AIF runs the inverse/forward pair, which takes no factor input at all (Table 7.10). *The factor the estimator identifies best cannot be perturbed where the controller runs, and the surface that can be perturbed there carries the factor it identifies worst.* That is a property of the architecture as built, not of the estimator, and it bounds what any mid-episode experiment on this stack could have shown.

Two experiments follow from it, and they are specified in place of the single one this section reports. The first is *native*: tool mass steps mid-episode, a property that genuinely exists in the deployed simulation and reaches every stack unclamped through the physics engine, with no emulation anywhere. The estimator may well not benefit, since mass is the factor it decodes worst, and that is a legitimate outcome rather than a failed experiment. The second is a *positive control aligned with the estimator*: a seal-associated *gain proxy*, reproducing the amplitude half of the reduced-order model's seal law as a scaling of realised actuator motion, $\alpha_{\mathrm{eq}}(w) = (1 + 0.625\, w)^{-2}$. It is deliberately not called seal degradation: it omits the leakage term and the longer valve time constant that accompany seal condition in the training domain, and reproduces only the amplitude effect. It answers the narrower question of whether the adaptation path can exploit a change along the direction the estimator demonstrably learned. Both draw their magnitude from the degradation curve of Section 7.4.2, measured on the non-adaptive stack, so that the perturbation is sized to leave recovery headroom rather than chosen against the adaptive stack's own response. Neither is run here.

What the experiment does establish stands: the perturbation was applied ($0.893 \rightarrow 0.536$, exactly as configured), the sham arm shows the post-change rise in error occurs equally without any plant change, and no arm separates. That is a null with an identified cause, and the cause is the perturbation rather than the controller or the estimator.

One methodological point generalises beyond this experiment. Whole-episode means cannot express recovery: an arm that recovers in two seconds and one that never recovers differ by a few per cent once averaged over a 23-second window, which is what the arms

above report. The replacement experiments are scored on the transient instead – integrated absolute error over $[t_c, t_c + 1]$, $[t_c + 1, t_c + 3]$ and $[t_c + 3, t_c + 5]$, peak error within three seconds, and the time to re-enter and hold a band around the pre-change level, with *not recovered* reported as an outcome rather than a missing value.

The envelope boundary.

A perturbation reveals the edge of the envelope only if it degrades performance. Transport delay does not qualify: the 117 ms dead time is present throughout training, so the policies already accommodate it. A step in the *commanded* line produces a measurable response on every stack (Section 7.3.3) and is the instrument for that measurement.

7.6.6 An Oracle Reference for Plant-Identity Conditioning

One question decides whether building an estimator will pay: would perfect knowledge of the plant help this closed loop? It is answerable directly, by supplying the true factors through the privileged encoder, and the answer is obtained without training an estimator. The result is an oracle reference *within this encoder, adapter and policy*, not a universal bound on what plant knowledge could ever be worth: a different representation or a different downstream consumer could use the same information differently.

One property of these arms should be read before their scores. They differ only in what occupies the two latent channels of the actor observation, and they differ measurably, which establishes that the deployed policy consumes the latent rather than ignoring it. That is worth stating because it is the precondition for everything else in this section: a null from an actor indifferent to $\hat{\mathbf{z}}$ would say nothing about plant knowledge, whereas a null from an actor demonstrably sensitive to it is a statement about the loop. It is also why the deployed estimate can finish *worse* than an uninformative constant, since only an input that is used can do harm; that ordering is the subject of Section 7.7.

The plant is drawn per episode from the held-out factor set and is not perturbed mid-episode, so plant identity is a real and stable property of each run. Four latents are compared on the adaptive stack: the deployed estimate, a *privileged* latent read from the true factors by the frozen encoder, an in-distribution *mean* carrying no plant information, and a *shuffled* latent with the right distribution and the wrong plant. R and IF are scored under the same factor domain.

The absolute errors in Table 7.17 are larger than those in Table 7.4 for the same stacks, and the reason is the condition rather than the measurement: Section 7.2 scores the nominal plant, whereas every arm here faces a plant drawn from the held-out factor set. Held-out plants are harder, and the two tables should be read against their own rows rather than against each other.

The oracle result.

Knowing the plant exactly is worth -3.4% in lateral error and -4.5% in cut height, and neither survives its sign test: the oracle beats the no-information arm in 18 and 19 of 33 cells respectively, which is indistinguishable from chance. A latent read directly from the true factors cannot outperform a latent carrying no plant information at all.

Table 7.17: Closed-loop performance under per-episode plant variation, over 33 paired (mesh, family) cells. The privileged arm reads the true factors, so it bounds what an estimator could contribute *through this conditioning architecture*.

Latent supplied	lateral (m)	cut height (m)	ground contact
privileged (oracle)	0.07204	0.01082	0.94150
mean, no plant info	0.07456	0.01133	0.94060
shuffled, wrong plant	0.07021	0.00878	0.94551
estimated (deployed)	0.09002	0.01579	0.93200
R reference	0.08121	0.01034	0.96138
IF reference	0.09754	0.00824	0.94183

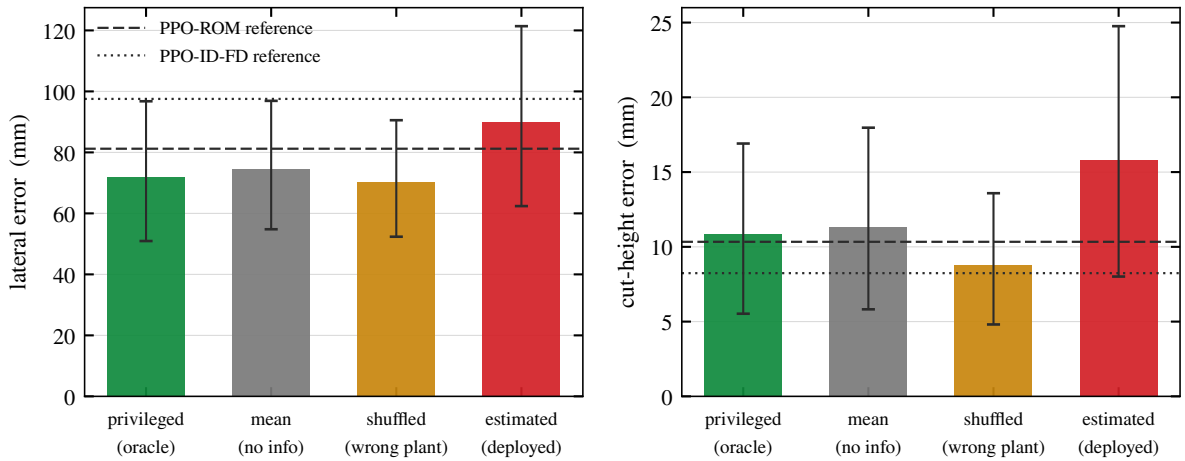


Figure 7.16: What plant knowledge is worth in closed loop. Bars are means over 33 paired cells with 95 % bootstrap intervals; the two reference stacks are drawn as lines. The privileged, mean and shuffled arms are indistinguishable, which is the flat ceiling. The deployed estimate trails the three constant-latent arms on both metrics.

Within this conditioning architecture, estimator accuracy is not the limiting factor: a latent read from the true factors produces only a small improvement that the sample cannot resolve, so effort spent improving the AAM would not have been repaid here. Run before an estimator is trained, this measurement redirects that effort. Whether it generalises to other latent-conditioned designs is a hypothesis this experiment cannot test, because it varies the latent inside one fixed encoder, adapter and policy.

Scope.

Three limits belong with these numbers, and the first is the most consequential. The privileged latent is read from all six true factors, but four of them – flow, compliance, seal condition and supply pressure – do not act on the inverse/forward plant this stack runs on (Table 7.10). What separates one held-out draw from another here is tool mass and centre-of-mass height, and those are the two factors the latent represents worst, at $R^2 = -0.008$ and 0.056 (Section 5.6.1). The oracle arm therefore supplies exact knowledge of four quantities the plant ignores and a poor encoding of the two it does not. That is

a bound on what this measurement could have shown: it is evidence that plant identity does not help *through this latent, on this plant*, and it is not the general statement that perfect plant knowledge is worth nothing to this controller. Re-running it on a plant that consumes the hydraulic factors, or with a latent that encodes the mechanical ones, are both open.

Two further limits belong with these numbers. The oracle arm necessarily differs from the others in the adapter as well as the latent, because an adapter must be trained for the latent it consumes and one trained on estimated latents refuses a privileged one; the -3.4% is therefore a comparison across two changes rather than one. And 33 cells resolve only large effects, so the flat ceiling bounds the available benefit rather than proving it exactly zero. What the experiment establishes is that the benefit is not large enough to find with the sample this chapter uses, under conditions chosen to favour it.

7.7 Why the Model-Level Gain Does Not Reach the Controller

The evidence assembled so far contains a discrepancy that has to be accounted for rather than reported. At model level the latent is worth a median of 22.3 % in free-running state error, its time-variation alone is worth 13.6 %, and the estimate is statistically indistinguishable from a privileged latent read off the true factors. In the closed loop none of that appears: the paired intervals against IF span zero (Section 7.2), a mid-episode plant change is absorbed without any arm separating (Section 7.6.5), and an oracle latent beats an uninformative one in 18 of 33 cells, which is chance (Section 7.6.6).

Four explanations are available and they are not equally supported. This section takes them in order of how directly they are measured: the ceiling itself, the plant information the baseline already receives, the two roles the architecture asks one latent to play, and the time constants of the loop that consumes it.

7.7.1 The Ceiling Bounds the Estimator Out of the Question

The first explanation is the one that removes an entire class of alternatives. A better estimator cannot recover the model-level gain, because the estimator is not what limits the closed loop: supplying the *true* factors through the privileged encoder is worth $-3.4\,\%$ in lateral error and $-4.5\,\%$ in cut height, and neither survives its sign test (Section 7.6.6). Whatever separates the model-level result from the closed-loop one therefore lies downstream of the latent's accuracy, in what the controller does with plant information rather than in how well that information is recovered.

One qualification bounds how far that reading travels, and it is the reason this section reports four explanations rather than one. The oracle is flat in part because most of what it knows is inert on this plant: four of the six factors do not enter the inverse/forward dynamics at all, and the two that do are the two the latent encodes worst (Table 7.10). A flat ceiling measured under those conditions bounds the benefit available *here* without establishing that plant knowledge is generally worthless to a controller of this kind. The three explanations that follow concern the downstream path and are measured independently of that qualification.

7.7.2 The Baseline Is Not Plant-Blind

Part of the explanation precedes the estimator entirely. The robust stacks receive the observable consequence of plant variation through the twelve model-error channels of Section 6.2, which carry the actuator's standing residual under whichever plant is active. That residual is what the policy corrects against, and it differs by plant. Conditioning on an estimated latent therefore does not supply the first plant-dependent signal the controller has seen; it adds an identity label on top of a consequence the policy was already acting on. Where the consequence is already observable and the controller already responds to it, naming the plant that produced it has less left to contribute, which is consistent with the size of the oracle result in Table 7.17.

7.7.3 The Two Consumers of the Latent

The ranking of the four arms identifies a property of the architecture rather than a deficiency of the estimator. The deployed estimate is the worst of the four, by 20.7% on lateral error and 39.3% on cut height against the mean latent: a genuine but imperfect estimate performs worse than a constant carrying no information, and worse than one that is actively wrong about the plant. That ordering is only explicable if what separates the arms is not their information content, and the paragraph below establishes that it is not.

The mechanism follows from Section 7.5.3, and it is the same property seen from the other side. The estimate varies within an episode by about nine times the spread between plants. The mean and shuffled arms supply a perfectly smooth signal; the deployed arm supplies one that moves continuously. The arms therefore differ in two respects, information and variability, and on this evidence the second dominates: the policy is better served by a constant it can ignore than by a signal that moves under it.

That is not a contradiction of the model-level result but its complement. In Section 7.5.3 the same time-variation was worth 13.6% to a one-step state predictor, which consumes the latent afresh at every tick and benefits from its responsiveness. The deployed actor is a memoryless MLP and does not integrate anything itself; what accumulates is the closed-loop trajectory, because a latent that moves under the actor changes its action from tick to tick and the plant carries the consequences forward. The same quantity is therefore plausibly informative to a one-step predictor and disruptive to a feedback loop, and the two roles were never separated in the architecture: one latent is routed to both. The arms differ in information and in variability at once, so this reading is the mechanism the evidence favours rather than one this experiment isolates. Section 7.7.4 separates the two.

7.7.4 Latent Bandwidth on the Policy Path

The arms of Section 7.6.6 change what the latent knows and how fast it moves at the same time, so they cannot say which of the two costs the task anything. This section holds the content fixed and varies only the speed. A one-pole low-pass with time constant τ is applied to the latent on the policy path alone; the inverse-model adapter keeps the full-rate estimate in every arm. The checkpoint, the plant draw and the 33 cells are those of Section 7.6.6, and no network is retrained.

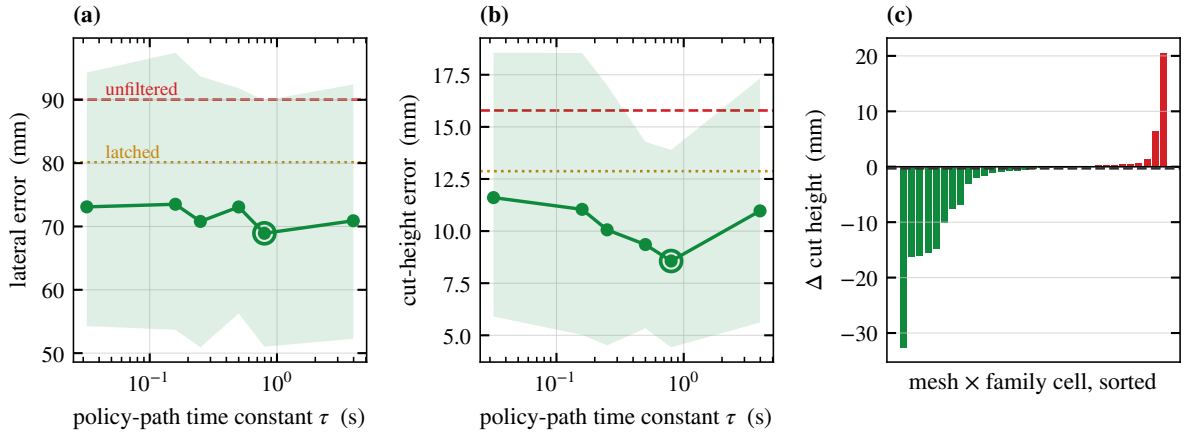


Figure 7.17: Task error against the policy-path time constant. (a) Lateral and (b) cut-height error, mean over 33 cells with a 95 % paired bootstrap band; the dashed line is the unfiltered estimate and the dotted line the latched one, which are the two ends of the axis. The ringed point is the minimum, at $\tau = 0.796$ s on both metrics and on ground contact. (c) Paired per-cell differences for the contrast between $\tau = 0.796$ s and $\tau = 0.032$ s on cut height, sorted, with the median dashed. The improvement is carried by a minority of cells.

Table 7.18: The contrast that resolves. Both arms are filtered, so the comparison isolates bandwidth rather than filtering. Paired over 33 cells.

	mean	median	cells ahead	Wilcoxon p
cut-height error	−26.2%	−14.5%	22/33	0.015
lateral error	−5.7%	−5.5%	19/33	0.150

Figure 7.17 reports the sweep. All three metrics take their best value at the same interior τ, and both ends of the axis are worse, so the response has an optimum rather than a trend. Cut-height error falls monotonically across the five filter strengths below that optimum.

Table 7.18 gives the contrast that resolves, and it is the one closed-loop result in this chapter that does so in favour of a design change. Slowing the latent from $\tau = 0.032$ s to 0.796 s improves cut-height tracking by 14.5% on the median as well as 26.2% on the mean, ahead in 22 of 33 cells at $p = 0.015$, on a metric fixed before the sweep was run. Lateral error moves the same way and does not resolve. The speed at which the latent is allowed to move therefore changes closed-loop tracking, at fixed content and with no network retrained, which is the prediction the two-consumers reading makes.

Three limits belong with that result. Against the *unfiltered* arm every filtered arm improves the mean by 11 to 46 % while the medians stay flat and no Wilcoxon test reaches $p < 0.10$; panel (c) shows why, since a minority of cells carry the mean. The mechanism is also not confirmed: the temporal variance of the latent as the actor reads it is 0.277 unfiltered and between 0.270 and 0.288 across every filtered arm, with no ordering in τ, so the latent's variation is mostly slower than the cutoffs applied and the proposed cause remains unmeasured. And the sweep varies one input to one checkpoint that was trained on the unfiltered signal, so the optimum may be a property of that training rather than of the architecture.

What survives is narrow and is stated as such: on this checkpoint, the speed at which the latent is allowed to move changes cut-height tracking, and the best setting is neither the raw estimate nor a constant.

7.7.5 The Loop Is Slower Than Anything the Latent Changes

The last explanation is a matter of time constants, and it is the one that generalises beyond this architecture. The measurements of Section 7.3 place four rates on the same axis. The estimator re-reads its window and issues a new latent every 50 ms. The hydraulics answer a valve command in 117 ms, amplitude-invariant and with an exact kinematic control. The command rate limit alone puts a 417 ms floor under a 100 mm step, three times the hydraulic dead time. And the stacks themselves settle such a step in 2.2 to 3.2 s, five to eight times that floor.

The latent therefore moves one to two orders of magnitude faster than the quantity it is trying to influence. Whatever it tells the actor about the plant, the actor's authority over the tool is exercised through a rate-limited command and a loop whose response is measured in seconds, so a plant change arrives, is observed through the model-error channels, and is corrected long before identifying *which* plant produced it could shorten the correction. It also predicts that a slow disturbance will be rejected without identification, which is consistent with – though not established by – the unmoved task under a 40 % loss of valve effectiveness (Section 7.6.5), a perturbation too small to separate the two explanations.

The four explanations are consistent with one another and with the data, and they differ in standing. The ceiling and the baseline's existing plant information are measured directly. The two-consumers reading is the mechanism the evidence favours, and the bandwidth sweep is its one direct test, which resolves on cut height and not on lateral error. The timescale argument is an account of why the ceiling is flat rather than an independent measurement, and it is the part of this explanation that would transfer to another machine as a prediction rather than as a result.

7.8 Integrated Discussion

The two research questions of this chapter are answered separately below, because the evidence behind them differs in kind: RQ5 rests on a controlled single-variable pair and RQ6 on within-checkpoint substitutions and a pre-estimator ceiling.

RQ5: What the Actuator Representation Changes

How does the choice of actuator representation affect tracking, command behaviour, latency and robustness under plant variation?

It changes the controller substantially, and the direction is not uniform across metrics. On the matched pair, driving the plant through the frozen inverse and forward models rather than the command reduced-order model lowers mean lateral error from 0.1026 m to 0.0703 m and cut-height error from 8.4 to 6.3 mm, and it holds the lateral criterion in 32 of 33 (mesh, family) cells against 30 of 33 (Section 7.2). The command behaviour behind that is the more informative result. R spends the *least* actuation of the three

and tracks worst, while reversing the direction of motion 40 times a second against 10.6 for IF, so its actuation is largely spent fighting itself (Section 7.3.5); that also accounts mechanically for its tool-acceleration and impact figures, which would otherwise read as unrelated observations. Representation therefore buys tracking by buying smoothness, not by buying effort.

Latency separates the representations in the hydraulic half of the chain and not the mechanical one, where all three stacks read the one-step resolution floor (Section 7.3.2). But the ordering there does not survive into the task: a shorter path from command to cylinder does not produce a quicker response at the tool, because the response to a commanded change is set by the command rate limit and the policy's own dynamics rather than by the actuation chain (Section 7.3.3). This is the most transferable finding of the RQ5 strand, and it is the reason a faster actuator path is not by itself a design goal on a machine of this class.

Two findings belong beside that comparison even though they are not attributable to the representation. IF fails the head-alignment gate badly, at 0.2247 with -0.855 on one mesh, which is the collapse under unconstrained fine-tuning seen against the acceptance contract; the adaptive stack is the only one of the three to hold alignment, at 0.9763, and it pays for its ground contact in hard impacts and tool acceleration (Section 7.2). And the lateral gate is exceeded by all three stacks on worst-family statistics, on a different family in each case, which makes that threshold a statement about the hardest trajectory in the library rather than a discriminator between architectures. No stack dominates the scorecard.

Robustness does not separate the representations at all, and both are robust in the strong sense. Every closed-loop figure in this chapter is measured on held-out meshes, on eleven trajectory families scored individually, and on plants drawn from sixteen reserved factor configurations disjoint from training by construction, of which the mechanical half reaches every stack and the hydraulic half only R (Section 7.4.1); underneath the policy the frozen inverse model was identified on Simscape alone and scores flat to a factor of 1.09 across domains on 64 IsaacLab runs outside its training distribution (Section 5.2.2). Generalisation across simulator, plant draw, terrain and trajectory is therefore established for both representations, and established without online adaptation.

Robustness *beyond* that held-out population is a weaker claim than this work can make, and weaker than an earlier reading of the sweep suggested. No stack's tracking moves across the randomisation sweep, but on IF and AIF that sweep did not vary the hydraulics at all – the inverse/forward plant takes no factor input, so only tool mass and centre-of-mass height act – and on R, where they do act, the upper half of the sweep is progressively pinned to the training envelope and the path cannot remove more than 62 % of actuator authority in any case (Section 7.4.1). The correct reading is that no boundary was located within a bounded instrument, not that the stacks are insensitive to plant variation. Locating the boundary is specified in Section 7.4.2 and left to future work.

RQ6: Whether Online Adaptation Pays

Where plant variation survives that robustness, is it identifiable from recent observations, and does online adaptation by latent identification improve closed-loop performance?

The first half is answered yes, with a condition. Plant identity is recoverable from observable history, but only from data that excites the plant: on open-loop excitation

a latent-conditioned predictor lowers free-running error by a median of 22.3 % in 5 of 5 seeds, while on data gathered under a task controller the identical architecture finishes behind its own no-latent baseline (Section 7.5.2). Even within the open-loop collection the recoverable band is narrow: the seal-condition decode falls from $R^2 = 0.703$ to zero between the second and third amplitude tier, and the cause is operating-point drift rather than saturation (Section 7.5.1). Inside that band the estimator is not the weak link – it is statistically indistinguishable from a privileged latent at every prediction horizon and re-identifies a constructed plant switch within its own observation window (Sections 7.6.2 and 7.6.3). What it recovers is a moving operating condition rather than a plant label: freezing the estimate at its own correct per-episode value costs 13.6 %, and making that frozen value wrong costs a further 0.43 %, a factor of thirty-two (Section 7.5.3).

The second half is answered no, under the conditions tested, and the null is bounded rather than merely observed. Every paired closed-loop interval spans zero, and an oracle latent read from the true factors is worth −3.4 % in lateral error at 18 of 33 cells, which is chance. The mid-episode plant change carries less weight than the other two: a 40 % reduction in valve effectiveness passes unnoticed by the task, by the estimator and by an unperturbed sham arm, but it perturbs a factor the estimator barely encodes and by an amount too small to create recoverable error, so it bounds the perturbation rather than the adaptation path (Section 7.6.5). That last measurement is the load-bearing one, because it is obtained *before* an estimator is built and shows that estimator accuracy was never the constraint (Section 7.6.6).

Three determinants explain the null and were separated in Section 7.7. The baseline is not plant-blind: the twelve model-error channels already carry the observable consequence of plant variation, so conditioning adds an identity label on top of a consequence the policy is already correcting. The architecture routes one latent to two consumers with opposite requirements – a one-step predictor rewarded for responsiveness and a memoryless actor in a feedback loop that the same responsiveness disturbs – and the bandwidth sweep, which holds content fixed and varies only speed, improves cut-height tracking by 14.5 % on the median at $p = 0.015$ with no network retrained. And the loop's own time constants leave adaptation little to do: a latent updating every 50 ms acts on a controller whose step response is floored at 417 ms by the command rate limit and settles in seconds.

The design that follows is to hold or filter the latent on the policy path while leaving it responsive on the model path. It is specified here and not tested beyond the single sweep of Section 7.7.4.

Outcome of the Declared Hypotheses

H1 is supported: an estimated latent lowers free-running state error in 5 of 5 matched seeds, at a median of −22.3 % with a paired interval excluding zero, so the residual gap between the Simscape training distribution and the deployment plant is recoverable without touching the inverse model, which is what the frozen-inverse architecture requires. H2 is supported: the estimate's time-variation is worth 13.6 % and its plant identity 0.43 %, both consistently signed across seeds, so the latent is a time-varying operating condition and not a plant label. H3 returns an interval rather than a verdict, and the interval is informative on ground contact – less than about 2.4 percentage points either way – while on the two tracking errors it is comparable to or wider than the baseline mean itself, so the

effect is bounded only loosely and equivalence is not established. H4 is supported in the sense that matters practically and refuted in the sense that was hoped for: the closed-loop effect *was* boundable before an estimator was built, and the bound it returned was flat, since a privileged latent read from the true factors does not beat an uninformative one at better than chance.

Relation to the Research Gap

Chapter 2 positioned this work against history-based adaptation methods that infer a low-dimensional plant code and condition a policy on it. This chapter locates two conditions those arguments leave implicit. A plant latent earns its place when it feeds a consumer that benefits from its responsiveness, and when the plant variation is large enough for identity to be worth having. Both are measurable before an estimator is trained: the identifiability envelope of Section 7.5.1 establishes the first and the oracle ceiling of Section 7.6.6 the second. Together they form a pre-deployment protocol, and that is the contribution of this chapter.

A third condition follows from the results taken together, and it reconciles the null reported here with the gains that motivate the method. Rapid motor adaptation and its descendants are demonstrated on legged robots, where the plant changes discontinuously at contact, the correction window is a fraction of a gait cycle, and the reported gains are success rates on terrain that makes the unconditioned baseline fall [Lee 20, Kumar 21]. In that setting feedback cannot substitute for identification, because the consequence of an unknown plant arrives before the loop can measure and correct it. The present system is the opposite case. Its plant is fixed within an episode, its actuator is heavily damped and in sustained contact, its task response is set by a command rate limit three times the hydraulic dead time, and it settles over seconds (Section 7.3.4). A loop with that much time observes the plant's effect through the model-error channels of Section 6.2 and corrects it, so the identity behind the effect adds little.

The condition can therefore be stated as a criterion rather than as a property of this machine: *latent conditioning pays where plant variation acts faster than the loop can observe and correct it.* That criterion covers both the locomotion results and the null reported here, and it is the boundary the adaptation literature reviewed in Chapter 2 generally leaves unstated. It also predicts where this estimator would pay on the present machine, since Section 7.6.2 shows the latent improving a model asked to predict before observing, which is the quantity a predictive controller consumes and a reactive one does not.

7.9 Threats to Validity

The Plant as a Learned Model

The deployment plant is the command reduced-order model driving PhysX, and R's actuator *is* that model. It is therefore a reference condition rather than a third competing approximator, and its advantages should not be read as an approximation result. The reduced-order model was calibrated against PhysX using parked shapes rather than the moving-verge task, so the calibration is weakest exactly where the cutter meets the ground, and the inverse and forward models received no comparable calibration.

An Unmeasured Input

The differential pressure the adaptive module takes as an input is produced by the actuator model and integrated at every tick, not sensed (Section 3.3.1). Within this simulation that is legitimate, because the model is the plant. On hardware it would require pressure transducers the FME 500 does not carry, and this is the channel where the module's measured advantage is concentrated. Any transfer of the model-level result therefore depends on instrumentation that is currently absent. No sensor-noise or quantisation model is applied anywhere in this work, so all observations are exact.

This leaves one question about the observation contract genuinely open, and it is carried here as a limitation rather than presented as settled. Every result in this work uses the pressure-inclusive observation row, in which measured differential pressure is one of the four signal groups the module consumes. A stricter alternative contract was specified alongside it, excluding differential pressure precisely because a deployed machine cannot supply it, and the choice between the two was never resolved for deployment. The cost of the stricter contract has since been measured. Retraining the estimator on the pressure-excluded row, under a matched budget and an otherwise identical dataset, split, objective and seed, raises the latent error by 3.93 % on validation and 3.41 % on the reserved test split, worse in both of two seeds. The deployable variant is therefore worse, as expected, but by a margin that does not disqualify it: removing the one channel carrying most of the hydraulic information costs a few per cent rather than the estimator's usefulness. Three limits belong with that figure. It is measured at model level and says nothing directly about task tracking; it rests on two seeds rather than the five used elsewhere in this chapter; and it uses a twenty-epoch budget rather than the schedule that produced the deployed checkpoint, so the absolute errors sit above those reported in Section 5.6 while the paired difference is the quantity of interest.

Resolution of the Evidence

The closed-loop comparisons use 33 (mesh, family) cells and bound differences between architectures at a few per cent; they do not establish equivalence. The evaluation is deterministic, so the replication unit is the cell and not the seed, and a sign count over evaluation seeds would report one observation several times.

Two experiments bound their disturbance rather than the architecture, and one of them bounds it more tightly than its design intended. The randomisation sweep found no breakdown, but the plant does not receive what the sweep requests. On the inverse/forward plant the four hydraulic factors are not consumed at all, so for two of the three stacks the sweep varied tool mass and centre-of-mass height only; on the command reduced-order plant they are consumed but clamped to the identified range above scale 1, and no more than 62 % of actuator authority can be removed through that path (Section 7.4.1). This affects the oracle experiment as well as the sweep: the privileged latent supplies four factors the deployed plant ignores alongside the two it does not, which bounds what that measurement could have shown (Section 7.6.6). The mid-episode plant change is a real perturbation but a small one, applied to a factor the estimator represents poorly. The replacement experiments are specified against a stress axis that reaches every stack – Equation (7.1) – rather than against the factor set, and their results are not part of this thesis. Neither licenses a robustness claim, and neither licenses the converse claim that

the plant cannot be degraded: what they establish is that the perturbations available in this work are mild and, in the sweep's case, partly bounded by the plant model itself. The sweep also stretches recorded factor draws rather than sampling new ones, so it says nothing about combinations the manifest never visited.

Conditions Not Identical Across Stacks

The three stacks are matched at evaluation but not in training. R and IF come from runs whose manifests agree on every recorded field; the adaptive stack is a different run, chosen for the reason in Section 7.1.5, and it differs from that pair in three recorded respects beyond the latent itself: a different seed (41, with terrain seed 41), an actor learning rate two orders of magnitude lower (3×10^{-6}) and no frozen-actor warm-up at all (0 against 150 iterations). Those settings were not arbitrary. Each stack was trained under the best configuration found for it during development, and the adaptive stack's lower learning rate reflects a stability judgement specific to an actor that consumes a latent moving within the episode; that rationale is the author's engineering judgement, since the manifests store the settings and not the reasoning behind them. One field is missing where it would be most useful: the R and IF manifests do not record the truck-speed schedule at all, while the adaptive run records 2 to 7 km/h sampled per episode, so it is the reference pair rather than the adaptive run that is undocumented in this respect. Its absence does not differentiate the matched pair, whose other fields agree, and the evaluation speed is pinned identically for all three. The oracle arm of Section 7.6.6 additionally uses a different adapter checkpoint, because an adapter must be trained for the latent it consumes, so that contrast spans two changes rather than one.

Quantities Not Recoverable from Operational Data

Closed-loop delay compensation is not measurable from a normal drive: reference and achieved path are coherent only below 0.3 Hz, where phase reflects controller dynamics rather than transport, so Section 7.3.3 injects a step instead. The valve's dynamic model is likewise not identifiable in either dataset, although its transport latency is.

Coverage

The factor design is finite and sparse: 118 runs with 16 reserved configurations, six factors, and an operating envelope that does not cover the full machine. Excitation amplitude is swept monotonically within each run of the open-loop collection by design, so that collection supports statements about amplitude rather than about elapsed time within an episode. All results are simulation-to-simulation; the Simscape-to-IsaacLab abstraction gap is characterised in Chapter 5 but the simulation-to-reality gap is not addressed anywhere in this work.

8. Conclusion and Future Work

This thesis asked how hydraulic actuator dynamics learned from high-fidelity simulation can be reduced, transferred into a computationally efficient robotic simulation and used for reinforcement-learning control, and under what conditions online adaptation adds anything to the robustness those learned models already provide. The six research questions of Section 1.4 are answered in the order they were asked, which is also the order in which the answers depend on one another.

8.1 Summary of Findings

The findings are stated below in the order of the research questions. The first four concern how hydraulic actuation is learned, transferred and controlled. The last two concern online adaptation.

The inverse and forward representation identified in Simscape transfers to the deployment simulation without retraining (RQ1, RQ2). The inverse model was trained on randomised Simscape campaigns and evaluated on 64 IsaacLab deployment runs drawn entirely outside that training distribution, where it reproduces the valve command to within 0.1739 against a recorded signal of RMS magnitude 0.4350, which is roughly 84 % of the valve-command variance. The accuracy is nearly flat across domains, a factor of 1.09 between the best and the worst, and the ordering does not follow plant severity. No deployment-specific retraining was required, and the inverse model stays frozen everywhere downstream. The reduced-order model is the exception and is treated as one throughout: it received a small bounded calibration against the deployment plant, so it is transferred with a correction rather than without one.

Supervised accuracy is not evidence of control authority (RQ1). The first forward model scored near-perfectly and had learned to ignore the valve command, reconstructing motion from the state history instead. Nothing in the validation loss showed it. Ablating the command exposed it, and redesigning the excitation repaired it. Rollout error fell from 75.2 mm to 12.4 mm at 0.5 s and from 157.9 mm to 27.3 mm at 1.0 s, and removing the valve input from the repaired model then costs a factor of 1.81. Excitation design and input ablation are therefore reported here as required tests for a learned actuator

intended for control. This finding concerns how such a model is validated rather than what it represents, so it applies beyond the machine studied here.

Two actuator representations both support closed-loop control, and they do not produce the same controller (RQ3, RQ5). The command-conditioned reduced-order model reaches $R^2 = 0.998$ on the internal valve state and 0.97 on extension, with a diagonally dominant extension–command Jacobian of condition number 2.14, and it runs inside GPU-parallel training. The inverse–forward chain composes two separately learned models in series, which could have compounded their errors and did not. The two representations differ in the control strategy they induce from the same policy interface. R spends the least command and reverses about 40 times per second, against 10.6 for IF, which converts greater actuator effort into lower lateral error. The choice of representation therefore affects the resulting controller and is not an implementation detail.

The standing of that comparison is stated here because it governs how the three stacks are read throughout. R and IF agree on every recorded training field and differ only in the actuator path, so the contrast between them is controlled and the difference above *is* attributable to the representation. AIF is a separately trained run at a different seed, a lower actor learning rate and no frozen-actor warm-up (Section 6.9); it is reported as a deployed controller demonstrating the adaptive stack end to end, not as a ranked competitor, and no claim here rests on its position relative to the other two. Only one training run exists per stack, so seed variance is not estimated even for the matched pair. Claims about the adaptation path are carried by within-checkpoint ablations in which one input is substituted and everything else is held fixed.

A learned policy can replace the classical controller at runtime (RQ4). The damped-least-squares controller supplies privileged demonstrations during training and is then removed; the distilled policy emits desired cylinder motion directly into the learned actuator and is the entire deployed controller. Over 33 evaluation cells IF holds a mean lateral error of 70.3 mm, a median of 28.1 mm and 95.29 % ground contact. Reward ascent alone does not deliver this result. Unconstrained fine-tuning raised total return while degrading the primary tracking objective by up to a factor of 12.9, with the critic remaining healthy throughout. The failure is therefore one of objective mismatch rather than of optimisation, and it is the reason the deployed policies are anchored to the demonstrated behaviour.

The robust stacks generalise across simulator, plant, terrain and trajectory without online adaptation (RQ2, RQ5). Every closed-loop result is measured on held-out meshes, on eleven trajectory families scored individually, and on plants drawn from sixteen reserved factor configurations whose keys are disjoint from training and validation by construction, so a scored run is an unseen plant rather than an unseen episode – on the inverse/forward stacks, unseen in its mechanical factors, since the hydraulic ones do not reach that plant. Under those conditions IF holds the lateral criterion in 32 of 33 cells at a median of 0.0281 m, and the frozen inverse model beneath it, identified on Simscape alone, scores flat to a factor of 1.09 across 64 IsaacLab runs outside its training distribution. This is the broadest claim the work supports, and it requires no plant estimation of any kind.

No robustness boundary was located, and the instrument that looked for it is bounded (RQ5). Doubling every deployment-factor deviation moves no stack's median, and a 40 % reduction in valve effectiveness does not move the task either. Neither result

licenses a robustness claim, because an audit of the perturbation path shows that it cannot deliver the stress it requests: the inverse/forward plant consumes no factor input, so on two of the three stacks the sweep varied tool mass and centre-of-mass height alone, and on the third the hydraulic factors are clamped back to their identified ranges above scale 1, with at most 62 % of actuator authority removable by that route (Section 7.4.1). What the closed loop absorbs is therefore established only for the perturbations this work can actually apply, and the point at which command authority becomes insufficient remains unmeasured. This qualification carries into the last finding: knowledge of plant identity has little to contribute *where the plant variation reaching the controller is small*, which is the condition these experiments establish rather than the general one.

The remaining findings concern adaptation itself (RQ6).

Under the controller, signal set and excitation regime examined here, plant identity is recoverable only from deliberate excitation, and the limit has a mechanism. Data gathered while this task controller drives the machine does not separate plants, because the command is computed from the state and the two are nearly redundant. This is a statement about the dataset the controller produces, not a claim that closed-loop identification is impossible in general: dedicated closed-loop identification methods exist and were not applied. Open-loop excitation repairs it here. Identity is then legible at low amplitude and gone by amplitude 0.35, and the reason is operating-point drift rather than actuator saturation: by that amplitude the plants have desynchronised in position by half a probe stroke, and a one-second window cannot separate a leaky cylinder from one sitting five centimetres lower. This gives a mechanism-level account of the excitation amplitude at which identification stops working on this plant, which among the excitation-design work reviewed in Chapter 2 is generally left implicit.

The frozen inverse model is sufficient. A state predictor conditioned on an estimated latent, trained without the inverse model in the loop, lowers free-running prediction error by 24.7% against a matched no-latent baseline, in all five seeds (H1, Section 7.6.1). The residual gap between the training distribution and the deployment plant is recoverable without retraining the inverse model, which is what the frozen-inverse architecture requires. The same comparison on controller-driven data put the latent 1.8% behind, so the excitation result above is what makes this one possible.

The value of the latent is its variation over time, not the plant identity it encodes. Substituting a per-episode constant for the estimate costs 13.6%; substituting a constant that is wrong about the plant costs 0.43%, a factor of about thirty-two between the two (H2, Section 7.5.3). The estimate carries little plant identity that the consumer uses, and what it does carry is temporal. Whether the same holds for other architectures that infer a plant code is untested here, but it is a cheap thing to check and it is not what such architectures are usually designed around.

The estimator recovers what the privileged latent carries, and re-identifies when the plant changes (RQ6). Supplied to the frozen dynamics model, the causal estimate is indistinguishable from a latent read off the true factors, ahead in 18 of 36 held-out runs, while both are clearly ahead of an uninformative latent, 33 of 36, and of a wrong one, 29 of 36, with paired intervals excluding zero on extension and velocity as well as pressure (Section 7.6.2). On a constructed plant switch the estimate moves to the new plant in all eight pairs tested, crossing in 0.18 s and settling in 0.70 s, both inside its

own one-second window. The estimator is therefore not the limiting component. One expectation it does not meet is that the benefit should grow with the size of the plant deviation: it does not, with a correlation of -0.143 and a ratio of 0.73 between severe and mild plants, so the gain is uniform across the family rather than concentrated where the plant is hardest.

The closed-loop benefit of plant knowledge is bounded by what perfect plant knowledge achieves. A privileged latent read from the true factors, which is the most an estimator could supply *through this encoder, adapter and policy*, improves lateral error by 3.4% and cut-height error by 4.5%, ahead in 18 and 19 of 33 cells (H4, Section 7.6.6), which is indistinguishable from chance. The deployed estimator's effect is smaller than the protocol resolves: every paired interval on the three tracking metrics spans zero. What the measurement returns is the size of the effect rather than its sign, and the bound it supplies is uneven. Ground contact is constrained usefully, to within about 2.4 percentage points; the tracking intervals are as wide as the baseline means themselves and bound the effect only loosely. Neither is an equivalence claim, and the tracking interval width is the sample-size specification a later equivalence test would start from. Two contrasts do exclude zero, and both go against the adaptive stack: hard impacts and p_{95} tool acceleration are worse by 33% and 32%. Both aggregate per mesh, so they rest on three units and indicate a sign rather than a bound.

8.2 Design Implications

The results carry design consequences for each of the three parts of this work: how a hydraulic actuator is identified and transferred, how a policy is trained to act through it, and how plant adaptation is fitted around both. They are stated below in that order, because that is the order in which they constrain a successor design. Several were established by a measured failure rather than by a successful result, and are stronger for it.

Identification and Transfer

Design the excitation for identifiability, and accept the model on an ablation rather than on a score. A task controller computes its command from the state, so the two are nearly redundant and the valve is not causally identifiable from the resulting data however varied the control effort looks: the closed-loop batch spends a quarter of its time saturated and half at mid-authority, and is still uninformative. A forward model trained on it reached one-step $R^2 \approx 0.999$ while ignoring the valve entirely, and no validation loss computed on that distribution could have revealed it. Zeroing the command did, and open-loop excitation repaired it, at which point removing the valve cost a factor of 1.81. The implication is that a learned actuator intended for control needs a dedicated excitation campaign and an input-ablation acceptance test, and that predictive accuracy should never be the criterion on which such a model is promoted.

Embed the physics that is known and bound what is learned. The reduced-order model propagates pressure, force, velocity and extension through explicit updates with trainable effective parameters, and confines the neural terms to valve realisation, configuration load and residuals bounded to 10 % of the pressure and velocity step limits and 1 cm on position. That bound is what makes the model safe to place inside a training

loop: even a badly fitted residual cannot produce an unphysical state jump. The structure also earns its keep at identification time, since encoding the orifice and continuity relations makes the pressure dependence of valve authority architectural rather than something to be rediscovered. Pinning a single global valve gain instead was tried and rejected, because measured authority varies by a factor of 15–75 across pressure quartiles and can change sign.

Prefer a frozen model with a bounded, gated correction to retraining. The inverse model transferred across the deployment plant family with a spread of only a factor of 1.09 between the best and worst domain, and an ordering that does not follow plant severity, so it was deployed frozen everywhere downstream. Where a deployment offset did have to be absorbed, it was absorbed by a correction capped at 3 mm and gated to vanish at zero command, with every hydraulic parameter frozen and the original data retained as replay. That construction preserves the identified physics, keeps a commanded hold a hold, and confines the deployment-specific part of the model to a component whose authority is bounded by construction.

Control Methodology

When the acceptance criterion is not the reward, bound the ascent rather than repair the reward. Unconstrained PPO from a distilled checkpoint raised total return while degrading the primary tracking objective by up to a factor of 12.9, with critic explained variance healthy in the 0.85–0.95 band throughout. That is the optimizer working correctly on the wrong functional: the reward is a proxy, acceptance is a vector of physical gates, and the proxy's optimum lies outside the feasible set while the distilled policy lies inside it. The remedy adopted here constrains the update instead of rewriting the objective. A forward-KL anchor to a frozen reference penalises the *abandonment* of reference behaviour, where the reverse direction would permit collapse onto a convenient subset of it; the exploration scale is held fixed so that entropy collapse cannot shrink the divergence while the commanded action drifts; and the anchor advances only through checkpoints that have passed the gates. The general form is that a competent teacher plus a proxy objective calls for a trust region referred to a verified policy, not for more reward engineering.

Price what is gated, and log every term by name. A quantity that is gated but not priced is traded away by the optimizer: the only adaptive checkpoint to clear the ground-contact gate over the full path was the one whose reward priced cylinder headroom explicitly. The same lesson appears in the reward's own history, where an inherited formulation of 53 weighted fields carried three terms working against the task for thirteen training campaigns, undetected because logging reported an aggregate and omitted the two largest lateral terms. Shaping must also retain a far-field gradient, since the Gaussian form previously used is effectively flat beyond about two scale lengths and was measured to be a factor of 520 weaker at 0.5 m of error where the failing trajectory families live.

Where no baseline is inherited, the action contract is the safety argument. The deployed policy is not a residual: a zero action leaves the arm stationary, and nothing is summed with the policy output at runtime. Authority is therefore bounded by the chain the action passes through rather than by proximity to a nominal controller, with a rate limit, a filter, a shield that fades authority as travel headroom is consumed, a command

scale, and an anti-windup cap on how far the integrated target may lead the measured extension. That last cap is load-bearing and axis-dependent: relaxing it from 18 mm to 50 mm halved lateral error on the families needing a sustained standing offset, while the same relaxation on the head-pivot cylinder drove alignment from 0.9507 to 0.4636.

Plant Adaptation

Route a plant latent to the consumer whose objective rewards its responsiveness. The value of the estimated latent lies in its variation over time rather than in the plant identity it carries, by a factor of about thirty-two, and the two consumers in this architecture want opposite things from that variation. A one-step predictor reads the latent afresh at each tick and gains 13.6 % from its movement; the deployed actor is a memoryless MLP that integrates nothing, so a latent moving under it changes the action from tick to tick and the closed-loop trajectory accumulates the consequence. Since one signal is routed to both, the design that follows is to split the paths: hold or filter the latent where a feedback policy consumes it and leave it responsive where a one-step model consumes it. Filtering on the policy path alone, at fixed content, improves cut-height tracking by 14.5 % on the median in 22 of 33 cells with an interior optimum at $\tau = 0.796\,\mathrm{s}$. Two qualifications belong with that: the temporal variance of the latent as the actor reads it does not order with τ, so the proposed cause is not the measured one, and the optimum may belong to a policy trained on the unfiltered signal rather than to the architecture.

Run the two pre-deployment measurements first. The identifiability envelope establishes whether plant identity is present in the data the machine produces. The oracle ceiling establishes whether the controller's behaviour depends on what that identity describes. Both are cheap, both precede estimator training, and either can end the effort before it is incurred. On this task the second alone would have shown that no estimator feeding this conditioning architecture, however accurate, recovers a resolvable improvement, and it would have shown it before the estimator, the adapter and the conditioned policy were built. It converts an open-ended engineering effort into a measurement, and it is the design contribution this thesis most expects to transfer.

8.3 Contributions

Section 1.4 stated five contributions. Their standing at the end of the work is as follows.

Transferable hydraulic actuator learning is delivered. Both representations were identified from Simscape and realised in IsaacLab, and the inverse model transfers without deployment-specific retraining.

The **control-relevant identification and validation** procedure is delivered, and it is the contribution most likely to transfer independently of this machine. It covers both the input-ablation test that exposed a forward model with no command authority and the identifiability audit that established the excitation envelope, including the mechanism behind the upper amplitude limit.

The **reduced-order actuator model** is delivered and carries everything downstream, being accurate enough to serve as the deployment plant and cheap enough to evaluate inside GPU-parallel training, which the stiff implicit solver of the high-fidelity model it

replaces cannot support. The qualification stated throughout stands. It was calibrated against IsaacLab and is therefore the reference condition rather than an independent competitor to the inverse and forward pair.

Policy-only control through learned hydraulics is delivered for both representations, with the classical controller absent from the deployed loop.

The measurements of **robustness and the limits of adaptation** are delivered, and their main product is the discipline that produced them rather than any single number. Two findings exist only because of it: that a selection window covering 8.9 of 26 m certifies a third of an episode, and that a quantity which is gated but not priced is traded away by the optimizer. The causal plant estimator and the bounded adaptation path behave as specified, with no privileged access, no future information, bounded output, and reversion to the robust actuator path on invalid input. They remain enablers of this contribution rather than claims in their own right.

The **eight-gate promotion criterion** is delivered as the non-reward criterion the training loop and the checkpoint selection both use, which is the role it was built for and the role every result here gives it. One property is recorded for reuse: five of the eight gates separate these three stacks and carry the comparative claims, while lateral, hard impacts and tool acceleration do not (Section 3.9.1). Re-deriving those three against the widened trajectory library is specification work rather than control work.

8.4 Limitations

The evidence is simulation-to-simulation, and the deployment plant is itself a learned model. No claim here is a claim about hardware.

The differential pressure the estimator consumes is produced by the actuator model rather than measured, so transferring the model-level result to hardware requires instrumentation the FME 500 does not carry. An observation contract remains unresolved alongside this: every result uses the pressure-inclusive row, while the stricter deployable alternative excludes measured differential pressure. Since the estimator's advantage is concentrated in that channel, the deployable variant should be expected to perform worse, and by how much is not established.

The closed-loop comparisons resolve differences to a few per cent over 33 paired conditions and do not establish equivalence. The available plant variation bounds the disturbance rather than the architecture, and more tightly than the sweep's labels suggest: a stack shown to absorb a 40% reduction in valve effectiveness and a nominal doubling of every deployment factor has been shown to tolerate what actually reached it, which on the inverse/forward plant is a mechanical perturbation only. The three stacks are matched on evaluation but not on training budget, so the comparison describes three deployed controllers rather than three equally tuned training procedures.

Finally, the bound is a bound on this loop. A commanded step cannot complete faster than 417 ms at the command rate limit while the plant answers in 117 ms, so the task response is set downstream of anything the latent influences. That is close to the least favourable case for adaptation of this kind. The finding generalises through the mechanism rather than through the number.

8.5 Future Work

The evidence in this thesis is simulation-to-simulation, and the step that would extend it furthest is the one it does not take: placing the learned actuator models and the distilled policy on the machine they were built for.

That step begins with instrumentation, because the FME 500 as delivered carries no cylinder-chamber pressure transducers, and the differential pressure the models propagate exists here only as an actuator-model state. Fitting pressure transducers, extension sensing and a rod load pin converts three modelled quantities into measured ones. The collection instrument is already designed and validated: the open-loop excitation schedule of Chapter 4 was built to make the valve causally identifiable, and the identifiability audit of Section 7.5.1 applies to machine data unchanged.

Fine-tuning on that data is then a well-posed problem rather than an open-ended one. The inverse model transferred across the deployment plant family with a spread of only a factor of 1.09, so the gap to a real machine is plausibly a bounded offset rather than a different function, and the mechanism for absorbing such an offset has already been validated once: a correction capped at 3 mm and gated to vanish at zero command, with every hydraulic parameter frozen and the original data retained as replay. The input-ablation test of Section 5.3.1 decides whether that suffices, since a correction obliged to carry command authority is no longer a correction. For the policy, the reference-anchored construction of Section 6.4 transfers with its guarantee intact, and that guarantee is worth more on a machine than in simulation: it holds the policy within a bounded divergence of a checkpoint verified against physical acceptance and restores it atomically when an update violates that bound.

A. System Parameters and Coordinate Mappings

The kinematic structure, the cylinder-to-joint mapping and the calibrated cutter Jacobian are derived in Section 3.2 and are not repeated here. This appendix records the quantities a reader needs in order to reproduce a run and cannot read off those derivations: the rate of every layer, the limits the command path enforces, and the geometry of the scored course. Every value below is read from the shipped configuration or from a stored evaluation record, and the source is named in each case.

A.1 Rates

Five different rates operate in this system and confusing them is the most common way to misread a result. The control step is 1/60 s, the physics substep is half that because the environment runs with a decimation of two, and every learned component operates on the 20 Hz grid at which the reduced-order model and the identification data were sampled.

Table A.1: The rate of every layer. Control and physics rates are read from the stored evaluation records (`control_dt_s`); the learned-model sample time is stored inside the frozen forward-model checkpoint (`sample_time_s`); the AAM update period is the runtime configuration `update_period_ticks`.

Layer	Rate	Period
PhysX physics substep	120 Hz	8.33 ms
Policy and control step	60 Hz	16.67 ms
Inverse / forward model	20 Hz	50 ms
AAM inference update	20 Hz	50 ms (every 3rd control step)
Simscape logging grid	20 Hz	50 ms

Two derived latencies follow from Table A.1 and are used in Section 7.3. The AAM consumes a history of 20 rows on the 50 ms grid, so its input window is exactly 1.0 s and no estimate at all is available for the first 1.0 s after a reset; until then the adapter is bypassed

and the frozen inverse action is returned unchanged. Its output smoother averages five consecutive predictions, which adds a further 250 ms of group delay.

A.2 Command-Path Limits

The command path enforces four limits, all of which bound the policy rather than advise it. They are listed in Table A.2.

Table A.2: Limits enforced on the command path, per cylinder where applicable. Valve command limits and the adapter residual bound are stored inside the frozen adapter checkpoint (`command_limits`, `residual_bound`); lead caps and the c3 reserve are read from the stored evaluation records.

Limit	c1	c2	c3	c4	Meaning
Valve command limit	0.95	0.95	0.80	0.80	accumulated command is clipped here
Adapter residual bound	0.15	0.15	0.15	0.15	maximum adapter correction
Target lead cap (m)	0.03	0.03	0.03	0.02	how far the target may lead the state
c3 extension reserve	—	0.008 m on c3			solver-enforced stroke reserve

The lead cap is the mechanism by which the reference governor prevents the commanded target from running away from the measured state, and the c3 reserve is enforced by the solver while headroom is still scored against the original stroke, so the gate is not satisfied by moving the goalposts. The adapter residual bound is what makes the adaptive path fail safe: an adapter output that is invalid, uninitialised or late is replaced by zero, which returns the stack to the frozen-inverse configuration rather than to an untested one.

A.3 Course and Episode Geometry

Every closed-loop result in Chapter 7 uses the geometry in Table A.3. The scored window is the full course: the count of control steps differs between evaluations only because the carrier speed does, and it is the travelled distance rather than the step count that defines the window.

Table A.3: Course geometry and the two evaluation speeds that appear in this thesis. Values are read from the stored evaluation records (`tracking_active_length_m`, `max_truck_travel_m`, `steps`).

Quantity	Chapter 7 protocol	Earlier 2 km/h records
Tracking line length	26.0 m	26.0 m
Carrier speed	4.0 km/h	2.0 km/h
Control steps scored	1403	2808
Truck travel achieved	25.98 m	26.00 m
Episode length	47.3 s	46.8 s
Trajectory families	11	8

The right-hand column is included because stored records under it exist in the project and are not comparable to the results of Chapter 7; Section 7.1.5 explains why one such record, which passes all eight gates, is not used.

B. Dataset Manifests and Factor Ranges

B.1 Trajectories

Each run receives its own reference trajectory, sampled rather than scripted. Between twelve and twenty waypoints are drawn by Latin hypercube sampling within the joint limits, and the arm moves between them without dwelling, so that the data contains transitions rather than long static holds. Segment durations are drawn from four speed classes, slow (1.5–2.5 s), medium (0.8–1.5 s), fast (0.3–0.7 s), and variable (0.5–2.5 s), and about a third of the runs receive additional exploratory perturbations on top of the waypoint path.

An earlier version of this generator used four to eight waypoints with dwell periods between them. It produced data in which the arm was stationary for a large fraction of the time, which flatters any model that learns to predict persistence. The current settings were chosen to remove that easy mass of near-static samples, and the increase in waypoint count is the single change that most improved the usefulness of the batch.

B.2 Execution and Logging

Each run simulates 22 s of operation. The solver is the stiff implicit `ode23t` integrator in accelerator mode with relative and absolute tolerances of 10^{-2} and 10^{-3}. The loose tolerances are a deliberate throughput compromise, and their effect is bounded by the consistency checks of Section 4.10. Output is written on a fixed 0.05 s grid, giving 441 logged samples per run, and the batch is checkpointed to disk every hundred runs so that a failure late in a multi-day batch does not cost the whole collection.

For every run the archive stores the time vector, the five valve commands, the four cylinder extensions, the four chamber pressure differences, the four rod forces, the five joint positions and velocities, the reference position and velocity signals, and, crucially, the complete parameter structure that produced the run. That last item is what makes the batch usable for the latent-adaptation work: the privileged plant descriptor $\boldsymbol{\xi}$ of Section 2.6 is not something that has to be reconstructed later, it is recorded alongside the trajectory.

The cost quantifies the argument made in Section 2.1.3. A single 64 s open-loop run takes roughly 2000 s of wall time on the collection machine, a real-time factor of about 0.03. The Simscape plant runs some thirty times slower than the machine it models. No policy-optimisation loop that needs millions of environment steps can be built on that, which is why the learned actuator surrogates exist.

B.3 Parameter Ranges and Dataset Audit

Table B.1: Randomized hydraulic parameters. The seal-wear coordinate is drawn independently for each of the four cylinders and drives three physically linked quantities at once.

Parameter	Nominal	Randomized range	Acts on
Entrained air fraction ε_{air}	0.005	0.001 – 0.080	fluid compliance
Dead-volume fraction ε_{vol}	0.020	0.005 – 0.080	fluid compliance
Discharge coefficient c_d	0.70	0.56 – 0.84	valve flow gain
Seal wear $w_{\text{seal},i}$	0	0 – 0.25 (per cylinder)	clearance, leakage, spool lag
Relief setting p_{set}	27.6 MPa	24.8 MPa – 30.4 MPa	pressure ceiling

Table B.2: Randomized mechanical parameters. The rigid-body perturbations are applied independently to each of the fourteen bodies.

Parameter	Nominal	Randomized range
Body mass	CAD value	±20 %
Centre-of-mass offset	CAD value	±5 %
Inertia tensor and products	CAD value	consistent with mass draw
Joint stiffness, J1 and J2	60 kN/m	40 – 120 kN/m
Joint stiffness, J3 and J4	60 kN/m	40 – 120 kN/m
Joint stiffness, J5	25 kN/m	15 – 60 kN/m
Joint damping	250	100 – 600

Table B.3: The six randomized physical factors of the IsaacLab adaptation collection and their final ranges. One factor vector is drawn per run and held fixed for the whole run.

Coordinate	Range	Domain	Acts on
e_{flow}	$[-0.25, 0.25]$	hydraulic	valve flow gain
$e_{\text{compliance}}$	$[-0.25, 0.10]$	hydraulic	effective chamber compliance
e_{seal}	$[0, 1]$	hydraulic	seal wear, leakage, spool lag
e_{pressure}	$[-0.15, 0.15]$	hydraulic	supply/relief pressure level
e_{mass}	$[0.8, 1.5]$	mechanical	cutter mass scale
e_{COM_z}	$[-0.10, 0.05]$	mechanical	cutter centre-of-mass height

B.4 The Frozen Adaptation Collection

The IsaacLab adaptation collection is frozen, and every model-level result in this thesis is fitted and scored on it. Its manifest records 118 runs, one factor vector per run, held fixed

Table B.4: Audit of the strict open-loop excitation export. All checks are computed by the extraction stage and stored with the data; the export is accepted only if every one of them passes.

Check	Result	Interpretation
Rows, runs, columns	25 600 / 4 / 49	four complete runs
Median sample time	0.01 s	as commanded
Maximum sample-time error	9.1×10^{-15} s	numerically exact grid
Maximum Δextension error	0	targets match state differences
Maximum Δvelocity error	0	targets match state differences
Maximum $\Delta(\Delta p)$ error	0 Pa	targets match state differences
Non-finite values	0	no solver escapes
Valid fraction	0.937	startup, gaps, shutdown excluded
Command rank	4	all four channels independent
Lagged excitation rank	80	history matrix full rank
Valve saturation	8.5 % per channel	limits reached but not dominant
Stroke coverage	12 % – 33 %	above floor, below target

for the duration of that run. The domain composition and the run-level split are given in Table B.5.

Table B.5: Composition of the frozen adaptation collection, counted directly from the manifest. The run-level split is 102 training runs against 16 held-out test runs; the training portion is further divided 82/20 into fitting and validation runs, so that no window from a validation or test run is ever seen during fitting.

Domain	Runs	Factors varied
`combined`	64	all six
`hydraulic`	40	flow, compliance, seal, pressure
`mechanical`	12	mass scale, centre-of-mass height
`nominal`	2	none (factors at nominal)
Total	118	
`train` split	102	further split 82 fit / 20 validation
`test` split	16	held out

The split is at *run* level rather than window level, and this is the single most important property of the collection. Within one run the factor vector is constant, so windows drawn from the same run are highly redundant; a window-level split would place near-duplicates of the same plant on both sides of the boundary and every held-out score in this thesis would be optimistic. The manifest stores the factor key of every run alongside its split label, so this property is checkable rather than asserted.

C. Model Architectures, Normalizers and Checkpoint Hashes

This appendix pins the frozen artifacts. Every model-level and closed-loop result in this thesis was produced with the exact files listed in Table C.1, and the SHA-256 digests are given so that a later reader can establish whether a file on disk is the one that produced a number here. This matters more than usual in this project, because resumed training runs overwrite checkpoints in place.

Table C.1: Frozen artifacts and their SHA-256 digests (first 32 hex digits shown). "Frozen" means the file's weights were not updated by any subsequent stage; the three policy checkpoints are the ones evaluated in Chapter 7.

Artifact	Role	SHA-256 (truncated)
`aam.pt`	AAM, frozen	`cd6a7f0b118b96c7ce66b889b9b3e422`
`zhat_adapter.pt`	ID adapter, frozen	`30629a4f83db5c2bed3c777bcf82dc57`
`best_id_cylinder_control.pt`	inverse model, frozen	`8e26c52e82d89efe193e001d0a8b07a6`
`best_fd_cylinder_strict.pt`	forward model, frozen	`1f41e44309a8f320430a3382ca88ce5f`
`headroom_priced_v1/model_299.pt`	AIF policy	`aaf6b4bab7cfeac53fb95a3d15470cde`
`rom_s6_aligned/model_598.pt`	R policy	`ce7e3ab9649b1193f123f9f91107bbf8`
`idfd_s6_aligned/model_598.pt`	IF policy	`06533b8e8614b0ad982dd8c238a17772`

C.1 The Inverse and Forward Models

Both learned actuator models share one architecture: a linear input projection to 512 units, four pre-norm residual blocks of two 512-wide linear layers each, a final normalisation and a linear output projection. They differ only in their input and output layouts, given in Table C.2. Each carries just over 2.12 million parameters. Both are trained on the Simscape campaign, both operate on the 50 ms grid, and both are frozen for every result in Chapters 6 and 7.

Only the predicted extension increment of the forward model is integrated in the control loop. Its velocity output failed validation on two axes (Section 5.3) and is carried as a diagnostic rather than used.

Table C.2: Input and output layouts of the two frozen actuator models, read from the `input_layout` and `output_layout` fields stored in the checkpoints themselves.

	Inverse model	Forward model (strict)
Input width	28	20
Input layout	extension(4), velocity(4), $\Delta p(4)$, force(4), target Δextension(4), target Δvelocity(4), $\Delta t(4)$	valve(4), extension(4), velocity(4), $\Delta p(4)$, force(4)
Output width	4	12
Output layout	valve command(4)	Δextension(4), Δvelocity(4), $\Delta(\Delta p)(4)$
Target mode	normalised Δ valve command	normalised Δ cylinder state
Parameters	2 124 292	2 124 300

C.2 The Adaptation Module

The AAM is a causal one-dimensional convolutional network over the last 20 rows of the 50 ms deployment observable. The input row is 18 columns wide,

$$h_t = \Big[\, q(5),\ \dot{q}(5),\ \Delta p(4),\ u_{t-1}(4) \,\Big],$$

and the trunk is four convolutional stages of width 64 and kernel three, the first carrying a 1×1 skip projection from 18 to 64 channels, each stage normalised. A two-layer head maps the final 64-channel activation to the 2-dimensional latent $\hat{z}$. The whole module is 42 562 parameters, which is what makes 20 Hz inference affordable alongside 256 parallel environments.

The latent dimension is 2 because the module regresses onto the frozen state predictor's latent, which is 2-dimensional by construction. This is the structural reason the deployed latent decodes only one of the six physical factors, and Section 7.5.3 treats it as a finding rather than a configuration choice.

The ID adapter is deliberately small: a two-layer network of width 64, mapping a 30-dimensional feature row to a 4-dimensional residual, 76 stored tensors' worth of normalisation statistics included. Its feature row is the inverse model's 28 inputs concatenated with the 2 latent components, which is the pressure-inclusive contract discussed in Section 7.9. Its output is bounded at 0.15 per cylinder (Table A.2) and scaled by a declared residual factor, 0.25 for every result reported here.

The adapter checkpoint additionally stores the digests of the inverse and forward models it was fitted against, so a latent adapter cannot silently be paired with a different frozen actuator than the one it was trained for.

D. PPO and IsaacLab Configuration

D.1 The Observation Vector

The policy observation is 66 columns for R and IF, and 68 for AIF: the same 66 with the two latent components appended. Table D.1 gives the layout by index. The final twelve columns are the model-error block, and they exist because the policy is a memoryless multilayer perceptron: it already observes the actuator's prediction and the current measured extension, but forming the error between them requires differencing across two steps, which a feedforward network cannot do. Without these columns the standing model error of a learned actuator is invisible to the policy no matter how long it trains.

Channel 46 caused a campaign-scale failure. It carries truck speed, and with a frozen observation normaliser inherited from a fixed-speed run its recorded standard deviation was zero. The normaliser divides by that standard deviation plus a small constant, which multiplied the channel by roughly one hundred everywhere except at the speed the statistics were collected at. The policy scored perfectly at that one speed and collapsed at every other. Freezing a normaliser is correct when resuming; freezing one whose statistics were gathered under a narrower distribution than the one about to be trained on is not.

D.2 The Action

The action is four-dimensional in every stack: a desired cylinder motion, not a valve command. What converts it to a valve command is the actuator under test, which is the only free variable in the three-way comparison. The limits enforced between the action and the plant are given in Table A.2.

D.3 The Reward

The ten reward terms and their shaping functions are given in Table 6.2 and are not repeated here. Two weights behind that table are the ones a reproduction is most likely to get wrong: contact loss is weighted at 20.0 and hard impacts at 20.0, against a support reward of 4.0. The hard-impact weight was introduced after the measurement described

Table D.1: Policy observation layout. Indices are zero-based and half-open. Columns 0–53 are the task block, 54–65 the model-error block, and 66–67 exist only for AIF.

Index	Width	Contents
0–9	9	verge height profile ahead of the cutter
9	1	cut-height error
10	1	lateral error
11	1	cutter vertical velocity
12–16	4	measured cylinder extensions
16–20	4	measured cylinder velocities
20–24	4	cylinder headroom, low side (normalised)
24–28	4	cylinder headroom, high side (normalised)
28–32	4	previous applied action
32–36	4	current valve commands
36–40	4	actuator-predicted Δextension
40–44	4	commanded target lead
44	1	head-down alignment
45	1	head-down alignment rate
46	1	truck speed
47	1	scan valid fraction
48–50	2	world-XY components of the cutter's local $+Z$ axis
50	1	normalised cutter–terrain interaction force
51–54	3	path preview at 0.5, 1.0, 2.0 m ahead
54–58	4	cylinder tracking residual (measured − commanded)
58–62	4	actuator prediction residual
62–66	4	normalised cylinder effort
66–68	2	$\hat{z}$, AIF only

in Section 3.9.1: the quantity was gated but not priced, and the optimizer traded it away while its return rose.

D.4 PPO Hyperparameters

Table D.2 lists the optimisation settings of the three evaluated runs, read from their stored run manifests rather than from the scripts that launched them. The manifests share a schema, which is what makes the comparison in Section 7.1 checkable; the fields on which the runs differ are exactly those discussed there.

Table D.2: PPO configuration of the three evaluated runs, from `run_manifest.json`. A dash marks a field the manifest does not record. Differences between the adaptive run and the matched pair are discussed in Section 7.1.

	R	IF	AIF
manifest schema		`terrain_shared_v2`	
seed	17	17	41
terrain base seed	17	17	41
environments	256	256	256
curriculum stage	6	6	6
actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-6}
entropy coefficient	0.001	0.001	0.001
frozen-actor warm-up	150	150	0
observation normaliser	freeze	freeze	freeze
observation width	66	66	68
discount γ	—	—	0.97
GAE λ	—	—	0.95
steps per environment	—	—	32
episode length	—	—	47.3 s
truck speed	—	—	2–7 km/h, per episode
residual scale	—	—	0.25
trajectory families		all 11	
actor parameters	—	—	59 080
critic parameters	—	—	58 881

The dashes are informative and are left visible rather than filled in from the launch scripts. The two reference runs' manifests do not record their truck speed schedule, discount or GAE settings, so those cannot be verified from the frozen record; the adaptive run's manifest does record them. Section 7.1 states the consequence.

E. Supplementary Results

E.1 Per-Family Results on the Reference Mesh

Table E.1 gives the per-family breakdown behind the pooled means of Table 7.4, on mesh 1337, for the adaptive stack. It is included because the pooled statistics of Chapter 7 are carried by a small number of cells, and the reader should be able to see which.

Table E.1: AIF per-family results on mesh 1337, full 26 m path at 4 km/h, checkpoint `headroom_priced_v1/299`. Families are ordered by lateral error. Nine of eleven sit near 0.03 m; the distribution is not approximately uniform and its mean is not a typical value.

Family	lateral (m)	cut height (m)	ground contact
`s_curve`	0.0288	0.0014	0.9768
`lateral_sine`	0.0293	0.0012	0.9754
`straight`	0.0309	0.0015	0.9694
`lane_change_hold`	0.0312	0.0026	0.9629
`gaussian_slalom`	0.0313	0.0015	0.9701
`random_spline`	0.0315	0.0020	0.9622
`boom_sweep`	0.0318	0.0016	0.9729
`fourier_meander`	0.0324	0.0013	0.9786
`lateral_chirp`	0.0342	0.0032	0.9615
`double_s_curve`	0.0951	0.0021	0.9772
`offset_return`	**0.2240**	**0.0532**	**0.8485**

Two features of Table E.1 govern how Chapter 7 should be read. The first is the gap between the ninth and eleventh rows: a factor of seven in lateral error separates the typical family from the worst one, so the worst-family aggregation used by the acceptance gates is a different measurement from the mean, not a conservative version of it. The second is that `offset_return` is the family that fails, and it fails on all three metrics at once. Lateral error, cut height and ground contact degrade together, which is the signature of the cutter leaving the ground rather than of a tracking error alone.

E.2 The Latent Substitution Arms

The four latent conditions of Section 7.6.6 share one checkpoint and one set of weights, and differ only in what occupies the latent slot. This is the comparison that carries the most weight in this thesis, because it is immune to the training-condition asymmetries of Section 7.1: no training differs between the arms, since no training occurs.

Table E.2: The four latent arms, as evaluated. All four use checkpoint `headroom_priced_v1/299` with identical weights; only the value supplied in the latent slot changes.

Arm	Latent supplied
`estimated`	the deployed AAM estimate, updated at 20 Hz
`true`	privileged encoding of the run's actual factor vector
`mean`	a constant carrying no plant information
`shuffled`	a constant belonging to a *different* plant

The ordering of these four arms is the mechanism result of this thesis. The `shuffled` arm is the control that distinguishes the two candidate explanations: if the deployed estimate were losing because it carries the wrong plant identity, then `shuffled`, which is by definition wrong about the plant, would lose by at least as much. It does not. What separates the estimate from both constants is not its information content but its movement.

E.3 Reproduction

Every closed-loop number in Chapter 7 is produced by the experiment scripts stored with the project under `experiments/E1` through `experiments/E5`, each of which records the checkpoint, the mesh set, the family set, the speed and the scored horizon in its own manifest. The model-level results come from the frozen collection of Appendix B.4. The artifacts named in Table C.1 are sufficient, together with those scripts, to regenerate every table in this thesis.

Bibliography

[Åström 95] K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Reading, MA: Addison-Wesley, 1995.

[Ba 16] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization", arXiv preprint arXiv:1607.06450, 2016.

[Bengio 15] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. 2015, pp. 1171–1179.

[Benner 15] P. Benner, S. Gugercin, K. Willcox, "A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems", *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.

[Cho 14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[Chua 18] K. Chua, R. Calandra, R. McAllister, S. Levine, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31. 2018.

[Egli 20] P. Egli, M. Hutter, "Towards RL-Based Hydraulic Excavator Automation", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2692–2697.

[Egli 22] P. Egli, M. Hutter, "A General Approach for the Automation of Hydraulic Excavator Arms Using Reinforcement Learning", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5679–5686, 2022.

[Elfwing 18] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", *Neural Networks*, vol. 107, pp. 3–11, 2018.

[He 16] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[Hochreiter 97] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[Hu 26] P. Hu, T. Wen, D. Zhang, H. Chen, J. Gong, "Bayesian reinforcement learning for adaptive control of energy recuperation in hydraulic excavator arms", *Scientific Reports*, vol. 16, p. 6195, 2026.

[Huber 64] P. J. Huber, "Robust Estimation of a Location Parameter", *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[Hwangbo 19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[Isermann 11] R. Isermann, M. Münchhof, *Identification of Dynamic Systems: An Introduction with Applications*, Berlin, Heidelberg: Springer, 2011.

[Jelali 03] M. Jelali, A. Kroll, *Hydraulic Servo-systems: Modelling, Identification and Control*, London: Springer, 2003.

[Johannink 19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, S. Levine, "Residual Reinforcement Learning for Robot Control", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.

[Kakade 02] S. Kakade, J. Langford, "Approximately Optimal Approximate Reinforcement Learning", in *Proceedings of the 19th International Conference on Machine Learning (ICML)*. 2002, pp. 267–274.

[Karniadakis 21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, "Physics-Informed Machine Learning", *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[Kingma 15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", in *International Conference on Learning Representations (ICLR)*. 2015.

[Kumar 21] A. Kumar, Z. Fu, D. Pathak, J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots", in *Proceedings of Robotics: Science and Systems (RSS)*. 2021.

[Lee 20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning Quadrupedal Locomotion over Challenging Terrain", *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[Liu 25] X. Liu, C. Lu, H. Wu, B. Hu, X. Li, Z. Li, X. Guo, "FRMA: Four-Phase Rapid Motor Adaptation Framework", *Machines*, vol. 13, no. 10, p. 885, 2025.

[Ljung 99] L. Ljung, *System Identification: Theory for the User*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 1999.

[Lutter 19] M. Lutter, C. Ritter, J. Peters, "Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning", in *International Conference on Learning Representations (ICLR)*. 2019.

[Maitan 26] L. Maitan, L. Toschi, C. Zanette, E. G. Vergamini, L. F. Santos, T. Boaventura, "Contraction Metric Based Safe Reinforcement Learning Force Control for a Hydraulic Actuator with Real-World Training", arXiv preprint arXiv:2602.08977, 2026.

[Makoviychuk 21] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning", arXiv preprint arXiv:2108.10470, 2021.

[Mattila 17] J. Mattila, J. Koivumäki, D. G. Caldwell, C. Semini, "A Survey on Control of Hydraulic Robotic Manipulators With Projection to Future Trends", *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 2, pp. 669–680, 2017.

[McKay 79] M. D. McKay, R. J. Beckman, W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code", *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[Merritt 67] H. E. Merritt, *Hydraulic Control Systems*, New York: John Wiley & Sons, 1967.

[Mittal 23] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments", *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[MULAG Fahrzeugwerk 26] MULAG Fahrzeugwerk, "FME 500 Front Mower — Product Documentation", Heinz Wössner GmbH u. Co. KG, Oppenau, Germany. `https://www.mulag.de/en/roadside-equipment/products/unimog-equipment/front-mower/fme-500/`, 2026. Accessed July 2026.

[Nagabandi 18] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 7559–7566.

[Nakamura 86] Y. Nakamura, H. Hanafusa, "Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control", *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[Nguyen-Tuong 11] D. Nguyen-Tuong, J. Peters, "Model Learning for Robot Control: A Survey", *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, 2011.

[Paszke 19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. 2019, pp. 8024–8035.

[Peng 18] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810.

[Rajeswaran 18] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, "Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations", in *Proceedings of Robotics: Science and Systems (RSS)*. 2018.

[Ross 11] S. Ross, G. J. Gordon, J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning", in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 627–635.

[Rudin 22] N. Rudin, D. Hoeller, P. Reist, M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", in *Proceedings of the Conference on Robot Learning (CoRL)*. 2022, pp. 91–100.

[Schoukens 19] J. Schoukens, L. Ljung, "Nonlinear System Identification: A User-Oriented Road Map", *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 28–99, 2019.

[Schulman 15] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, P. Abbeel, "Trust Region Policy Optimization", in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 37. 2015, pp. 1889–1897.

[Schulman 16] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation", in *International Conference on Learning Representations (ICLR)*. 2016.

[Schulman 17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv preprint arXiv:1707.06347, 2017.

[Siciliano 09] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, London: Springer, 2009.

[Silver 18] T. Silver, K. Allen, J. Tenenbaum, L. Kaelbling, "Residual Policy Learning", arXiv preprint arXiv:1812.06298, 2018.

[Slotine 91] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Englewood Cliffs, NJ: Prentice Hall, 1991.

[Sobol' 67] I. M. Sobol', "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals", *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[Sofrony 24] J. Sofrony, M. C. Turner, C. M. Richards, "Anti-windup compensation for model reference adaptive control schemes", *International Journal of Robust and Nonlinear Control*, vol. 34, no. 15, pp. 10176–10193, 2024.

[Spinelli 24] F. A. Spinelli, P. Egli, J. Nubert, F. Nan, T. Bleumer, P. Goegler, S. Brockes, F. Hofmann, M. Hutter, "Reinforcement Learning Control for Autonomous Hydraulic Material Handling Machines with Underactuated Tools", in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. arXiv:2410.05093.

[Srivastava 14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[Sutton 18] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.

[The MathWorks, Inc. 24a] The MathWorks, Inc , "Simscape Fluids: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-fluids.html`.

[The MathWorks, Inc. 24b] The MathWorks, Inc , "Simscape Multibody: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-multibody.html`.

[Tobin 17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[von Szadkowski 20] K. von Szadkowski, S. Reichel, "Phobos: A Tool for Creating Complex Robot Models", *Journal of Open Source Software*, vol. 5, no. 45, p. 1326, 2020.

[Wampler 86] C. W. Wampler, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods", *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[Yamamori 26] S. Yamamori, K. Ishihara, K. Minamikawa, R. Ohmori, T. Yasaki, N. Sugimoto, J. Morimoto, "Actuator Reality Shaping for Zero-Shot Sim-to-Real Robot Learning", arXiv preprint arXiv:2607.02205, 2026.

[Yang 26] N. Yang, C. Ji, J. Chen, H. Zheng, "State-Constrained Control for Hydraulic Manipulator Position Servo Systems with Valve Dead-Band Compensation", *Actuators*, vol. 15, no. 4, p. 196, 2026.

[Yin 25] P. Yin, T. Westenbroek, S. Bagaria, K. Huang, C.-A. Cheng, A. Kolobov, A. Gupta, "Rapidly Adapting Policies to the Real World via Simulation-Guided Fine-Tuning", arXiv preprint arXiv:2502.02705, 2025.

[Yu 26] S.-Y. Yu, J.-W. Wang, M. Horii, M. Tomizuka, V. Garg, "Learning Context-Aware Neural ODE Dynamics for Adaptive Robotic Control", arXiv preprint arXiv:2606.15469, 2026.